



FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY

BMCS3183 ADVANCED DATABASE MANAGEMENT

Assignment

Semester 202505

Programme (Year, Sem & Group)	:	e.g. RDS3S1G4
Date Submitted	:	e.g. 07/09/2025

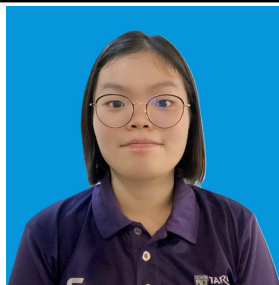
Team members:

No	Name (Block Letters - Alphabetical Order)	Student ID	Marks
1	Chan Jin Wei	24WMR08851	
2	Ethel Ng Yi Yan	24WMR08861	
3	Gan Khai Li	24WMR08863	
4	Kit Chin Jie Ying	24WMR08870	

Declaration

We confirm that we have read and shall comply with all the terms and conditions of TAR UMT's plagiarism policy.

We declare that this assignment is free from all forms of plagiarism and for all intents and purposes is my own properly derived work.

			
Name: Chan Jin Wei	Name: Ethel Ng Yi Yan	Name: Gan Khai Li	Name: Kit Chin Jie Ying
Signature: <i>Chan</i>	Signature: <i>Ethel</i>	Signature: <i>Gan</i>	Signature: <i>Kit</i>
Date: 07/09/2025	Date: 07/09/2025	Date: 07/09/2025	Date: 07/09/2025

Assignment Assessment Form

No	Member Name (Alphabetical order)	Programme (Year-Semester-Group)
1.	Chan Jin Wei	RDS(Y3-S1-G4)
2.	Ethel Ng Yi Yan	RDS(Y3-S1-G4)
3.	Gan Khai Li	RDS(Y3-S1-G4)
4.	Kit Chin Jie Ying	RDS(Y3-S1-G4)
5.		

Task No.	Task Descriptions	Weightage	Criteria	1	2	3	4	5	Comment
1 (CLO 1)	Entity Relationship Diagram	10%	- A complete ER data model in 3rd Normal Form. - All primary keys, foreign keys, relationships and attributes must be clearly shown. - Standalone system design will be given lower marks compare to online system design.						
2 (CLO 1)	Data Definition (DDL)	10%	- Relevant integrity constraints to ensure database integrity must be included. - Necessary check constraints and default values to enforce business rules should also be included.						
3 (CLO 1)	Data records	5%	Sufficient quality data records must be created for each table.						
4 (CLO 3)	Queries	5%	- Each team member is to design and produce two quality and useful queries for decision making at any two different management levels: strategic, tactical or operational. - Single table queries are not allowed. - Multiple table queries with aggregate functions (where appropriate) must be used. - View(s) is/are ought to be incorporated, where necessary. - These 2 queries cannot be used directly for report body.						
		5%							
5 (CLO 2)	Stored Procedures	5%	- Each team member is to design and create two stored procedures that cater for the use case scenarios for the system. - Quality, usefulness and importance of the stored procedures must be considered.						
		5%							

6 (CLO 2)	Triggers	5%	- Each team member is to design and create two triggers that enforces system-wide business rules and policies. - Quality, usefulness and functionality of the triggers must be considered.						
		5%							
7 (CLO 3)	Reports	10%	- Each team member is to create two procedures to generate two reports (summary, detail and on demand basis reports) for the company. - Parameter value(s) should be passed to the procedure, where necessary. - Cursor must be used in report generation (prefer nested cursors or multiple cursors). - Usefulness, complexity and presentation of the reports must be considered.						
		10%							
8 (CLO 2)	Extra Effort	10%	- Sequences, indexes, views, output formatting, exceptions and/or functions, must be incorporated where necessary. - Usefulness and application of each of the above to enhance the efficiency and effectiveness of the information system must be considered. - Linking of all the tasks of every team member in creating a quality information system must be considered.						
9 (CLO 3)	Presentation & Participation	15%	- Run the single script file from Task 2 – 3 to create the new system database on the lab server. - Individual presentation on Task 4 – 8. - Q & A - Actively participate in class discussion.						
Assignment Marks / 100									

*CLO 1: Develop the relational database system with the appropriate integrity constraints and security control. (P3, PLO3)

*CLO 2: Design the solutions to issues pertaining to database efficiency and effectiveness using appropriate techniques. (C4, PLO2)

*CLO 3: Extract information from the database using efficient SQL query construct. (C4, PLO6)

Table of Content

Chapter 1 Background of the System	8
1.1 Type of System	8
1.2 List of Modules	8
Chapter 2 Entity-Relationship Modeling	10
2.1 Business Rules and Assumptions	10
2.1.1 Business Rules	10
2.1.2 Assumptions	11
2.2 Entity Relationship Diagram (ERD) ADM ERD	12
Chapter 3 Data Definition	12
3.1 CREATE TABLE Statements	12
3.1.1 Parent Table	12
3.1.1.1 Staff Table	12
3.1.1.2 Shop Table	13
3.1.1.3 Gift Table	13
3.1.1.4 Payment Table	13
3.1.1.5 Bus Company Table	13
3.1.1.6 Service Table	13
3.1.1.7 Extension Table	14
3.1.1.8 Refund Table	14
3.1.2 Child Table	14
3.1.2.1 Member Table	14
3.1.2.2 Booking Table	14
3.1.2.3 Bus Table	15
3.1.2.4 Bus Schedule Table	15
3.1.2.5 Driver Table	15
3.1.2.6 Ticket Table	15
3.1.3 Associative Table	16
3.1.3.1 Staff Allocation Table	16
3.1.3.2 Driver List Table	16
3.1.3.3 Booking Details Table	16
3.1.3.4 Service Details Table	16
3.1.3.5 Request Complaint Table	17
3.1.3.6 Point Redemption Table	17
3.1.3.7 Rental Collection Table	17
3.2 Sample Records (10 sample Records for each table)	18
3.2.1 Parent Table Record	18
3.2.1.1 Staff Record	18
3.2.1.2 Shop Record	18
3.2.1.3 Gift Record	18
3.2.1.4 Payment Record	19
3.2.1.5 Bus Company Record	19
3.2.1.6 Service Record	20
3.2.1.7 Extension Record	20

3.2.1.8 Refund Record	20
3.2.2 Child Table Record	21
3.2.2.0 : Member Record	21
3.2.2.1 Booking Record	21
3.2.2.2 Bus Record	22
3.2.2.3 Bus Schedule Record	22
3.2.2.4 Driver Record	23
3.2.2.5 Ticket Record	23
3.2.3 Associative Table Record	24
3.2.3.1 Staff Allocation Record	24
3.2.3.2 Driver List Record	24
3.2.3.3 Booking Details Record	24
3.2.3.4 Service Details Record	24
3.2.3.5 Request Complaint Record	25
3.2.3.6 Point Redemption Record	25
3.2.3.7 Rental Collection Record	26
Chapter 4 Queries, Procedures, Triggers and Reports	27
4.1 Chan Jin Wei	27
4.1.1 Query 1: View Bus Schedule by Bus Company	27
4.1.2 Query 2: Bus Maintenance Records by Company	28
4.1.3 Procedure 1: Ticket Booking	29
4.1.4 Procedure 2: Staff Allocation	32
4.1.5 Trigger 1: Calculation of memberpoint when redeeming gift	33
4.1.6 Trigger 2: Gift Redemption Log	35
4.1.7 Report 1: Yearly Revenue Trend Report	36
4.1.8 Report 2: Maintenance Cost vs Ticket Revenue Report	39
4.2 Ethel Ng Yi Yan	42
4.2.1 Query 1: Query of Booking Details for Each Booking	42
4.2.2 Query 2: Query for Distribution of Driver Allocation	44
4.2.3 Procedure 1: Advanced Search for Bus Schedule	46
4.2.4 Procedure 2: Procedure of Full Refund of Bus Schedule Cancellation	49
4.2.5 Trigger 1: Calculation of Refund Amount and Update Booking Refund ID	52
4.2.6 Trigger 2: Audit Trail For Booking Table	54
4.2.7 Report 1: Bus Utilisation Efficiency Report	56
4.2.8 Report 2: Report of Bus Route and Revenue	60
4.3 Gan Khai Li	64
4.3.1 Query 1: Most Frequent Destinations by Member	64
4.3.2 Query 2: Top 5 Redeemed Gifts with Recent Member Activity	73
4.3.3 Procedure 1: Manage Bus Maintenance	75
4.3.4 Procedure 2: Member Registration	77
4.3.5 Trigger 1: Check Extension Timing and Update Booking	80
4.3.6 Trigger 2: Audit Trail for RequestComplain Table	81
4.3.7 Report 1: Most Popular vs Least Popular Destinations Report	83
4.3.8 Report 2: Overall Refund Approval vs Rejection Report by Reason	85

4.4 Kit Chin Jie Ying	88
4.4.1 Query 1: Refund History of Member	88
4.4.2 Query 2: Top 10 Members Purchasing Tickets	90
4.4.3 Procedure 1: Ticket Generation for Bus Scheduled	91
4.4.4 Procedure 2: Manage Shop Rental	96
4.4.5 Trigger 1: Booking Total and Seat Availability Management	99
4.4.6 Trigger 2: Bus Maintenance Audit Logger	102
4.4.7 Report 1: Driver Workload Report	103
4.4.8 Report 2: Revenue Earned vs Lost per Company	109
Chapter 5 Extra Effort Highlights	113
5.1 Chan Jin Wei	113
5.1.1 View	113
5.1.1.1 View Used in Query 1	113
5.1.1.2 View Used in Query 2	113
5.1.2 Indexes	114
5.1.2.1 Index for CompanyID in Bus Table (Used in Report 1)	114
5.1.2.2 Index for PlateNumber in Bus Table (Used in Report 2)	114
5.1.3 User Defined Functions	114
5.1.3.1 Function Used in Trigger 1	114
5.1.4 User Defined Exceptions	115
5.1.4.1 RAISE_APPLICATION_ERROR in Trigger 1	115
5.1.4.2 Default Exception in Procedure 2	116
5.1.5 Sequence	116
5.1.5.1 Sequence Used in Procedure 1	116
5.2 Ethel Ng Yi Yan	117
5.2.1 View	117
5.2.1.1 View Used in Query 1	117
5.2.1.2 View Used in Query 2	117
5.2.2 Indexes	118
5.2.2.1 Index for BusScheduleID, TicketStatus in Ticket Table (Used in Report 1)	118
5.2.2.2 Index for DepartBusStation, Destination in BusSchedule Table (Used in Report 2)	118
5.2.4 User Defined Exceptions	118
5.2.4.1 RAISE_APPLICATION_ERROR in Trigger 1	118
5.2.4.2 DEFAULT EXCEPTION in Procedure 1	119
5.2.5 Sequence	119
5.2.5.1 Sequence Used in Procedure 2 : Full Refund of Bus Schedule Cancellation	119
5.3 Gan Khai Li	120
5.3.1 View	120
5.3.1.1 View Used in Query 1	120
5.3.1.2 View Used in Query 2	120
5.3.2 Indexes	121
5.3.2.1 Index for Destination in BusSchedule Table (Used in Report 1)	121

5.3.2.2 Index for RefundStatus, RefundReason in Refund Table (Used in Report 2)	121
5.3.3 User Defined Functions	121
5.3.3.1 Function Used in Procedure 2	121
5.3.4 User Defined Exceptions	122
5.3.4.1 PRAGMA EXCEPTION_INIT Used in Procedure 1	122
5.3.4.2 RAISE_APPLICATION_ERROR Exception Used in Procedure 2	122
5.3.5 Sequence	123
5.3.5.1 ServiceID Sequence Used in Procedure 1	123
5.4 Kit Chin Jie Ying	124
5.4.1 View	124
5.4.1.1 View Used in Query 1	124
5.4.1.2 View Used in Query 2	124
5.4.2 Indexes	125
5.4.2.1 Index idx_bs_plate_dep on BusSchedule (Used in Report 1)	125
5.4.2.2 Index idx_booking_yrmon on Booking (Used in Report 2)	125
5.4.3 User Defined Functions	125
5.4.3.1 Function of Rental Fee Calculation	125
5.4.4 User Defined Exceptions	126
5.4.4.1 RAISE_APPLICATION_ERROR	126
5.4.4.2 PRAGMA EXCEPTION_INIT	126
5.4.5 Sequence	127
5.4.5.1 Shop Sequence Used in Procedure 2	127

Chapter 1 Background of the System

1.1 Type of System

The Bus Station Management System is an centralised online system designed to enhance the efficiency, transparency and user experience of managing bus station operations. It provides a wide range of functionalities for staff, members and bus operators, which ensures a smooth ticket booking, scheduling and service management.

The Bus Station Management System also includes a comprehensive member point redemption feature designed to enhance customer engagement and reward loyalty. Members are able to accumulate points automatically with every successful booking or purchase made through the platform. These points can be tracked in the user's account and redeemed for various rewards, such as discounted tickets, free upgrades or exclusive gifts offered through the system. This feature not only incentivizes repeat usage but also builds long-term customer satisfaction and loyalty, encouraging members to choose the platform for future travel needs.

In conclusion, the Bus Station Management System serves as a powerful all-in-one platform that streamlines operations, improves service quality and enhances the overall experience for both passengers and staff. By integrating essential features such as ticketing, scheduling, customer support and loyalty rewards, the system ensures efficient, user-friendly and future-ready bus station management.

1.2 List of Modules

1. Membership Registration Module

The Membership Registration Module allows users to sign up as members of the bus station management system by completing a registration form and paying a one-time registration fee of RM10. Once the payment is processed through the Payment Processing Module, a member profile is created in the system. With an active membership, users gain access to exclusive features such as online ticket booking and reward redemptions.

2. Member Management Module

The Member Management Module handles all member-related data. This includes viewing and updating member information, tracking whether the membership fee has been paid and linking each member to their ticket bookings and redemption activities. It also enforces membership rules, ensuring that only active status members can access certain services like booking or redeeming.

3. Ticket Booking and Management Module

The Ticket Booking and Management Module enables members to search for available buses and schedules, book seats and pay for their tickets online. It maintains a record of each member's past and upcoming bookings and integrates tightly with the Payment Processing Module to manage the financial aspects of the booking process.

4. Refund and Extension Module

The Refund and Extension Module manages refund and ticket extension requests. It applies specific rules: cancellations are only permitted if made at least two days before the travel date, with a 70% refund issued; extensions are also allowed within the same time frame but require an additional RM5 charge. If a bus schedule is cancelled by a bus company, affected tickets are eligible for a full refund. This module calculates refund or extension charges, updates ticket validity and communicates changes to members.

5. Bus Schedule Module

The Bus Schedule Module is responsible for managing all bus schedules, including creating, editing or deleting routes and time slots. It also handles the assignment of drivers and buses to schedules and ensures real-time schedule updates are available for users. This module supports advanced search features to allow users to find schedules based on various filters.

6. Staff and Shop Module

The Staff and Shop Module oversees the management of terminal personnel and commercial tenants. It registers different staff roles such as counter clerks and cleaners, assigns their work schedules and maintains staff records. It also manages food court stalls and other shop lots by handling shop registrations, rental fees and shop assignments.

7. Bus Company and Bus Driver Module

The Bus Company and Bus Driver Module maintains profiles of all bus companies and their employed drivers. It tracks important company details and links each driver to their company, along with recording driver credentials and assigned schedules. This ensures transparency and accountability in bus operations.

8. Maintenance and Service Module

The Maintenance and Service Module logs all maintenance activities related to buses. This includes scheduling and recording services such as repairs, tyre replacements and washing. Buses undergoing maintenance are marked as unavailable for scheduling to avoid disruptions in service and all service history is retained for administrative review.

9. Member Redemption Module

The Member Redemption Module allows members to redeem available rewards, which may be based on accumulated points or promotional offers. The system tracks each member's redemption history and available points or credits, provides a list of redeemable items or services and deducts the necessary value upon successful redemption.

10. Request and Complaint Module

The Request and Complaint Module is designed to handle customer service interactions efficiently. Members can submit requests, inquiries or complaints, which are then routed to the appropriate department for resolution. The system tracks each case, maintains its status and notifies users when updates or responses are available.

11. Payment Processing Module

The Payment Processing Module handles all monetary transactions within the system. This includes payments for membership registration, ticket bookings, ticket extensions and shop rentals, as well as refunds. It provides a secure method for processing payments and maintains a transaction history that can be accessed by both members and administrators.

12. Advanced Search Module

The Advanced Search Module enhances the user experience by allowing complex search queries across the bus schedule database. Users can search by bus ID, bus company, destination, ticket price, date, departure or arrival time and optionally by driver or platform. This improves navigation and allows members to find and book suitable trips more efficiently.

Chapter 2 Entity-Relationship Modeling

2.1 Business Rules and Assumptions

2.1.1 Business Rules

1. A staff can serve one or many members. A member can be served by one or many staff. (Many to Many)
2. A staff can handle one or many shops. Each shop can be handled by one or many staff. (Many to Many)
3. A rental collection can only have one and only one payment process. One payment process can only have one and only one rental collection. (One to One)
4. A registered member can redeem one or many gifts. A gift can be redeemed by one or many registered members. (Many to Many)
5. A member can have one or many bookings. A booking can only belong to one and only one member. (One to Many)
6. A member can only have one payment for the registration fee. One payment process can only belong to one member.
7. A booking has only one and only one payment. A payment can only belong to one and only one booking. (One to One)
8. A booking can have one or many tickets. A ticket can belongs to one or many booking(Many to Many)
9. A booking details can have zero or one extension. An extension can only belong to one and only one booking details. (One to Zero or One)
10. A booking details can have zero or one refund. A refund can only belong to one and only one booking details. (One to Zero or One)

11. A bus schedule can have one or many tickets. A ticket can only belong to one and only one bus schedule (One to Many).
12. A bus schedule can have one or many drivers. A driver can be included in one or many bus schedules. (Many to Many)
13. A bus company can have one or many bus drivers. A bus driver can only belong to one and only one bus company. (One to Many)
14. A bus can have one to many bus schedules. A bus schedule can only belong to one and only one bus. (One to Many)
15. A bus company can have one to many buses. A bus can only belong to one and only one bus company. (One to Many)
16. A bus can have one or many service details. A service can have one or many buses. (Many to Many)
17. A staff can be allocated to one or many services. A service can have one or many staff allocated. (Many to Many)
18. The registration fee of RM10 is mandatory and must be paid before a member can access ticket booking or other member-exclusive features.
19. Cancellation requests must be made at least 48 hours prior to the scheduled departure time to qualify for a 70% refund.
20. No cancellations or refunds are allowed less than 2 days before departure, except for bus schedule cancellations by the bus company.
21. Cancellations of bus schedules by the bus company will be given a 100% refund.
22. Each gift must have a predefined point value that determines the number of points required for redemption.
23. A points redemption transaction will deduct points immediately from the member's account upon successful redemption.
24. Each bus must have a capacity limit and bookings cannot exceed this limit. (seats available depending on bus type)
25. Business operation hours for on-site/walk-in services are from 9:00 am to 9:00 pm daily.
26. Online system services (ticket booking, membership, payments and redemption) are available 24/7.

2.1.2 Assumptions

1. Only registered and 'active' status members can book tickets online, make a request, complain and redeem points.
2. A registered member with member ID 'M99999' is assumed as all of the walk-in customers.
3. A member with member ID 'M99999' cannot redeem any gift using the point redemption module.
4. Gifts are redeemed on a first-come, first-served basis and are subject to availability.
5. Refunds will be processed within 7 business days after approval.
6. System time is assumed to be based on Malaysia Standard Time (GMT+8).
7. All users are assumed to agree to terms and conditions during registration or before each booking.
8. The member only can redeem one gift in one day.


```

PRIMARY KEY (StaffID)
);

```

3.1.1.2 Shop Table

```

CREATE TABLE Shop(
    ShopID          VARCHAR(6)          NOT NULL,
    ShopName        VARCHAR(100)        NOT NULL,
    ShopType        VARCHAR(100)        NOT NULL,
    OwnerName       VARCHAR(100)        NOT NULL,
    ShopLocation    VARCHAR(255)        NOT NULL,
    SquareFeet      NUMBER(10,2)        CHECK (SquareFeet >= 0) NOT NULL,
    PRIMARY KEY (ShopID)
);

```

3.1.1.3 Gift Table

```

CREATE TABLE Gift(
    GiftID          VARCHAR(6)          NOT NULL,
    GiftName        VARCHAR(100)        NOT NULL,
    PointRequired   NUMBER(6)           CHECK (PointRequired >= 0) NOT NULL,
    StockRemaining  NUMBER(6)           DEFAULT 0 CHECK (StockRemaining >= 0)
NOT NULL,
    GiftStatus      VARCHAR(50)         DEFAULT 'Available' CONSTRAINT
chk_GiftStatus CHECK (GiftStatus IN ('Available', 'Unavailable')) NOT
NULL,
    GiftDesc        VARCHAR(500)        NOT NULL,
    PRIMARY KEY (GiftID)
);

```

3.1.1.4 Payment Table

```

CREATE TABLE Payment(
    PaymentID       VARCHAR(6)          NOT NULL,
    PaymentDate     DATE                NOT NULL,
    PaymentMethod   VARCHAR(50)         NOT NULL,
    PaymentStatus   VARCHAR(50)         DEFAULT 'Pending' CONSTRAINT
chk_PaymentStatus CHECK (PaymentStatus IN ('Pending', 'Completed',
'Failed', 'Refunded' )) NOT NULL,
    BankAccount     VARCHAR(50),
    PRIMARY KEY (PaymentID)
);

```

3.1.1.5 Bus Company Table

```

CREATE TABLE BusCompany(
    CompanyID       VARCHAR(6)          NOT NULL,
    CompanyName     VARCHAR(100)        NOT NULL,
    ContactNo       VARCHAR(15)         NOT NULL,
    CompanyEmail    VARCHAR(100)        NOT NULL,
    CompanyAddress  VARCHAR(255)        NOT NULL,
    PRIMARY KEY (CompanyID)
);

```

3.1.1.6 Service Table

```

CREATE TABLE Service (
    ServiceID       VARCHAR2(6)         NOT NULL,
    ServiceType     VARCHAR2(50)        NOT NULL,
    ServiceCost     NUMBER(10, 2)       NOT NULL
CONSTRAINT chk_ServiceCost CHECK (ServiceCost >= 0),
    PRIMARY KEY (ServiceID)
);

```

```
);
```

3.1.1.7 Extension Table

```
CREATE TABLE Extension (
    ExtensionID      VARCHAR(6)      NOT NULL,
    ExtendedDate     DATE             NOT NULL,
    ExtensionFee      NUMBER(10,2)    CHECK (ExtensionFee >= 0) NOT NULL,
    ExtensionStatus   VARCHAR(50)     DEFAULT 'Requested'
                                CONSTRAINT chk_ExtensionStatus CHECK (ExtensionStatus
IN ('Requested', 'Approved', 'Rejected')) NOT NULL,
    ExtensionReason   VARCHAR(255)    NOT NULL,
    PRIMARY KEY (ExtensionID)
);
```

3.1.1.8 Refund Table

```
CREATE TABLE Refund (
    RefundID         VARCHAR(6)      NOT NULL,
    RefundDate        DATE            NOT NULL,
    RefundReason      VARCHAR(255)    NOT NULL,
    RefundAmount      NUMBER(10,2)    CHECK (RefundAmount >= 0) NOT NULL,
    RefundStatus      VARCHAR(50)     DEFAULT 'Pending'
                                CONSTRAINT chk_RefundStatus CHECK (RefundStatus IN
('Pending', 'Approved', 'Rejected')) NOT NULL,
    PRIMARY KEY (RefundID)
);
```

3.1.2 Child Table

3.1.2.1 Member Table

```
CREATE TABLE Member(
    MemberID         VARCHAR(6)      NOT NULL,
    MemberName       VARCHAR(100)    NOT NULL,
    DateOfBirth      DATE            NOT NULL,
    MemberGender     VARCHAR(10)     CONSTRAINT chk_Gender CHECK
(MemberGender IN ('Male', 'Female', 'Other')) NOT NULL,
    MemberEmail      VARCHAR(100)    NOT NULL,
    MemberPhoneNo    VARCHAR(15)     NOT NULL,
    RegistrationDate DATE            NOT NULL,
    MemberPoints     NUMBER(6)       DEFAULT 0 CHECK (MemberPoints >= 0) NOT
NULL,
    MemberStatus     VARCHAR(15)     DEFAULT 'Active' CONSTRAINT
chk_MemberStatus CHECK (MemberStatus IN ('Active', 'Inactive', 'Banned'))
NOT NULL,
    MemberIC        VARCHAR(14)     NOT NULL,
    PaymentID       VARCHAR(6),
    PRIMARY KEY (MemberID),
    FOREIGN KEY (PaymentID) REFERENCES Payment(PaymentID)
);
```

3.1.2.2 Booking Table

```
CREATE TABLE Booking(
    BookingID        VARCHAR(6)      NOT NULL,
    PurchaseDate     DATE            NOT NULL,
    BookingStatus    VARCHAR(50)     DEFAULT 'Pending' CONSTRAINT
chk_BookingStatus CHECK (BookingStatus IN ('Pending', 'Completed',
'Cancelled')),
    TotalPrice       NUMBER(10,2)    CHECK (TotalPrice >= 0)    NOT NULL,
    MemberID         VARCHAR(6)      NOT NULL,
```

```

PaymentID      VARCHAR(6)      NOT NULL,
PRIMARY KEY (BookingID),
FOREIGN KEY (MemberID) REFERENCES Member (MemberID),
FOREIGN KEY (PaymentID) REFERENCES Payment (PaymentID)
);

```

3.1.2.3 Bus Table

```

CREATE TABLE Bus(
    PlateNumber  VARCHAR(20)    NOT NULL,
    Capacity     NUMBER(4)      CHECK (Capacity > 0)    NOT NULL,
    BusType      VARCHAR(50)    NOT NULL,
    CompanyID    VARCHAR(6)     NOT NULL,
    PRIMARY KEY (PlateNumber),
    FOREIGN KEY (CompanyID) REFERENCES BusCompany (CompanyID)
);

```

3.1.2.4 Bus Schedule Table

```

CREATE TABLE BusSchedule(
    BusScheduleID  VARCHAR(6)      NOT NULL,
    DepartureTime  DATE            NOT NULL,
    ArrivalTime    DATE            NOT NULL,
    DepartureDate  DATE            NOT NULL,
    Destination    VARCHAR(100)    NOT NULL,
    ScheduleStatus VARCHAR(50)     DEFAULT 'Scheduled' CONSTRAINT
chk_ScheduleStatus CHECK (ScheduleStatus IN ('Scheduled', 'Delayed',
'Scheduled', 'Completed')) NOT NULL,
    DepartBusStation VARCHAR(100)  NOT NULL,
    ArrivalBusStation VARCHAR(100) NOT NULL,
    PlatformNo     VARCHAR(10)    NOT NULL,
    AvailableSeat   NUMBER(4)      CHECK (AvailableSeat >= 0)  NOT NULL,
    Distance        NUMBER(8)      NOT NULL,
    PlateNumber     VARCHAR(20)    NOT NULL,
    PRIMARY KEY (BusScheduleID),
    FOREIGN KEY (PlateNumber) REFERENCES Bus (PlateNumber)
);

```

3.1.2.5 Driver Table

```

CREATE TABLE Driver(
    DriverID      VARCHAR(6)      NOT NULL,
    DriverName    VARCHAR(100)    NOT NULL,
    ContactNo     VARCHAR(20)     NOT NULL,
    LicenseNo     VARCHAR(50)     NOT NULL,
    DriverIC      VARCHAR(20)     NOT NULL,
    EmploymentDate DATE          NOT NULL,
    DriverStatus  VARCHAR(50)     DEFAULT 'Active'
CONSTRAINT chk_DriverStatus
CHECK (DriverStatus IN ('Active', 'Inactive',
'Suspended')),
    LicenseExpiryDate DATE        NOT NULL,
    EmploymentPeriod NUMBER(3)    CHECK (EmploymentPeriod >= 0),
    CompanyID    VARCHAR(6)      NOT NULL,
    PRIMARY KEY (DriverID),
    FOREIGN KEY (CompanyID) REFERENCES BusCompany (CompanyID)
);

```

3.1.2.6 Ticket Table

```

CREATE TABLE Ticket(
    TicketID      VARCHAR(6)      NOT NULL,
    TicketPrice   NUMBER(10,2)    NOT NULL CHECK (TicketPrice >= 0),

```

```

        TicketStatus    VARCHAR(50)    DEFAULT 'Available' CONSTRAINT
chk_TicketStatus CHECK (TicketStatus IN ('Available', 'Booked')) NOT
NULL,
        TicketType      VARCHAR(50)    NOT NULL,
        SeatNo          VARCHAR(3)     NOT NULL,
        BusScheduleID   VARCHAR(6)     NOT NULL,
        PRIMARY KEY (TicketID),
        FOREIGN KEY (BusScheduleID) REFERENCES BusSchedule (BusScheduleID)
);

```

3.1.3 Associative Table

3.1.3.1 Staff Allocation Table

```

CREATE TABLE StaffAllocation (
    ServiceID          VARCHAR(6)      NOT NULL,
    StaffID            VARCHAR(6)      NOT NULL,
    AllocationDate     DATE             NOT NULL,
    TimeSlot           VARCHAR(20)     NOT NULL,
    PRIMARY KEY (ServiceID, StaffID, AllocationDate),
    FOREIGN KEY (ServiceID) REFERENCES Service (ServiceID),
    FOREIGN KEY (StaffID) REFERENCES Staff (StaffID)
);

```

3.1.3.2 Driver List Table

```

CREATE TABLE DriverList(
    BusScheduleID     VARCHAR(6)      NOT NULL,
    DriverID          VARCHAR(6)      NOT NULL,
    DriverRole         VARCHAR(50)    NOT NULL,
    PRIMARY KEY (BusScheduleID, DriverID),
    FOREIGN KEY (BusScheduleID) REFERENCES BusSchedule (BusScheduleID),
    FOREIGN KEY (DriverID) REFERENCES Driver (DriverID)
);

```

3.1.3.3 Booking Details Table

```

CREATE TABLE BookingDetails(
    BookingID          VARCHAR(6)      NOT NULL,
    TicketID           VARCHAR(6)      NOT NULL,
    PassengerIC        VARCHAR(20)     NOT NULL,
    PassengerName       VARCHAR(100)   NOT NULL,
    BookingDetailsStatus VARCHAR(20)   CHECK (BookingDetailsStatus IN
('Expired', 'Cancelled', 'Extended', 'Completed')) NOT NULL,
    ExtensionID        VARCHAR(6),
    RefundID           VARCHAR(6),
    PRIMARY KEY (BookingID, TicketID),
    FOREIGN KEY (BookingID) REFERENCES Booking (BookingID),
    FOREIGN KEY (TicketID) REFERENCES Ticket (TicketID),
    FOREIGN KEY (ExtensionID) REFERENCES Extension (ExtensionID),
    FOREIGN KEY (RefundID) REFERENCES Refund (RefundID)
);

```

3.1.3.4 Service Details Table

```

CREATE TABLE ServiceDetails(
    ServiceTransactionID VARCHAR(6)    NOT NULL,
    ServiceStatus         VARCHAR(50)  DEFAULT 'In Progress' CONSTRAINT
chk_ServiceStatus CHECK (ServiceStatus IN ('In Progress', 'Completed',
'Pending')) NOT NULL,
    ServiceDate           DATE          NOT NULL,
    ServiceCompleteDate   DATE          NOT NULL,

```

```

PlateNumber          VARCHAR(20)  NOT NULL,
ServiceID            VARCHAR(6)   NOT NULL,
PRIMARY KEY (ServiceTransactionID),
FOREIGN KEY (PlateNumber) REFERENCES Bus(PlateNumber),
FOREIGN KEY (ServiceID) REFERENCES Service(ServiceID)
);

```

3.1.3.5 Request Complaint Table

```

CREATE TABLE RequestComplain(
    RequestID          VARCHAR(6)   NOT NULL,
    RequestTopic        VARCHAR(100) NOT NULL,
    RequestDesc         VARCHAR(500) NOT NULL,
    RequestStatus       VARCHAR(50)  DEFAULT 'Submitted' CONSTRAINT
chk_RequestStatus    CHECK (RequestStatus IN ('Submitted', 'In Progress',
'Resolved', 'Rejected')) NOT NULL,
    RequestMemberID     VARCHAR(6)   NOT NULL,
    RequestStaffID      VARCHAR(6)   NOT NULL,
    PRIMARY KEY (RequestID),
    FOREIGN KEY (RequestMemberID) REFERENCES Member(MemberID),
    FOREIGN KEY (RequestStaffID) REFERENCES Staff(StaffID)
);

```

3.1.3.6 Point Redemption Table

```

CREATE TABLE PointRedemption(
    MemberID           VARCHAR(6)   NOT NULL,
    GiftID             VARCHAR(6)   NOT NULL,
    RedemptionDate      DATE         NOT NULL,
    RedemptionStatus    VARCHAR(50)  DEFAULT 'Pending'
                                CONSTRAINT chk_RedemptionStatus
                                CHECK (RedemptionStatus IN ('Pending',
'Collected', 'Cancelled')) NOT NULL,
    UsedDate           DATE,
    PRIMARY KEY (MemberID, GiftID, RedemptionDate),
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
    FOREIGN KEY (GiftID) REFERENCES Gift(GiftID)
);

```

3.1.3.7 Rental Collection Table

```

CREATE TABLE RentalCollection(
    StaffID            VARCHAR(6)   NOT NULL,
    ShopID             VARCHAR(6)   NOT NULL,
    MonthYear          DATE         NOT NULL,
    StartDate          DATE         NOT NULL,
    EndDate            DATE         NOT NULL,
    RentAmount         NUMBER(10,2) CHECK (RentAmount >= 0) NOT NULL,
    PaymentID          VARCHAR(6)   NOT NULL,
    PRIMARY KEY (StaffID, ShopID, MonthYear),
    FOREIGN KEY (StaffID) REFERENCES Staff(StaffID),
    FOREIGN KEY (ShopID) REFERENCES Shop(ShopID),
    FOREIGN KEY (PaymentID) REFERENCES Payment(PaymentID)
);

```

3.2 Sample Records (10 sample Records for each table)

3.2.1 Parent Table Record

3.2.1.1 Staff Record

```
INSERT INTO Staff VALUES ('S00001', 'Alice Tan', 'Counter Staff',
'0123456789', '900101-14-5678', TO_DATE('01-12-2020','DD-MM-YYYY'),
2500.00);
INSERT INTO Staff VALUES ('S00002', 'Benjamin Lee', 'Station Announcer',
'0134567890', '880202-10-2345', TO_DATE('15-01-2021','DD-MM-YYYY'),
2800.00);
INSERT INTO Staff VALUES ('S00003', 'Chong Wei', 'Cleaners', '0145678901',
'920303-08-9876', TO_DATE('10-03-2021','DD-MM-YYYY'), 1800.00);
INSERT INTO Staff VALUES ('S00004', 'Daphne Wong', 'Maintenance Staff',
'0167890123', '950404-12-1111', TO_DATE('20-04-2021','DD-MM-YYYY'),
3000.00);
INSERT INTO Staff VALUES ('S00005', 'Edward Lim', 'First Aid Staff',
'0178901234', '870505-06-2222', TO_DATE('05-05-2021','DD-MM-YYYY'),
3200.00);
INSERT INTO Staff VALUES ('S00006', 'Farah Ahmad', 'Security Staff',
'0189012345', '910606-05-3333', TO_DATE('18-06-2021','DD-MM-YYYY'),
2700.00);
INSERT INTO Staff VALUES ('S00007', 'George Tan', 'Queue and Boarding
Coordinator', '0190123456', '930707-14-4444',
TO_DATE('01-07-2021','DD-MM-YYYY'), 2600.00);
INSERT INTO Staff VALUES ('S00008', 'Helen Ng', 'Counter Staff',
'0111234567', '960808-02-5555', TO_DATE('12-08-2021','DD-MM-YYYY'),
2550.00);
INSERT INTO Staff VALUES ('S00009', 'Ivan Goh', 'Station Announcer',
'0122345678', '940909-09-6666', TO_DATE('09-09-2021','DD-MM-YYYY'),
2850.00);
INSERT INTO Staff VALUES ('S00010', 'Jennifer Lee', 'Cleaners',
'0133456789', '970101-10-7777', TO_DATE('01-10-2021','DD-MM-YYYY'),
1850.00);
```

3.2.1.2 Shop Record

```
INSERT INTO Shop VALUES ('SH0001', 'FreshMart', 'Grocery', 'Alice Wong',
'Lot 1', 1500.00);
INSERT INTO Shop VALUES ('SH0002', 'TechWorld', 'Electronics', 'John Tan',
'Lot 2', 2200.00);
INSERT INTO Shop VALUES ('SH0003', 'StyleHub', 'Clothing', 'Mei Ling',
'Lot 3', 1800.00);
INSERT INTO Shop VALUES ('SH0004', 'BookNest', 'Bookstore', 'Raj Kumar',
'Lot 4', 900.00);
INSERT INTO Shop VALUES ('SH0005', 'Happy Bites', 'Restaurant', 'Ahmad
Zain', 'Lot 5', 2500.00);
INSERT INTO Shop VALUES ('SH0006', 'PharmaCare', 'Pharmacy', 'Grace Lim',
'Lot 6', 800.00);
INSERT INTO Shop VALUES ('SH0007', 'HomeDeco', 'Furniture', 'Siti Aisyah',
'Lot 7', 3000.00);
INSERT INTO Shop VALUES ('SH0008', 'QuickPrint', 'Printing', 'David Lee',
'Lot 8', 700.00);
INSERT INTO Shop VALUES ('SH0009', 'ToyLand', 'Toys', 'Samuel Ng', 'Lot
9', 1200.00);
INSERT INTO Shop VALUES ('SH0010', 'FitZone', 'Gym', 'Michael Chan', 'Lot
10', 2800.00);
```

3.2.1.3 Gift Record

```
INSERT INTO Gift VALUES ('GF0001', 'Shopping Bag (Foldable, with Company
Logo)', 800, 100, 'Available', 'Eco-friendly foldable shopping bag with
printed company logo.');
```

```

INSERT INTO Gift VALUES ('GF0002', 'Keychain (Bus/Landmark Design)', 400,
150, 'Available', 'Collectible keychain featuring bus or local landmark
design.');
```

```

INSERT INTO Gift VALUES ('GF0003', 'Water Bottle / Tumbler', 1200, 60,
'Available', 'Reusable stainless steel tumbler for hot or cold drinks.');
```

```

INSERT INTO Gift VALUES ('GF0004', 'Phone Holder / Stand', 1000, 70,
'Available', 'Compact phone holder/stand suitable for travel use.');
```

```

INSERT INTO Gift VALUES ('GF0005', 'Mini Umbrella (Foldable)', 1500, 40,
'Available', 'Lightweight foldable umbrella for unexpected rain.');
```

```

INSERT INTO Gift VALUES ('GF0006', 'Travel Pouch (Passport, Tickets,
Coins)', 1800, 35, 'Available', 'Multipurpose travel pouch for important
travel documents and small items.');
```

```

INSERT INTO Gift VALUES ('GF0007', 'Notebook + Pen Set', 1000, 80,
'Available', 'Compact notebook with matching pen, perfect for travel
notes.');
```

```

INSERT INTO Gift VALUES ('GF0008', 'Luggage Tag', 900, 90, 'Available',
'Durable luggage tag for easy bag identification.');
```

```

INSERT INTO Gift VALUES ('GF0009', 'USB Charging Cable', 1100, 75,
'Available', 'Portable USB charging cable compatible with multiple
devices.');
```

```

INSERT INTO Gift VALUES ('GF0010', 'Portable Mini Fan', 2000, 30,
'Available', 'USB rechargeable mini fan, ideal for travel and outdoor
use.');
```

3.2.1.4 Payment Record

```

INSERT INTO Payment VALUES ('P00001', TO_DATE('01-01-2022', 'DD-MM-YYYY'),
'Credit Card', 'Completed', '1234567890');
```

```

INSERT INTO Payment VALUES ('P00002', TO_DATE('02-01-2022', 'DD-MM-YYYY'),
'Online Banking', 'Pending', '9876543210');
```

```

INSERT INTO Payment VALUES ('P00003', TO_DATE('03-01-2022', 'DD-MM-YYYY'),
'Debit Card', 'Failed', '4567891230');
```

```

INSERT INTO Payment VALUES ('P00004', TO_DATE('04-01-2022', 'DD-MM-YYYY'),
'Credit Card', 'Refunded', '1112223333');
```

```

INSERT INTO Payment VALUES ('P00005', TO_DATE('05-01-2022', 'DD-MM-YYYY'),
'Online Banking', 'Completed', '2223334444');
```

```

INSERT INTO Payment VALUES ('P00006', TO_DATE('06-01-2022', 'DD-MM-YYYY'),
'Debit Card', 'Pending', '3334445555');
```

```

INSERT INTO Payment VALUES ('P00007', TO_DATE('07-01-2022', 'DD-MM-YYYY'),
'Credit Card', 'Failed', '4445556666');
```

```

INSERT INTO Payment VALUES ('P00008', TO_DATE('08-01-2022', 'DD-MM-YYYY'),
'Online Banking', 'Refunded', '5556667777');
```

```

INSERT INTO Payment VALUES ('P00009', TO_DATE('09-01-2022', 'DD-MM-YYYY'),
'Debit Card', 'Completed', '6667778888');
```

```

INSERT INTO Payment VALUES ('P00010', TO_DATE('10-01-2022', 'DD-MM-YYYY'),
'Credit Card', 'Pending', '7778889999');
```

3.2.1.5 Bus Company Record

```

INSERT INTO BusCompany VALUES ('C00001', 'Starline Express', '0123456789',
'contact@starline.com', '123 Jalan Bukit Bintang, Kuala Lumpur');
```

```

INSERT INTO BusCompany VALUES ('C00002', 'Metro Bus Services',
'0123456790', 'info@metrobus.com', '45 Jalan Raja Chulan, Kuala Lumpur');
```

```

INSERT INTO BusCompany VALUES ('C00003', 'Rapid Travels', '0123456791',
'support@rapidtravels.com', '88 Jalan Sultan, Kuala Lumpur');
```

```

INSERT INTO BusCompany VALUES ('C00004', 'Greenway Transport',
'0123456792', 'hello@greenway.com', '12 Jalan Ipoh, Kuala Lumpur');
```

```

INSERT INTO BusCompany VALUES ('C00005', 'BlueSky Coaches', '0123456793',
'service@bluesky.com', '56 Jalan Gombak, Kuala Lumpur');
```

```

INSERT INTO BusCompany VALUES ('C00006', 'Citylink Travels', '0123456794',
'info@citylink.com', '101 Jalan Kuching, Kuala Lumpur');
```

```

INSERT INTO BusCompany VALUES ('C00007', 'Sunrise Express', '0123456795',
'contact@sunrise.com', '22 Jalan Pudu, Kuala Lumpur');
```

```

INSERT INTO BusCompany VALUES ('C00008', 'Galaxy Busline', '0123456796',
'info@galaxybus.com', '75 Jalan Imbi, Kuala Lumpur');
```

```
INSERT INTO BusCompany VALUES ('C00009', 'Speedway Coaches', '0123456797',
'support@speedway.com', '48 Jalan Ampang, Kuala Lumpur');
INSERT INTO BusCompany VALUES ('C00010', 'Silverline Transport',
'0123456798', 'info@silverline.com', '99 Jalan Cheras, Kuala Lumpur');
```

3.2.1.6 Service Record

```
INSERT INTO Service VALUES ('SV0001', 'Maintenance', 150.00);
INSERT INTO Service VALUES ('SV0002', 'Repair', 250.00);
INSERT INTO Service VALUES ('SV0003', 'Tyre Replace', 400.00);
INSERT INTO Service VALUES ('SV0004', 'Washing Services', 50.00);
INSERT INTO Service VALUES ('SV0005', 'Maintenance', 180.00);
INSERT INTO Service VALUES ('SV0006', 'Repair', 320.00);
INSERT INTO Service VALUES ('SV0007', 'Tyre Replace', 450.00);
INSERT INTO Service VALUES ('SV0008', 'Washing Services', 60.00);
INSERT INTO Service VALUES ('SV0009', 'Maintenance', 200.00);
INSERT INTO Service VALUES ('SV0010', 'Repair', 280.00);
```

3.2.1.7 Extension Record

```
INSERT INTO Extension VALUES ('E00001',
TO_DATE('15-09-2025','DD-MM-YYYY'), 5.00, 'Requested', 'Need more time due
to personal reasons');
INSERT INTO Extension VALUES ('E00002',
TO_DATE('20-09-2025','DD-MM-YYYY'), 5.00, 'Approved', 'Medical appointment
conflict');
INSERT INTO Extension VALUES ('E00003',
TO_DATE('25-09-2025','DD-MM-YYYY'), 5.00, 'Rejected', 'Reason not
sufficient');
INSERT INTO Extension VALUES ('E00004',
TO_DATE('30-09-2025','DD-MM-YYYY'), 5.00, 'Requested', 'Traveling
abroad');
INSERT INTO Extension VALUES ('E00005',
TO_DATE('05-10-2025','DD-MM-YYYY'), 5.00, 'Approved', 'Unexpected work
schedule');
INSERT INTO Extension VALUES ('E00006',
TO_DATE('10-10-2025','DD-MM-YYYY'), 5.00, 'Requested', 'Family
emergency');
INSERT INTO Extension VALUES ('E00007',
TO_DATE('15-10-2025','DD-MM-YYYY'), 5.00, 'Rejected', 'Exceeded maximum
extension limit');
INSERT INTO Extension VALUES ('E00008',
TO_DATE('20-10-2025','DD-MM-YYYY'), 5.00, 'Approved', 'Project deadline
extension');
INSERT INTO Extension VALUES ('E00009',
TO_DATE('25-10-2025','DD-MM-YYYY'), 5.00, 'Requested', 'Car breakdown
issue');
INSERT INTO Extension VALUES ('E00010',
TO_DATE('30-10-2025','DD-MM-YYYY'), 5.00, 'Approved', 'University
assignment submission');
```

3.2.1.8 Refund Record

```
INSERT INTO Refund VALUES ('R00001', TO_DATE('05-09-2025','DD-MM-YYYY'),
'Trip canceled by bus company due to maintenance', 120.50, 'Pending');
INSERT INTO Refund VALUES ('R00002', TO_DATE('07-09-2025','DD-MM-YYYY'),
'Route canceled by bus company on short notice', 200.00, 'Approved');
INSERT INTO Refund VALUES ('R00003', TO_DATE('09-09-2025','DD-MM-YYYY'),
'Missed the bus', 95.75, 'Rejected');
INSERT INTO Refund VALUES ('R00004', TO_DATE('11-09-2025','DD-MM-YYYY'),
'Service canceled due to driver unavailability', 150.00, 'Pending');
INSERT INTO Refund VALUES ('R00005', TO_DATE('13-09-2025','DD-MM-YYYY'),
'Bus breakdown caused trip cancellation', 180.00, 'Approved');
INSERT INTO Refund VALUES ('R00006', TO_DATE('15-09-2025','DD-MM-YYYY'),
```

```
'Trip canceled by operator due to weather conditions', 220.00, 'Pending');
INSERT INTO Refund VALUES ('R00007', TO_DATE('17-09-2025','DD-MM-YYYY'),
'Missed the bus', 75.00, 'Rejected');
INSERT INTO Refund VALUES ('R00008', TO_DATE('19-09-2025','DD-MM-YYYY'),
'Trip rescheduled by company causing conflict', 130.00, 'Approved');
INSERT INTO Refund VALUES ('R00009', TO_DATE('21-09-2025','DD-MM-YYYY'),
'Service auto-canceled by system due to driver absence', 60.00,
'Pending');
INSERT INTO Refund VALUES ('R00010', TO_DATE('23-09-2025','DD-MM-YYYY'),
'Trip removed from schedule after booking confirmation', 250.00,
'Approved');
```

3.2.2 Child Table Record

3.2.2.0 : Member Record

```
INSERT INTO Member VALUES('M00001', 'Alice Tan Mei Ling',
TO_DATE('12-03-1995','DD-MM-YYYY'), 'Female', 'alice.tan@example.com',
01234567890, TO_DATE('15-01-2023','DD-MM-YYYY'), 120, 'Active',
'950312-10-1234', 'P00401');
INSERT INTO Member VALUES('M00002', 'Benjamin Lee Wei',
TO_DATE('24-07-1992','DD-MM-YYYY'), 'Male', 'ben.lee@example.com',
01345678901, TO_DATE('20-11-2022','DD-MM-YYYY'), 450, 'Active',
'920724-08-5677', 'P00402');
INSERT INTO Member VALUES('M00003', 'Chong Li Hua',
TO_DATE('30-05-1988','DD-MM-YYYY'), 'Female', 'lihua.chong@example.com',
01456789012, TO_DATE('10-03-2023','DD-MM-YYYY'), 75, 'Inactive',
'880530-05-9876', 'P00403');
INSERT INTO Member VALUES('M00004', 'Daniel Wong Jun',
TO_DATE('05-12-2000','DD-MM-YYYY'), 'Male', 'daniel.wong@example.com',
01567890123, TO_DATE('01-02-2024','DD-MM-YYYY'), 0, 'Active',
'001205-07-1235', 'P00404');
INSERT INTO Member VALUES('M00005', 'Emily Ng Jia',
TO_DATE('17-09-1997','DD-MM-YYYY'), 'Female', 'emily.ng@example.com',
01678901234, TO_DATE('22-05-2023','DD-MM-YYYY'), 300, 'Banned',
'970917-14-4568', 'P00405');
INSERT INTO Member VALUES('M00006', 'Farid Ahmad',
TO_DATE('11-11-1994','DD-MM-YYYY'), 'Male', 'farid.ahmad@example.com',
01789012345, TO_DATE('30-09-2022','DD-MM-YYYY'), 95, 'Active',
'941111-06-8889', 'P00406');
INSERT INTO Member VALUES('M00007', 'Grace Lim Siew',
TO_DATE('14-02-1985','DD-MM-YYYY'), 'Female', 'grace.lim@example.com',
01890123456, TO_DATE('10-07-2021','DD-MM-YYYY'), 800, 'Active',
'850214-12-2346', 'P00407');
INSERT INTO Member VALUES('M00008', 'Hafiz Roslan',
TO_DATE('20-01-1999','DD-MM-YYYY'), 'Male', 'hafiz.roslan@example.com',
01901234567, TO_DATE('02-01-2024','DD-MM-YYYY'), 10, 'Inactive',
'990120-03-8765', 'P00408');
INSERT INTO Member VALUES('M00009', 'Isabella Tan',
TO_DATE('07-06-1993','DD-MM-YYYY'), 'Female', 'isabella.tan@example.com',
01112233445, TO_DATE('15-06-2023','DD-MM-YYYY'), 260, 'Active',
'930607-01-2348', 'P00409');
INSERT INTO Member VALUES('M00010', 'Jason Ong',
TO_DATE('29-08-1989','DD-MM-YYYY'), 'Male', 'jason.ong@example.com',
01223344556, TO_DATE('20-04-2022','DD-MM-YYYY'), 520, 'Banned',
'890829-09-5673', 'P00410');
```

3.2.2.1 Booking Record

```
INSERT INTO Booking VALUES ('B00001', TO_DATE('01-01-2022','DD-MM-YYYY'),
'Completed', 295.00, 'M00001', 'P00001');
INSERT INTO Booking VALUES ('B00002', TO_DATE('02-01-2022','DD-MM-YYYY'),
'Pending', 250.00, 'M00002', 'P00002');
INSERT INTO Booking VALUES ('B00003', TO_DATE('03-01-2022','DD-MM-YYYY'),
```

```
'Cancelled', 80.75, 'M00003', 'P00003');
INSERT INTO Booking VALUES ('B00004', TO_DATE('04-01-2022','DD-MM-YYYY'),
'Completed', 300.00, 'M00004', 'P00004');
INSERT INTO Booking VALUES ('B00005', TO_DATE('05-01-2022','DD-MM-YYYY'),
'Completed', 175.25, 'M00005', 'P00005');
INSERT INTO Booking VALUES ('B00006', TO_DATE('06-01-2022','DD-MM-YYYY'),
'Pending', 220.00, 'M00006', 'P00006');
INSERT INTO Booking VALUES ('B00007', TO_DATE('07-01-2022','DD-MM-YYYY'),
'Cancelled', 95.90, 'M00007', 'P00007');
INSERT INTO Booking VALUES ('B00008', TO_DATE('08-01-2022','DD-MM-YYYY'),
'Completed', 310.40, 'M00008', 'P00008');
INSERT INTO Booking VALUES ('B00009', TO_DATE('09-01-2022','DD-MM-YYYY'),
'Completed', 450.00, 'M00009', 'P00009');
INSERT INTO Booking VALUES ('B00010', TO_DATE('10-01-2022','DD-MM-YYYY'),
'Pending', 125.60, 'M00010', 'P00010');
```

3.2.2.2 Bus Record

```
INSERT INTO Bus VALUES ('BU0001', 40, 'Single Deck', 'C00001');
INSERT INTO Bus VALUES ('BU0002', 80, 'Double Decker', 'C00002');
INSERT INTO Bus VALUES ('BU0003', 30, 'Luxury Coach', 'C00003');
INSERT INTO Bus VALUES ('BU0004', 25, 'Mini Bus', 'C00004');
INSERT INTO Bus VALUES ('BU0005', 40, 'Single Deck', 'C00005');
INSERT INTO Bus VALUES ('BU0006', 80, 'Double Decker', 'C00006');
INSERT INTO Bus VALUES ('BU0007', 30, 'Luxury Coach', 'C00007');
INSERT INTO Bus VALUES ('BU0008', 25, 'Mini Bus', 'C00008');
INSERT INTO Bus VALUES ('BU0009', 40, 'Single Deck', 'C00009');
INSERT INTO Bus VALUES ('BU0010', 80, 'Double Decker', 'C00010');
```

3.2.2.3 Bus Schedule Record

```
INSERT INTO BusSchedule VALUES ('BS0001', TO_DATE('12-01-2023
08:00','DD-MM-YYYY HH24:MI'), TO_DATE('12-01-2023 12:30','DD-MM-YYYY
HH24:MI'), TO_DATE('12-01-2023','DD-MM-YYYY'), 'Penang', 'Scheduled',
'Kuala Lumpur', 'Penang Sentral', 'P1', 14, 350, 'BU0001');
INSERT INTO BusSchedule VALUES ('BS0002', TO_DATE('22-01-2023
09:30','DD-MM-YYYY HH24:MI'), TO_DATE('22-01-2023 13:45','DD-MM-YYYY
HH24:MI'), TO_DATE('22-01-2023','DD-MM-YYYY'), 'Ipoh', 'Scheduled', 'Kuala
Lumpur', 'Ipoh Amanjaya', 'P2', 23, 200, 'BU0002');
INSERT INTO BusSchedule VALUES ('BS0003', TO_DATE('05-02-2023
07:00','DD-MM-YYYY HH24:MI'), TO_DATE('05-02-2023 11:15','DD-MM-YYYY
HH24:MI'), TO_DATE('05-02-2023','DD-MM-YYYY'), 'Johor Bahru', 'Delayed',
'Kuala Lumpur', 'Larkin Sentral', 'P3', 20, 330, 'BU0003');
INSERT INTO BusSchedule VALUES ('BS0004', TO_DATE('15-02-2023
10:00','DD-MM-YYYY HH24:MI'), TO_DATE('15-02-2023 14:45','DD-MM-YYYY
HH24:MI'), TO_DATE('15-02-2023','DD-MM-YYYY'), 'Kuantan', 'Scheduled',
'Kuala Lumpur', 'Kuantan Sentral', 'P4', 17, 250, 'BU0004');
INSERT INTO BusSchedule VALUES ('BS0005', TO_DATE('25-02-2023
06:30','DD-MM-YYYY HH24:MI'), TO_DATE('25-02-2023 11:00','DD-MM-YYYY
HH24:MI'), TO_DATE('25-02-2023','DD-MM-YYYY'), 'Melaka', 'Completed',
'Kuala Lumpur', 'Melaka Sentral', 'P5', 30, 145, 'BU0005');
INSERT INTO BusSchedule VALUES ('BS0006', TO_DATE('05-03-2023
08:15','DD-MM-YYYY HH24:MI'), TO_DATE('05-03-2023 12:45','DD-MM-YYYY
HH24:MI'), TO_DATE('05-03-2023','DD-MM-YYYY'), 'Alor Setar', 'Scheduled',
'Penang Sentral', 'Alor Setar Terminal', 'P6', 70, 120, 'BU0006');
INSERT INTO BusSchedule VALUES ('BS0007', TO_DATE('15-03-2023
09:00','DD-MM-YYYY HH24:MI'), TO_DATE('15-03-2023 13:30','DD-MM-YYYY
HH24:MI'), TO_DATE('15-03-2023','DD-MM-YYYY'), 'Kuala Terengganu',
'Cancelled', 'Kuantan Sentral', 'Kuala Terengganu Terminal', 'P7', 0, 170,
'BU0007');
INSERT INTO BusSchedule VALUES ('BS0008', TO_DATE('25-03-2023
14:00','DD-MM-YYYY HH24:MI'), TO_DATE('25-03-2023 18:30','DD-MM-YYYY
HH24:MI'), TO_DATE('25-03-2023','DD-MM-YYYY'), 'Kota Bharu', 'Scheduled',
'Kuala Lumpur', 'Kota Bharu Terminal', 'P8', 25, 440, 'BU0008');
INSERT INTO BusSchedule VALUES ('BS0009', TO_DATE('05-04-2023
```

```
07:45','DD-MM-YYYY HH24:MI'), TO_DATE('05-04-2023 12:00','DD-MM-YYYY
HH24:MI'), TO_DATE('05-04-2023','DD-MM-YYYY'), 'Seremban', 'Completed',
'Kuala Lumpur', 'Seremban Terminal', 'P9', 12, 65, 'BU0009');
INSERT INTO BusSchedule VALUES ('BS0010', TO_DATE('15-04-2023
06:00','DD-MM-YYYY HH24:MI'), TO_DATE('15-04-2023 10:20','DD-MM-YYYY
HH24:MI'), TO_DATE('15-04-2023','DD-MM-YYYY'), 'Ipoh', 'Scheduled',
'Penang Sentral', 'Ipoh Amanjaya', 'P10', 39, 160, 'BU0010');
```

3.2.2.4 Driver Record

```
INSERT INTO Driver VALUES ('D00001', 'Ahmad
Faizal', '0123456001', 'LIC0001', '900101-01-1234', TO_DATE('01-01-2015', 'DD-M
M-YYYY'), 'Active', TO_DATE('01-01-2026', 'DD-MM-YYYY'), 10, 'C00001');
INSERT INTO Driver VALUES ('D00002', 'Siti
Nurhaliza', '0123456002', 'LIC0002', '880202-02-2345', TO_DATE('15-03-2016', 'D
D-MM-YYYY'), 'Active', TO_DATE('15-03-2025', 'DD-MM-YYYY'), 9, 'C00002');
INSERT INTO Driver VALUES ('D00003', 'Mohd
Hafiz', '0123456003', 'LIC0003', '850303-03-3456', TO_DATE('20-07-2017', 'DD-MM
-YYYY'), 'Suspended', TO_DATE('20-07-2025', 'DD-MM-YYYY'), 8, 'C00003');
INSERT INTO Driver VALUES ('D00004', 'Nur
Aisyah', '0123456004', 'LIC0004', '920404-04-4567', TO_DATE('05-11-2018', 'DD-M
M-YYYY'), 'Inactive', TO_DATE('05-11-2025', 'DD-MM-YYYY'), 6, 'C00004');
INSERT INTO Driver VALUES ('D00005', 'Ahmad
Zaki', '0123456005', 'LIC0005', '870505-05-5678', TO_DATE('12-01-2019', 'DD-MM-
YYYY'), 'Active', TO_DATE('12-01-2025', 'DD-MM-YYYY'), 5, 'C00005');
INSERT INTO Driver VALUES ('D00006', 'Farah
Hani', '0123456006', 'LIC0006', '910606-06-6789', TO_DATE('18-06-2020', 'DD-MM-
YYYY'), 'Active', TO_DATE('18-06-2025', 'DD-MM-YYYY'), 4, 'C00006');
INSERT INTO Driver VALUES ('D00007', 'Mohd
Rizal', '0123456007', 'LIC0007', '880707-07-7890', TO_DATE('22-08-2020', 'DD-MM
-YYYY'), 'Suspended', TO_DATE('22-08-2025', 'DD-MM-YYYY'), 3, 'C00007');
INSERT INTO Driver VALUES ('D00008', 'Aina
Sofea', '0123456008', 'LIC0008', '900808-08-8901', TO_DATE('30-09-2021', 'DD-MM
-YYYY'), 'Active', TO_DATE('30-09-2025', 'DD-MM-YYYY'), 2, 'C00008');
INSERT INTO Driver VALUES ('D00009', 'Faiz
Rahman', '0123456009', 'LIC0009', '860909-09-9012', TO_DATE('15-02-2022', 'DD-M
M-YYYY'), 'Inactive', TO_DATE('15-02-2025', 'DD-MM-YYYY'), 1, 'C00009');
INSERT INTO Driver VALUES ('D00010', 'Hafizah
Ramli', '0123456010', 'LIC0010', '920101-10-0123', TO_DATE('01-07-2023', 'DD-MM
-YYYY'), 'Active', TO_DATE('01-07-2025', 'DD-MM-YYYY'), 0, 'C00010');
```

3.2.2.5 Ticket Record

```
INSERT INTO Ticket VALUES ('T00001', 50.00, 'Booked', 'Adult', '1',
'BS0001');
INSERT INTO Ticket VALUES ('T00002', 50.00, 'Booked', 'Adult', '2',
'BS0001');
INSERT INTO Ticket VALUES ('T00003', 50.00, 'Booked', 'Adult', '3',
'BS0001');
INSERT INTO Ticket VALUES ('T00004', 50.00, 'Available', 'Adult', '4',
'BS0001');
INSERT INTO Ticket VALUES ('T00005', 50.00, 'Booked', 'Adult', '5',
'BS0001');
INSERT INTO Ticket VALUES ('T00006', 50.00, 'Available', 'Adult', '6',
'BS0001');
INSERT INTO Ticket VALUES ('T00007', 50.00, 'Booked', 'Child', '7',
'BS0001');
INSERT INTO Ticket VALUES ('T00008', 50.00, 'Booked', 'Adult', '8',
'BS0001');
INSERT INTO Ticket VALUES ('T00009', 50.00, 'Available', 'Child', '9',
'BS0001');
INSERT INTO Ticket VALUES ('T00010', 50.00, 'Booked', 'Adult', '10',
'BS0001');
```

3.2.3 Associative Table Record

3.2.3.1 Staff Allocation Record

```
INSERT INTO StaffAllocation VALUES ('SV0001', 'S00011',
TO_DATE('11-04-2023', 'dd-mm-yyyy'), 'Afternoon');
INSERT INTO StaffAllocation VALUES ('SV0001', 'S00019',
TO_DATE('14-12-2023', 'dd-mm-yyyy'), 'Morning');
INSERT INTO StaffAllocation VALUES ('SV0020', 'S00013',
TO_DATE('18-01-2023', 'dd-mm-yyyy'), 'Morning');
INSERT INTO StaffAllocation VALUES ('SV0014', 'S00007',
TO_DATE('02-08-2023', 'dd-mm-yyyy'), 'Evening');
INSERT INTO StaffAllocation VALUES ('SV0001', 'S00001',
TO_DATE('25-12-2023', 'dd-mm-yyyy'), 'Evening');
INSERT INTO StaffAllocation VALUES ('SV0006', 'S00004',
TO_DATE('23-06-2023', 'dd-mm-yyyy'), 'Morning');
INSERT INTO StaffAllocation VALUES ('SV0020', 'S00020',
TO_DATE('31-01-2023', 'dd-mm-yyyy'), 'Morning');
INSERT INTO StaffAllocation VALUES ('SV0013', 'S00009',
TO_DATE('10-10-2023', 'dd-mm-yyyy'), 'Afternoon');
INSERT INTO StaffAllocation VALUES ('SV0020', 'S00020',
TO_DATE('30-12-2023', 'dd-mm-yyyy'), 'Evening');
INSERT INTO StaffAllocation VALUES ('SV0005', 'S00004',
TO_DATE('09-04-2023', 'dd-mm-yyyy'), 'Morning');
```

3.2.3.2 Driver List Record

```
INSERT INTO DriverList VALUES ('BS0023', 'D00030', 'Assistant Driver');
INSERT INTO DriverList VALUES ('BS0007', 'D00008', 'Main Driver');
INSERT INTO DriverList VALUES ('BS0059', 'D00053', 'Main Driver');
INSERT INTO DriverList VALUES ('BS0055', 'D00078', 'Main Driver');
INSERT INTO DriverList VALUES ('BS0013', 'D00048', 'Main Driver');
INSERT INTO DriverList VALUES ('BS0070', 'D00065', 'Assistant Driver');
INSERT INTO DriverList VALUES ('BS0047', 'D00093', 'Main Driver');
INSERT INTO DriverList VALUES ('BS0033', 'D00062', 'Main Driver');
INSERT INTO DriverList VALUES ('BS0085', 'D00072', 'Main Driver');
INSERT INTO DriverList VALUES ('BS0032', 'D00009', 'Assistant Driver');
```

3.2.3.3 Booking Details Record

```
INSERT INTO BookingDetails VALUES ('B00001', 'T00001', '480011754894',
'Tina Hughes', 'Completed', NULL, NULL);
INSERT INTO BookingDetails VALUES ('B00001', 'T00002', '907046713240',
'Kristen Jenkins', 'Expired', NULL, NULL);
INSERT INTO BookingDetails VALUES ('B00001', 'T00003', '511991255499',
'Matthew Adams', 'Expired', NULL, NULL);
INSERT INTO BookingDetails VALUES ('B00001', 'T00004', '559437281434',
'Jordan Alvarado', 'Extended', 'E00021', NULL);
INSERT INTO BookingDetails VALUES ('B00001', 'T00005', '637593947842',
'Katie Smith', 'Completed', NULL, NULL);
INSERT INTO BookingDetails VALUES ('B00002', 'T00006', '147766679204',
'Jonathan Barron', 'Cancelled', NULL, 'R00021');
INSERT INTO BookingDetails VALUES ('B00002', 'T00007', '172988291656',
'Jeffrey Weaver', 'Completed', NULL, NULL);
INSERT INTO BookingDetails VALUES ('B00002', 'T00008', '190970529185',
'Ana Williams', 'Expired', NULL, NULL);
INSERT INTO BookingDetails VALUES ('B00003', 'T00009', '518396112938',
'Hayley Hawkins', 'Cancelled', NULL, 'R00022');
INSERT INTO BookingDetails VALUES ('B00003', 'T00010', '565894262680',
'Becky Walter', 'Expired', NULL, NULL);
```

3.2.3.4 Service Details Record

```
INSERT INTO ServiceDetails VALUES ('ST0001', 'Completed',
```

```

TO_DATE('23-03-2024', 'dd-mm-yyyy'), TO_DATE('01-04-2024', 'dd-mm-yyyy'),
'BU0087', 'SV0012');
INSERT INTO ServiceDetails VALUES ('ST0002', 'In Progress',
TO_DATE('25-07-2024', 'dd-mm-yyyy'), TO_DATE('30-07-2024', 'dd-mm-yyyy'),
'BU0080', 'SV0017');
INSERT INTO ServiceDetails VALUES ('ST0003', 'Pending',
TO_DATE('27-10-2024', 'dd-mm-yyyy'), TO_DATE('29-10-2024', 'dd-mm-yyyy'),
'BU0056', 'SV0013');
INSERT INTO ServiceDetails VALUES ('ST0004', 'In Progress',
TO_DATE('25-08-2023', 'dd-mm-yyyy'), TO_DATE('27-08-2023', 'dd-mm-yyyy'),
'BU0059', 'SV0019');
INSERT INTO ServiceDetails VALUES ('ST0005', 'Pending',
TO_DATE('20-04-2023', 'dd-mm-yyyy'), TO_DATE('23-04-2023', 'dd-mm-yyyy'),
'BU0030', 'SV0013');
INSERT INTO ServiceDetails VALUES ('ST0006', 'Completed',
TO_DATE('22-09-2023', 'dd-mm-yyyy'), TO_DATE('25-09-2023', 'dd-mm-yyyy'),
'BU0066', 'SV0003');
INSERT INTO ServiceDetails VALUES ('ST0007', 'Pending',
TO_DATE('30-01-2023', 'dd-mm-yyyy'), TO_DATE('05-02-2023', 'dd-mm-yyyy'),
'BU0084', 'SV0008');
INSERT INTO ServiceDetails VALUES ('ST0008', 'In Progress',
TO_DATE('04-03-2023', 'dd-mm-yyyy'), TO_DATE('12-03-2023', 'dd-mm-yyyy'),
'BU0094', 'SV0003');
INSERT INTO ServiceDetails VALUES ('ST0009', 'Completed',
TO_DATE('28-11-2023', 'dd-mm-yyyy'), TO_DATE('05-12-2023', 'dd-mm-yyyy'),
'BU0041', 'SV0016');
INSERT INTO ServiceDetails VALUES ('ST0010', 'Pending',
TO_DATE('21-01-2024', 'dd-mm-yyyy'), TO_DATE('25-01-2024', 'dd-mm-yyyy'),
'BU0076', 'SV0002');

```

3.2.3.5 Request Complaint Record

```

INSERT INTO RequestComplain VALUES ('RQ0001', 'Crowded Waiting Area',
'There are too few seats for waiting passengers.', 'In Progress',
'M99999', 'S00014');
INSERT INTO RequestComplain VALUES ('RQ0002', 'Late Bus Announcements',
'Delays are not announced in time.', 'Rejected', 'M00008', 'S00002');
INSERT INTO RequestComplain VALUES ('RQ0003', 'No Information Display',
'There is no functioning screen for bus schedules.', 'Submitted',
'M00013', 'S00018');
INSERT INTO RequestComplain VALUES ('RQ0004', 'Toilet Maintenance
Required', 'The toilets at the station are in poor condition.',
'Rejected', 'M00004', 'S00006');
INSERT INTO RequestComplain VALUES ('RQ0005', 'Crowded Waiting Area',
'There are too few seats for waiting passengers.', 'Rejected', 'M00007',
'S00009');
INSERT INTO RequestComplain VALUES ('RQ0006', 'No Shelter from Rain', 'The
station does not provide enough cover from rain.', 'In Progress',
'M00017', 'S00004');
INSERT INTO RequestComplain VALUES ('RQ0007', 'No Shelter from Rain', 'The
station does not provide enough cover from rain.', 'In Progress',
'M00015', 'S00008');
INSERT INTO RequestComplain VALUES ('RQ0008', 'No Information Display',
'There is no functioning screen for bus schedules.', 'Submitted',
'M00014', 'S00005');
INSERT INTO RequestComplain VALUES ('RQ0009', 'No Shelter from Rain', 'The
station does not provide enough cover from rain.', 'Resolved', 'M00004',
'S00013');
INSERT INTO RequestComplain VALUES ('RQ0010', 'Toilet Maintenance
Required', 'The toilets at the station are in poor condition.', 'In
Progress', 'M00005', 'S00005');

```

3.2.3.6 Point Redemption Record

```

INSERT INTO PointRedemption VALUES ('M00019', 'GF0005',

```

```

TO_DATE('18-01-2023', 'dd-mm-yyyy'), 'Cancelled', NULL);
INSERT INTO PointRedemption VALUES ('M00019', 'GF0007',
TO_DATE('08-11-2023', 'dd-mm-yyyy'), 'Collected', TO_DATE('11-11-2023',
'dd-mm-yyyy'));
INSERT INTO PointRedemption VALUES ('M00016', 'GF0015',
TO_DATE('16-02-2024', 'dd-mm-yyyy'), 'Cancelled', NULL);
INSERT INTO PointRedemption VALUES ('M00016', 'GF0016',
TO_DATE('09-10-2023', 'dd-mm-yyyy'), 'Cancelled', NULL);
INSERT INTO PointRedemption VALUES ('M00015', 'GF0001',
TO_DATE('23-08-2023', 'dd-mm-yyyy'), 'Collected', TO_DATE('02-09-2023',
'dd-mm-yyyy'));
INSERT INTO PointRedemption VALUES ('M00012', 'GF0016',
TO_DATE('10-05-2024', 'dd-mm-yyyy'), 'Pending', NULL);
INSERT INTO PointRedemption VALUES ('M00005', 'GF0017',
TO_DATE('23-01-2023', 'dd-mm-yyyy'), 'Pending', NULL);
INSERT INTO PointRedemption VALUES ('M00015', 'GF0018',
TO_DATE('08-08-2024', 'dd-mm-yyyy'), 'Collected', TO_DATE('31-08-2024',
'dd-mm-yyyy'));
INSERT INTO PointRedemption VALUES ('M00019', 'GF0017',
TO_DATE('17-11-2023', 'dd-mm-yyyy'), 'Collected', TO_DATE('18-11-2023',
'dd-mm-yyyy'));
INSERT INTO PointRedemption VALUES ('M00011', 'GF0004',
TO_DATE('09-11-2023', 'dd-mm-yyyy'), 'Collected', TO_DATE('12-11-2023',
'dd-mm-yyyy'));

```

3.2.3.7 Rental Collection Record

```

INSERT INTO RentalCollection VALUES ('S00007', 'SH0001',
TO_DATE('01-2025', 'MM-YYYY'), TO_DATE('01-01-2025', 'DD-MM-YYYY'),
TO_DATE('31-01-2025', 'DD-MM-YYYY'), 2385.48, 'P00101');
INSERT INTO RentalCollection VALUES ('S00008', 'SH0002',
TO_DATE('01-2025', 'MM-YYYY'), TO_DATE('01-01-2025', 'DD-MM-YYYY'),
TO_DATE('31-01-2025', 'DD-MM-YYYY'), 1663.86, 'P00102');
INSERT INTO RentalCollection VALUES ('S00013', 'SH0003',
TO_DATE('01-2025', 'MM-YYYY'), TO_DATE('01-01-2025', 'DD-MM-YYYY'),
TO_DATE('31-01-2025', 'DD-MM-YYYY'), 3664.54, 'P00103');
INSERT INTO RentalCollection VALUES ('S00006', 'SH0004',
TO_DATE('01-2025', 'MM-YYYY'), TO_DATE('01-01-2025', 'DD-MM-YYYY'),
TO_DATE('31-01-2025', 'DD-MM-YYYY'), 3100.93, 'P00104');
INSERT INTO RentalCollection VALUES ('S00003', 'SH0005',
TO_DATE('01-2025', 'MM-YYYY'), TO_DATE('01-01-2025', 'DD-MM-YYYY'),
TO_DATE('31-01-2025', 'DD-MM-YYYY'), 3661.70, 'P00105');
INSERT INTO RentalCollection VALUES ('S00002', 'SH0006',
TO_DATE('01-2025', 'MM-YYYY'), TO_DATE('01-01-2025', 'DD-MM-YYYY'),
TO_DATE('31-01-2025', 'DD-MM-YYYY'), 4039.26, 'P00106');
INSERT INTO RentalCollection VALUES ('S00017', 'SH0007',
TO_DATE('01-2025', 'MM-YYYY'), TO_DATE('01-01-2025', 'DD-MM-YYYY'),
TO_DATE('31-01-2025', 'DD-MM-YYYY'), 3532.84, 'P00107');
INSERT INTO RentalCollection VALUES ('S00015', 'SH0008',
TO_DATE('01-2025', 'MM-YYYY'), TO_DATE('01-01-2025', 'DD-MM-YYYY'),
TO_DATE('31-01-2025', 'DD-MM-YYYY'), 1738.36, 'P00108');
INSERT INTO RentalCollection VALUES ('S00007', 'SH0009',
TO_DATE('01-2025', 'MM-YYYY'), TO_DATE('01-01-2025', 'DD-MM-YYYY'),
TO_DATE('31-01-2025', 'DD-MM-YYYY'), 1656.30, 'P00109');
INSERT INTO RentalCollection VALUES ('S00001', 'SH0010',
TO_DATE('01-2025', 'MM-YYYY'), TO_DATE('01-01-2025', 'DD-MM-YYYY'),
TO_DATE('31-01-2025', 'DD-MM-YYYY'), 3034.16, 'P00110');

```

Chapter 4 Queries, Procedures, Triggers and Reports

4.1 Chan Jin Wei

4.1.1 Query 1: View Bus Schedule by Bus Company

Purpose:

The purpose of this query is to generate a structured view of upcoming bus schedules by combining details from bus companies, buses and schedules. It displays essential trip information such as departure and arrival times, stations, destinations, seat availability and status. By filtering to show only future departures, it ensures the report remains relevant for passengers and management. This supports better service monitoring, operational planning and customer information.

SQL statement:

```
CREATE OR REPLACE VIEW BusScheduleView AS
SELECT
    bc.CompanyName          AS Company_Name,
    b.BusType               AS Bus_Type,          -- first column for break
    b.PlateNumber           AS Bus_Plate,        -- second column
    bs.BusScheduleID        AS ScheduleID,
    TO_CHAR(bs.DepartureDate, 'DD-MM-YYYY') AS Departure_Date,
    TO_CHAR(bs.DepartureTime, 'HH24:MI')      AS Depart_Time,
    TO_CHAR(bs.ArrivalTime, 'HH24:MI')        AS Arr_Time,
    bs.DepartBusStation     AS Departure_Station,
    bs.ArrivalBusStation   AS Arrival_Station,
    bs.Destination         AS Destination,
    bs.PlatformNo          AS Platform,
    bs.AvailableSeat        AS Seats_Available,
    bs.ScheduleStatus      AS Status
FROM   BusCompany bc
JOIN   Bus b
      ON bc.CompanyID = b.CompanyID
JOIN   BusSchedule bs
      ON b.PlateNumber = bs.PlateNumber
WHERE  bs.DepartureDate > TRUNC(SYSDATE);

SET PAGESIZE 100
SET LINESIZE 210
SET TRIMSPOOL ON

COLUMN Bus_Type          FORMAT A15
COLUMN Company_Name     FORMAT A20
COLUMN Bus_Plate        FORMAT A10
COLUMN ScheduleID       FORMAT A10
COLUMN Departure_Date   FORMAT A14
COLUMN Depart_Time      FORMAT A14
COLUMN Arr_Time         FORMAT A10
COLUMN Departure_Station FORMAT A20
COLUMN Arrival_Station  FORMAT A27
COLUMN Destination      FORMAT A16
COLUMN Platform_No      FORMAT A10
COLUMN Seats_Available  FORMAT 999
COLUMN Status           FORMAT A12;

BREAK ON Company_Name SKIP 1 ON Bus_Type ON Bus_Plate

SELECT *
FROM   BusScheduleView
ORDER BY Bus_Type, Company_Name, Bus_Plate, Departure_Date, Depart_Time;

CLEAR COLUMN
CLEAR BREAK;
```

Sample Output:

COMPANY_NAME	BUS_TYPE	BUS_PLATE	SCHEDULEID	DEPARTURE_DATE	DEPART_TIME	ARR_TIME	DEPARTURE_STATION	ARRIVAL_STATION	DESTINATION	PLATFORM	SEATS_AVAILABLE	STATUS
Orbit Express	Double Decker	BU0161	BS6110	21-07-2026	06:12	08:05	Sungai Petani	KL Sentral	Butterworth	P5	26	Delayed
			BS8694	21-12-2026	17:50	23:54	Alor Setar	Larkin Sentral Johor Bahru	Johor Bahru	P5	65	Scheduled
			BS1830	22-09-2025	16:16	17:56	Sungai Petani	Seremban Terminal	Kuala Lumpur	P9	19	Completed
			BS5273	26-05-2026	11:48	13:37	Ipoh	Taipung Mamunting Terminal	Shah Alam	P1	16	Completed
			BS2026	26-10-2025	06:33	11:14	Kuala Lumpur	Penang Sungai Nibong	Kuala Terengganu	P4	51	Delayed
			BS6368	30-07-2026	07:52	13:29	Taipung	Butterworth Terminal	Malacca	P6	36	Scheduled
			BS2387	30-10-2025	15:36	19:37	Kota Bharu	Alor Setar Terminal	Ipoh	P8	21	Scheduled
Prineline Travels	Double Decker	BU0116	BS4166	01-02-2026	12:57	16:42	Penang	Larkin Sentral Johor Bahru	Ipoh	P3	10	Cancelled
			BS5919	04-06-2026	07:59	12:31	Ipoh	Penang Sungai Nibong	Alor Setar	P2	33	Completed
			BS6147	05-07-2026	10:34	15:34	Seremban	Seremban Terminal	Malacca	P8	22	Completed
			BS6525	06-08-2026	12:25	16:51	Kuala Lumpur	Kota Bharu Terminal	Butterworth	P2	46	Cancelled
			BS5862	07-06-2026	19:30	21:33	Kuantan	Larkin Sentral Johor Bahru	Kuala Lumpur	P5	6	Delayed
			BS4490	09-03-2026	13:49	20:33	Singapore	Sungai Petani Terminal	Penang	P7	21	Delayed
			BS2784	10-11-2025	13:15	15:20	Kuantan	Taipung Mamunting Terminal	Penang	P6	50	Completed
			BS2789	10-11-2025	16:55	23:22	Kuantan	Kuantan Sentral	Seremban	P2	25	Scheduled
			BS3649	11-06-2026	13:58	18:54	Sungai Petani	Kuala Terengganu MBKT	Kuala Lumpur	P8	43	Cancelled
			BS1805	11-09-2025	16:13	17:37	Kuala Lumpur	Sungai Petani Terminal	Batu Pahat	P3	14	Cancelled
			BS3198	11-12-2025	08:50	12:00	Butterworth	Penang Sungai Nibong	Kuantan	P8	7	Delayed
			BS3496	13-01-2026	07:32	09:28	Kuala Terengganu	Cameron Highlands Tanah Rat	Batu Pahat	P3	7	Cancelled

4.1.2 Query 2: Bus Maintenance Records by Company**Purpose:**

The purpose of this query is to provide a clear overview of bus service and maintenance activities, integrating data from bus companies, buses and service records. It highlights details such as service type, dates, costs and status. By limiting the results to services that are either in progress or completed, it focuses on actionable maintenance data. This enables management to track operational efficiency, control costs and monitor fleet upkeep effectively.

SQL statement:

```

CREATE OR REPLACE VIEW BusServiceRecordsView AS
SELECT
    bc.CompanyName          AS Company_Name,
    b.BusType               AS Bus_Type,
    b.PlateNumber           AS Bus_Plate,
    sd.ServiceTransactionID AS ServiceID,
    s.ServiceType           AS Service_Type,
    TO_CHAR(sd.ServiceDate, 'DD-MM-YYYY') AS Service_Date,
    TO_CHAR(sd.ServiceCompleteDate, 'DD-MM-YYYY') AS
Service_Complete_Date,
    s.ServiceCost           AS Service_Cost,
    sd.ServiceStatus        AS Status
FROM
    BusCompany bc
JOIN
    Bus b
    ON bc.CompanyID = b.CompanyID
JOIN
    ServiceDetails sd
    ON b.PlateNumber = sd.PlateNumber
JOIN
    Service s
    ON sd.ServiceID = s.ServiceID
WHERE
    sd.ServiceStatus IN ('In Progress', 'Completed');

SET PAGESIZE 100
SET LINESIZE 210
SET TRIMSPOOL ON

COLUMN Bus_Type          FORMAT A15
COLUMN Bus_Plate         FORMAT A12

```

```

COLUMN Company_Name          FORMAT A20
COLUMN ServiceID              FORMAT A10
COLUMN Service_Type           FORMAT A20
COLUMN Service_Date           FORMAT A12
COLUMN Service_Complete_Date  FORMAT A12
COLUMN Service_Cost           FORMAT 999999.99
COLUMN Status                 FORMAT A12

BREAK ON Company_Name SKIP 1 ON Bus_Type ON Bus_Plate

SELECT *
FROM BusServiceRecordsView
ORDER BY Bus_Type, Company_Name, Bus_Plate, Service_Date, ServiceID;

CLEAR COLUMN
CLEAR BREAK;
```

Sample Output:

COMPANY_NAME	BUS_TYPE	BUS_PLATE	SERVICEID	SERVICE_TYPE	SERVICE_DATE	SERVICE_COMP	SERVICE_COST	STATUS
TransCity Coaches	Single Deck	BU0017	ST0268	Electrical System Check	04-12-2023	05-12-2023	160.00	Pending
			ST0255	Lighting System Service	15-09-2023	19-09-2023	120.00	Pending
			ST0049	Wheel Alignment and Balancing	26-04-2023	02-05-2023	180.00	Pending
	Single Deck	BU0077	ST0248	Electrical System Check	18-05-2023	22-05-2023	160.00	Pending
			ST0017	Emergency Equipment Inspection	21-10-2024	26-10-2024	70.00	Pending
			ST0061	Door Mechanism Repair	30-01-2024	05-02-2024	100.00	In Progress
			ST0011	Lighting System Service	30-06-2023	01-07-2023	120.00	Pending

4.1.3 Procedure 1: Ticket Booking**Purpose:**

The purpose of this procedure is to handle the complete bus ticket booking process by validating member and passenger details, checking seat availability, reserving seats, recording payment, creating booking records and ensuring that no overbooking occurs. It also provides confirmation details, including booking ID, payment ID, passenger list and assigned seats.

Procedure code:

```

CREATE OR REPLACE PROCEDURE prc_make_ticket_booking (
    p_memberID          IN VARCHAR2,
    p_paymentMethod      IN VARCHAR2,
    p_bankAccount        IN VARCHAR2 DEFAULT NULL,
    p_busScheduleID     IN VARCHAR2,
    p_passengerICs       IN SYS.ODCIVARCHAR2LIST,
    p_passengerNames     IN SYS.ODCIVARCHAR2LIST
) IS
    v_memberExists       NUMBER;
    v_paymentID          VARCHAR2(6);
    v_bookingID          VARCHAR2(6);
    v_ticketPrice        NUMBER(10,2);
    v_totalPrice         NUMBER(10,2) := 0;
    v_ticketID           VARCHAR2(6);
    v_seatNo             VARCHAR2(3);
    v_availableSeat      NUMBER;
```

```

TYPE t_alloc IS RECORD (
    ticketID VARCHAR2(6),
    seatNo   VARCHAR2(3),
    ic       VARCHAR2(20),
    name     VARCHAR2(100)
);
TYPE t_alloc_list IS TABLE OF t_alloc INDEX BY PLS_INTEGER;
v_allocations t_alloc_list;
BEGIN
    -- Validate member
    SELECT COUNT(*) INTO v_memberExists
    FROM Member
    WHERE MemberID = p_memberID;

    IF v_memberExists = 0 THEN
        RAISE NO_DATA_FOUND;
    END IF;

    -- Validate array sizes
    IF p_passengerICs.COUNT != p_passengerNames.COUNT THEN
        RAISE_APPLICATION_ERROR(-20003, 'Passenger ICs and Names count
mismatch. ');
    END IF;

    -- Check available seats
    SELECT AvailableSeat INTO v_availableSeat
    FROM BusSchedule
    WHERE BusScheduleID = p_busScheduleID;

    IF v_availableSeat < p_passengerICs.COUNT THEN
        RAISE_APPLICATION_ERROR(
            -20004,
            'Insufficient available seats. Requested: ' ||
            p_passengerICs.COUNT || ', Available: ' || v_availableSeat
        );
    END IF;

    -- Pick tickets, mark as booked immediately, and calculate total
    FOR i IN 1 .. p_passengerICs.COUNT LOOP
        SELECT TicketID, SeatNo, TicketPrice
        INTO v_ticketID, v_seatNo, v_ticketPrice
        FROM (
            SELECT TicketID, SeatNo, TicketPrice
            FROM Ticket
            WHERE BusScheduleID = p_busScheduleID
            AND TicketStatus = 'Available'
            ORDER BY SeatNo
        )
        WHERE ROWNUM = 1;

        v_totalPrice := v_totalPrice + v_ticketPrice;

        -- Mark ticket as booked immediately
        UPDATE Ticket
        SET TicketStatus = 'Booked'
        WHERE TicketID = v_ticketID;

        -- Reduce available seats immediately
        UPDATE BusSchedule
        SET AvailableSeat = AvailableSeat - 1
        WHERE BusScheduleID = p_busScheduleID;

        v_allocations(i).ticketID := v_ticketID;
        v_allocations(i).seatNo   := v_seatNo;
        v_allocations(i).ic       := p_passengerICs(i);
        v_allocations(i).name     := p_passengerNames(i);
    END LOOP;

```

```

-- Insert Payment
INSERT INTO Payment (
    PaymentID, PaymentDate, PaymentMethod, PaymentStatus, BankAccount
)
VALUES (
    'P' || LPAD(Payment_seq.NEXTVAL, 5, '0'),
    SYSDATE,
    p_paymentMethod,
    'Completed',
    CASE
        WHEN UPPER(p_paymentMethod) = 'CASH' THEN NULL
        ELSE p_bankAccount
    END
)
RETURNING PaymentID INTO v_paymentID;

-- Insert Booking
INSERT INTO Booking (
    BookingID, PurchaseDate, BookingStatus, TotalPrice, MemberID,
    PaymentID
)
VALUES (
    'B' || LPAD(Booking_seq.NEXTVAL, 5, '0'),
    SYSDATE,
    'Completed',
    v_totalPrice,
    p_memberID,
    v_paymentID
)
RETURNING BookingID INTO v_bookingID;

-- Finalize BookingDetails (no more ticket updates here)
FOR i IN 1 .. v_allocations.COUNT LOOP
    INSERT INTO BookingDetails (
        BookingID, TicketID, PassengerIC, PassengerName,
        BookingDetailsStatus
    )
    VALUES (
        v_bookingID,
        v_allocations(i).ticketID,
        v_allocations(i).ic,
        v_allocations(i).name,
        'Completed'
    );
END LOOP;

COMMIT;

-- Success output
DBMS_OUTPUT.PUT_LINE('Ticket booking successful');
DBMS_OUTPUT.PUT_LINE('Booking ID      : ' || v_bookingID);
DBMS_OUTPUT.PUT_LINE('Payment ID      : ' || v_paymentID);
DBMS_OUTPUT.PUT_LINE('Member ID       : ' || p_memberID);
DBMS_OUTPUT.PUT_LINE('Bus Schedule    : ' || p_busScheduleID);
DBMS_OUTPUT.PUT_LINE('Passengers      : ' || v_allocations.COUNT);
DBMS_OUTPUT.PUT_LINE('Payment Method: ' || p_paymentMethod);
DBMS_OUTPUT.PUT_LINE('Payment Status: Completed');
DBMS_OUTPUT.PUT_LINE('Total Price     : RM ' ||
TO_CHAR(v_totalPrice, '999,999.99'));

DBMS_OUTPUT.PUT_LINE(CHR(10) || 'Seat Allocations:');
FOR i IN 1 .. v_allocations.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE(' - Passenger: ' || v_allocations(i).name || ' ('
||
        v_allocations(i).ic || ')' || ' -> Seat ' ||
        v_allocations(i).seatNo);

```

```

        END LOOP;

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            ROLLBACK;
            DBMS_OUTPUT.PUT_LINE('Member ' || p_memberID || ' missing from
records. ');
        WHEN OTHERS THEN
            ROLLBACK;
            DBMS_OUTPUT.PUT_LINE('Booking process failed: ' || SQLERRM);
    END;
/

```

Sample Output:

```

SQL> DECLARE
2     v_ICs SYS.ODCIVARCHAR2LIST := SYS.ODCIVARCHAR2LIST('900101-01-1234', '880202-02-5678');
3     v_Names SYS.ODCIVARCHAR2LIST := SYS.ODCIVARCHAR2LIST('Ali Bin Abu', 'Siti Binti Ahmad');
4 BEGIN
5     prc_make_ticket_booking(
6         p_memberID => 'M00001',
7         p_paymentMethod => 'OnlineBanking',
8         p_bankAccount => '123-456789-01',
9         p_busScheduleID => 'BS0001',
10        p_passengerICs => v_ICs,
11        p_passengerNames => v_Names
12    );
13 END;
14 /
Ticket booking successful
Booking ID      : B00268
Payment ID      : P00669
Member ID       : M00001
Bus Schedule    : BS0001
Passengers      : 2
Payment Method: OnlineBanking
Payment Status: Completed
Total Price     : RM      100.00

```

4.1.4 Procedure 2: Staff Allocation

Purpose:

The purpose of this procedure is to allocate a specific service task to a staff member for a given date and time slot. It validates that both the staff and service exist, prevents duplicate allocations on the same date and time and records the assignment in the staff allocation table. The procedure ensures accurate scheduling of staff resources while avoiding conflicts. It also provides confirmation output with details of the assigned staff, service, date and time slot.

Procedure code:

```

CREATE OR REPLACE PROCEDURE prc_allocate_work_to_staff (
    p_staffID    IN VARCHAR2,
    p_serviceID  IN VARCHAR2,
    p_timeSlot   IN VARCHAR2,
    p_allocDate  IN DATE DEFAULT SYSDATE
) IS
    v_staffName  Staff.StaffName%TYPE;
    v_serviceType Service.ServiceType%TYPE;
    v_duplicate   NUMBER;
BEGIN
    -- Check Staff exists + get name
    SELECT StaffName
    INTO v_staffName
    FROM Staff
    WHERE StaffID = p_staffID;

    -- Check Service exists + get type

```

```

SELECT ServiceType
INTO v_serviceType
FROM Service
WHERE ServiceID = p_serviceID;

-- Check if allocation already exists (compare only DATE part, ignore
time)
SELECT COUNT(*)
INTO v_duplicate
FROM StaffAllocation
WHERE StaffID = p_staffID
  AND TimeSlot = p_timeSlot
  AND TRUNC(AllocationDate) = TRUNC(NVL(p_allocDate, SYSDATE));

IF v_duplicate > 0 THEN
  DBMS_OUTPUT.PUT_LINE('Duplicate allocation: Staff ' || p_staffID
||
                        ' already allocated on ' ||
TO_CHAR(TRUNC(NVL(p_allocDate, SYSDATE)), 'DD-MM-YYYY') ||
                        ' at ' || p_timeSlot);
  RETURN;
END IF;

-- Insert allocation (keep full timestamp)
INSERT INTO StaffAllocation (ServiceID, StaffID, AllocationDate,
TimeSlot)
VALUES (p_serviceID, p_staffID, NVL(p_allocDate, SYSDATE),
p_timeSlot);

  DBMS_OUTPUT.PUT_LINE('Work allocated successfully:');
  DBMS_OUTPUT.PUT_LINE('  Staff ID   : ' || p_staffID || ' (' ||
v_staffName || ')');
  DBMS_OUTPUT.PUT_LINE('  Service ID : ' || p_serviceID || ' (' ||
v_serviceType || ')');
  DBMS_OUTPUT.PUT_LINE('  Date       : ' || TO_CHAR(NVL(p_allocDate,
SYSDATE), 'DD-MM-YYYY HH24:MI:SS'));
  DBMS_OUTPUT.PUT_LINE('  Time Slot  : ' || p_timeSlot);

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Staff ID ' || p_staffID || ' or Service ID '
|| p_serviceID || ' not found.');
```

Sample Output:

```

SQL> EXEC prc_allocate_work_to_staff('S00001', 'SV0005', 'Morning');
Work allocated successfully:
Staff ID   : S00001 (Alice Tan)
Service ID : SV0005 (Bus Washing n Cleaning)
Date       : 07-09-2025 19:53:52
Time Slot  : Morning
```

4.1.5 Trigger 1: Calculation of memberpoint when redeeming gift**Purpose:**

The purpose of this trigger is to manage the redemption of gifts by members, ensuring that redemptions only proceed when the gift is in stock and the member has sufficient points. It leverages a supporting function that checks whether a member's balance meets the point requirement for the selected gift. When conditions are satisfied, the trigger deducts the

required points from the member's balance and reduces the available stock of the gift. This ensures data integrity, prevents invalid redemptions and maintains accurate records of member points and gift inventory.

Trigger Code:

```
CREATE OR REPLACE TRIGGER trg_member_points_redemption
AFTER INSERT OR UPDATE ON PointRedemption
FOR EACH ROW
DECLARE
    v_required NUMBER := 0;
    v_stock    NUMBER := 0;
BEGIN
    -- Only when redemption is Collected
    IF :NEW.RedemptionStatus = 'Collected' THEN
        -- Get gift info
        SELECT PointRequired, StockRemaining
        INTO v_required, v_stock
        FROM Gift
        WHERE GiftID = :NEW.GiftID;

        -- ===== VALIDATION =====

        -- Check stock
        IF v_stock <= 0 THEN
            RAISE_APPLICATION_ERROR(-20002,
                'Gift ' || :NEW.GiftID || ' is out of stock.');
        END IF;

        -- Use function to check points
        IF fn_check_points_enough(:NEW.MemberID, :NEW.GiftID) = 'N' THEN
            RAISE_APPLICATION_ERROR(-20001,
                'Member ' || :NEW.MemberID ||
                ' does not have enough points to redeem gift ' ||
:NEW.GiftID);
        END IF;

        -- ===== ACTION =====
        UPDATE Member
        SET MemberPoints = MemberPoints - v_required
        WHERE MemberID = :NEW.MemberID;

        UPDATE Gift
        SET StockRemaining = StockRemaining - 1
        WHERE GiftID = :NEW.GiftID;
    END IF;
END;
/
```

Sample Output:

```
SQL> UPDATE Gift SET StockRemaining = 0 WHERE GiftID = 'GF0002';

1 row updated.

SQL> INSERT INTO PointRedemption (MemberID, GiftID, RedemptionDate, RedemptionStatus, UsedDate)
  2 VALUES ('M00100', 'GF0002', SYSDATE, 'Collected', SYSDATE);
INSERT INTO PointRedemption (MemberID, GiftID, RedemptionDate, RedemptionStatus, UsedDate)
      *
ERROR at line 1:
ORA-20002: Gift GF0002 is out of stock.
ORA-06512: at "STUDENT1.TRG_MEMBER_POINTS_REDEMPTION", line 17
ORA-04088: error during execution of trigger 'STUDENT1.TRG_MEMBER_POINTS_REDEMPTION'
```

```

SQL> UPDATE Gift SET StockRemaining = 0 WHERE GiftID = 'GF0002';

1 row updated.

SQL> INSERT INTO PointRedemption (MemberID, GiftID, RedemptionDate, RedemptionStatus, UsedDate)
  2 VALUES ('M00100', 'GF0002', SYSDATE, 'Collected', SYSDATE);
INSERT INTO PointRedemption (MemberID, GiftID, RedemptionDate, RedemptionStatus, UsedDate)
      *
ERROR at line 1:
ORA-20002: Gift GF0002 is out of stock.
ORA-06512: at "STUDENT1.TRG_MEMBER_POINTS_REDEMPTION", line 17
ORA-04088: error during execution of trigger 'STUDENT1.TRG_MEMBER_POINTS_REDEMPTION'

SQL> UPDATE Gift SET StockRemaining = 150 WHERE GiftID = 'GF0002';

1 row updated.

SQL> update member
  2 set memberpoints = 300
  3 where memberid = 'M00100';

1 row updated.

SQL> INSERT INTO PointRedemption (MemberID, GiftID, RedemptionDate, RedemptionStatus, UsedDate)
  2 VALUES ('M00100', 'GF0002', SYSDATE, 'Collected', SYSDATE);
INSERT INTO PointRedemption (MemberID, GiftID, RedemptionDate, RedemptionStatus, UsedDate)
      *
ERROR at line 1:
ORA-20001: Member M00100 does not have enough points to redeem gift GF0002
ORA-06512: at "STUDENT1.TRG_MEMBER_POINTS_REDEMPTION", line 23
ORA-04088: error during execution of trigger 'STUDENT1.TRG_MEMBER_POINTS_REDEMPTION'

```

4.1.6 Trigger 2: Gift Redemption Log

Purpose:

The purpose of this trigger is to maintain an audit trail of all point redemption activities. Whenever a record in the PointRedemption table is inserted, updated or deleted, the trigger automatically logs the action into the ActionPointRedemption table with details such as member ID, gift ID, redemption status, date, user, timestamp and the type of operation performed. This ensures accountability, supports tracking of historical changes and strengthens data integrity by capturing every modification for reporting and auditing purposes.

Trigger Code:

```

CREATE OR REPLACE TRIGGER trg_track_pointredemption
AFTER INSERT OR UPDATE OR DELETE ON PointRedemption
FOR EACH ROW
DECLARE
    v_logid VARCHAR2(10);
BEGIN
    -- Generate ID like LR0001
    SELECT 'LR' || LPAD(seq_redemption_log.NEXTVAL, 4, '0')
    INTO v_logid
    FROM dual;

    CASE
        WHEN INSERTING THEN
            INSERT INTO ActionPointRedemption(LogID, MemberID, GiftID,
            RedemptionStatus, RedemptionDate,
            UserID, TransDate,
            TransTime, TransAction)
            VALUES(v_logid, :NEW.MemberID, :NEW.GiftID,
            :NEW.REDEMPTIONSTATUS, :NEW.RedemptionDate,
            USER, SYSDATE, TO_CHAR(SYSDATE, 'HH24:MI:SS'),
            'INSERT');
    
```

```

        WHEN UPDATING THEN
            INSERT INTO ActionPointRedemption(LogID, MemberID, GiftID,
RedemptionStatus, RedemptionDate,
                                UserID, TransDate,
TransTime, TransAction)
            VALUES(v_logid, :NEW.MemberID, :NEW.GiftID,
:NEW.REDEMPTIONSTATUS, :NEW.RedemptionDate,
                                USER, SYSDATE, TO_CHAR(SYSDATE, 'HH24:MI:SS'),
'UPDATE');

        WHEN DELETING THEN
            INSERT INTO ActionPointRedemption(LogID, MemberID, GiftID,
RedemptionStatus, RedemptionDate,
                                UserID, TransDate,
TransTime, TransAction)
            VALUES(v_logid, :OLD.MemberID, :OLD.GiftID,
:OLD.REDEMPTIONSTATUS, :OLD.RedemptionDate,
                                USER, SYSDATE, TO_CHAR(SYSDATE, 'HH24:MI:SS'),
'DELETE');
    END CASE;
END;
/

```

Sample Output:

```
SQL> INSERT INTO PointRedemption(MemberID, GiftID, RedemptionDate, RedemptionStatus)
2 VALUES('M00100', 'GF0002', SYSDATE, 'Collected');
```

1 row created.

```
SQL> INSERT INTO PointRedemption(MemberID, GiftID, RedemptionDate, RedemptionStatus)
2 VALUES('M00100', 'GF0002', SYSDATE, 'Pending');
```

1 row created.

```
SQL> UPDATE PointRedemption
2 SET RedemptionStatus = 'Collected',
3     UsedDate = SYSDATE
4 WHERE MemberID = 'M00100'
5     AND GiftID = 'GF0002'
6     AND RedemptionStatus = 'Pending';
```

1 row updated.

```
SQL> SELECT LogID, MemberID, GiftID, TransAction, TransDate, TransTime
2 FROM ActionPointRedemption;
```

LOGID	MEMBERID	GIFTID	TRANSACTION	TRANSDATE	TRANSTIME
LR0001	M00100	GF0002	INSERT	06-SEP-25	22:12:06
LR0002	M00100	GF0002	UPDATE	06-SEP-25	22:13:06

4.1.7 Report 1: Yearly Revenue Trend Report**Purpose:**

The purpose of this report is to present yearly revenue trends for bus companies based on completed bookings. It calculates total revenue per company for each year and then provides grand totals across all years. This allows management to compare company performance over time and identify key contributors to overall revenue. By consolidating the data into a structured report, it supports strategic planning and decision-making.

Report Code:

```

CREATE OR REPLACE PROCEDURE prc_yearly_revenue_trend IS
    v_line_width CONSTANT NUMBER := 50;

    -- Cursor to get distinct years
    CURSOR yrCursor IS
        SELECT DISTINCT TO_CHAR(b.PurchaseDate, 'YYYY') AS YearPeriod
        FROM Booking b
        JOIN Payment p ON b.PaymentID = p.PaymentID AND p.PaymentStatus =
'Completed'
        ORDER BY TO_CHAR(b.PurchaseDate, 'YYYY');

    -- Cursor to get company revenue by year
    CURSOR compCursor(p_year VARCHAR2) IS
        SELECT bc.CompanyName, SUM(b.TotalPrice) AS TotalRevenue
        FROM Booking b
        JOIN Payment p ON b.PaymentID = p.PaymentID AND p.PaymentStatus =
'Completed'
        JOIN BusSchedule bs ON b.BookingID IS NOT NULL
        JOIN Bus bus ON bs.PlateNumber = bus.PlateNumber
        JOIN BusCompany bc ON bus.CompanyID = bc.CompanyID
        WHERE TO_CHAR(b.PurchaseDate, 'YYYY') = p_year
        GROUP BY bc.CompanyName;

    -- Cursor to get grand totals by company across all years
    CURSOR grandCursor IS
        SELECT bc.CompanyName, SUM(b.TotalPrice) AS TotalRevenue
        FROM Booking b
        JOIN Payment p ON b.PaymentID = p.PaymentID AND p.PaymentStatus =
'Completed'
        JOIN BusSchedule bs ON b.BookingID IS NOT NULL
        JOIN Bus bus ON bs.PlateNumber = bus.PlateNumber
        JOIN BusCompany bc ON bus.CompanyID = bc.CompanyID
        GROUP BY bc.CompanyName
        ORDER BY bc.CompanyName;

    v_yearRec          yrCursor%ROWTYPE;
    v_compRec          compCursor%ROWTYPE;
    v_grandRec         grandCursor%ROWTYPE;

    v_yearTotal        NUMBER;
    v_totalRevenue     NUMBER := 0;
BEGIN
    DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));
    DBMS_OUTPUT.PUT_LINE('YEARLY REVENUE TREND REPORT');
    DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));

    OPEN yrCursor;
    LOOP
        FETCH yrCursor INTO v_yearRec;
        EXIT WHEN yrCursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(CHR(10) || '>>> Year: ' ||
v_yearRec.YearPeriod);
        DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));
        DBMS_OUTPUT.PUT_LINE(RPAD('Company', 30) || RPAD('Revenue
(RM)', 20));
        DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));

        v_yearTotal := 0;

        OPEN compCursor(v_yearRec.YearPeriod);
        LOOP
            FETCH compCursor INTO v_compRec;
            EXIT WHEN compCursor%NOTFOUND;

            DBMS_OUTPUT.PUT_LINE(RPAD(v_compRec.CompanyName, 30) ||

```

```

                                RPAD('RM ' ||
TO_CHAR(v_compRec.TotalRevenue, '999,999,999.99'),20));

        v_yearTotal := v_yearTotal + v_compRec.TotalRevenue;
    END LOOP;
    CLOSE compCursor;

    DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));
    DBMS_OUTPUT.PUT_LINE(RPAD('TOTAL ' || v_yearRec.YearPeriod,30) ||
                                RPAD('RM ' ||
TO_CHAR(v_yearTotal, '999,999,999.99'),20));

        v_totalRevenue := v_totalRevenue + v_yearTotal;
    END LOOP;
    CLOSE yrCursor;

    -- Grand totals by company across all years
    DBMS_OUTPUT.PUT_LINE(CHR(10) || 'GRAND TOTALS BY COMPANY');
    DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));
    DBMS_OUTPUT.PUT_LINE(RPAD('Company',30) || RPAD('Total Revenue
(RM)',20));
    DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));

    OPEN grandCursor;
    LOOP
        FETCH grandCursor INTO v_grandRec;
        EXIT WHEN grandCursor%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(RPAD(v_grandRec.CompanyName,30) ||
                                RPAD('RM ' ||
TO_CHAR(v_grandRec.TotalRevenue, '999,999,999.99'),20));
    END LOOP;
    CLOSE grandCursor;

    DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));
    DBMS_OUTPUT.PUT_LINE(RPAD('TOTAL REVENUE ALL COMPANIES',30) ||
                                RPAD('RM ' ||
TO_CHAR(v_totalRevenue, '999,999,999.99'),20));
    DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));
    DBMS_OUTPUT.PUT_LINE(LPAD('=',18,'=') || RPAD('END OF REPORT',32,'=')
|| CHR(10));
    DBMS_OUTPUT.PUT_LINE('Generated at: ' || TO_CHAR(SYSDATE, 'DD-MM-YYYY
HH24:MI:SS'));
END;
/

EXEC prc_yearly_revenue_trend;

```

Sample Output:

YEARLY REVENUE TREND REPORT			GRAND TOTALS BY COMPANY		
>>> Year: 2022			Company		
Company	Revenue (RM)		Total Revenue (RM)		
Sunrise Express	RM	3,268,254.00	BlueSky Coaches	RM	22,379,372.56
Galaxy Busline	RM	2,605,130.00	CityLink Travels	RM	16,876,248.16
Speedway Coaches	RM	1,930,164.50	EcoRide Coaches	RM	17,609,998.08
CityLink Travels	RM	2,178,836.00	Galaxy Busline	RM	20,178,122.80
Orbit Express	RM	1,936,085.25	Golden Way Travels	RM	14,996,013.99
Rapid Travels	RM	2,682,099.75	Greenway Transport	RM	19,673,669.73
Silverline Transport	RM	1,912,402.25	Mega Transit	RM	19,444,372.88
RapidLink Express	RM	1,782,145.75	Metro Bus Services	RM	24,351,325.47
Golden Way Travels	RM	1,936,085.25	NextBus Travel	RM	16,142,498.24
Royal Express	RM	3,244,571.00	Orbit Express	RM	14,996,013.99
Unity Express	RM	3,303,778.50	PrimeLine Travels	RM	26,139,840.90
Mega Transit	RM	2,510,398.00	Rapid Travels	RM	20,774,294.61
EcoRide Coaches	RM	2,273,568.00	RapidLink Express	RM	13,803,670.37
TransCity Coaches	RM	2,948,533.50	Royal Express	RM	25,130,934.76
NextBus Travel	RM	2,084,104.00	Silverline Transport	RM	14,812,576.51
PrimeLine Travels	RM	3,374,827.50	Speedway Coaches	RM	14,950,154.62
Metro Bus Services	RM	3,143,918.25	Starline Express	RM	23,892,731.77
Greenway Transport	RM	2,540,001.75	Sunrise Express	RM	25,314,372.24
BlueSky Coaches	RM	2,889,326.00	TransCity Coaches	RM	22,837,966.26
Starline Express	RM	3,084,710.75	Unity Express	RM	25,589,528.46
TOTAL 2022	RM	51,628,940.00	TOTAL REVENUE ALL COMPANIES	RM	399,893,706.40
			Generated at: 07-09-2025 20:54:27		

4.1.8 Report 2: Maintenance Cost vs Ticket Revenue Report

Purpose:

The purpose of this report is to compare ticket revenue against maintenance costs for each bus. It calculates revenue, cost and net profitability per bus and then summarizes the totals across all buses. This enables management to identify the most and least profitable buses, optimize maintenance spending and support strategic decisions on fleet performance. By consolidating revenue and cost information, the report strengthens financial transparency and operational efficiency.

Report Code:

```
CREATE OR REPLACE PROCEDURE prc_bus_profitability IS
    v_line_width CONSTANT NUMBER := 90;

    CURSOR busCursor IS
        SELECT PlateNumber FROM Bus ORDER BY PlateNumber;

    CURSOR busDetailCursor(p_plate Bus.PlateNumber%TYPE) IS
        SELECT SUM(b.TotalPrice) AS TicketRevenue,
               NVL(SUM(sv.ServiceCost),0) AS MaintenanceCost,
               SUM(b.TotalPrice) - NVL(SUM(sv.ServiceCost),0) AS
Profitability
        FROM Bus bus
        LEFT JOIN BusSchedule bs ON bus.PlateNumber = bs.PlateNumber
        LEFT JOIN Ticket t ON bs.BusScheduleID = t.BusScheduleID
        LEFT JOIN BookingDetails bd ON t.TicketID = bd.TicketID
        LEFT JOIN Booking b ON bd.BookingID = b.BookingID
        LEFT JOIN ServiceDetails sd ON bus.PlateNumber = sd.PlateNumber
        LEFT JOIN Service sv ON sd.ServiceID = sv.ServiceID
        JOIN Payment p ON b.PaymentID = p.PaymentID AND p.PaymentStatus =
'Completed'
        WHERE bus.PlateNumber = p_plate
        GROUP BY bus.PlateNumber;

    v_busRec          busCursor%ROWTYPE;
    v_detRec          busDetailCursor%ROWTYPE;

    v_totalRevenue NUMBER := 0;
```

```

v_totalCost    NUMBER := 0;
v_totalProfit  NUMBER := 0;
BEGIN
  DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));
  DBMS_OUTPUT.PUT_LINE('MAINTENANCE COST VS TICKET REVENUE REPORT');
  DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));

  DBMS_OUTPUT.PUT_LINE(RPAD('Bus Plate',20) ||
                        RPAD('Revenue ',25) ||
                        RPAD('Cost ',25) ||
                        RPAD('Profit ',20));
  DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));

  OPEN busCursor;
  LOOP
    FETCH busCursor INTO v_busRec;
    EXIT WHEN busCursor%NOTFOUND;

    OPEN busDetailCursor(v_busRec.PlateNumber);
    FETCH busDetailCursor INTO v_detRec;

    IF busDetailCursor%FOUND THEN
      DBMS_OUTPUT.PUT_LINE(RPAD(v_busRec.PlateNumber,20) ||
                          RPAD('RM ' ||
TO_CHAR(v_detRec.TicketRevenue,'999,999,999.99'),25) ||
                          RPAD('RM ' ||
TO_CHAR(v_detRec.MaintenanceCost,'999,999,999.99'),25) ||
                          RPAD('RM ' ||
TO_CHAR(v_detRec.Profitability,'999,999,999.99'),25));

      v_totalRevenue := v_totalRevenue +
NVL(v_detRec.TicketRevenue,0);
      v_totalCost    := v_totalCost +
NVL(v_detRec.MaintenanceCost,0);
      v_totalProfit  := v_totalProfit +
NVL(v_detRec.Profitability,0);
    END IF;

    CLOSE busDetailCursor;
  END LOOP;
  CLOSE busCursor;

  -- Footer
  DBMS_OUTPUT.PUT_LINE('');
  DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));
  DBMS_OUTPUT.PUT_LINE(RPAD('TOTAL (ALL BUSES)',20) ||
                        RPAD('RM ' ||
TO_CHAR(v_totalRevenue,'999,999,999.99'),18));
  DBMS_OUTPUT.PUT_LINE(RPAD('TOTAL COST',20) ||
                        RPAD('RM ' ||
TO_CHAR(v_totalCost,'999,999,999.99'),18));
  DBMS_OUTPUT.PUT_LINE(RPAD('TOTAL PROFIT',20) ||
                        RPAD('RM ' ||
TO_CHAR(v_totalProfit,'999,999,999.99'),18));
  DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));
  DBMS_OUTPUT.PUT_LINE(LPAD('=',38,'=') || RPAD('END OF REPORT',52,'=')
|| CHR(10));
  DBMS_OUTPUT.PUT_LINE('Generated at: ' || TO_CHAR(SYSDATE, 'DD-MM-YYYY
HH24:MI:SS'));
END;
/

EXEC prc_bus_profitability;

```

Sample Output:

=====				=====			
MAINTENANCE COST VS TICKET REVENUE REPORT							
Bus Plate	Revenue	Cost	Profit				
=====							
BU0001	RM 8,352.00	RM 3,000.00	RM 5,352.00	BU0062	RM 11,000.00	RM 8,500.00	RM 2,500.00
BU0002	RM 22,140.00	RM 10,900.00	RM 11,240.00	BU0063	RM 2,750.00	RM 3,500.00	RM -750.00
BU0003	RM 3,900.00	RM 1,900.00	RM 1,920.00	BU0064	RM 0,250.00	RM 9,200.00	RM -950.00
BU0004	RM 226.00	RM 800.00	RM -573.20	BU0065	RM 11,000.00	RM 6,400.00	RM 4,600.00
BU0005	RM 20.50	RM 100.00	RM -71.50	BU0066	RM 11,000.00	RM 8,000.00	RM 3,000.00
BU0006	RM 442.50	RM 1,650.00	RM -1,207.50	BU0067	RM 13,750.00	RM 11,500.00	RM 2,250.00
BU0009	RM 630.00	RM 2,250.00	RM -1,620.00	BU0068	RM 5,500.00	RM 3,000.00	RM 2,500.00
BU0010	RM 719.92	RM 1,800.00	RM -1,080.08	BU0069	RM 13,750.00	RM 4,600.00	RM 9,150.00
BU0011	RM 2,240.00	RM 5,100.00	RM -2,860.00	BU0070	RM 5,500.00	RM 4,500.00	RM 1,000.00
BU0012	RM 1,104.00	RM 1,280.00	RM -176.00	BU0071	RM 11,000.00	RM 10,900.00	RM 100.00
BU0013	RM 1,910.40	RM 6,720.00	RM -4,809.60	BU0072	RM 16,500.00	RM 5,100.00	RM 11,400.00
BU0016	RM 1,199.00	RM 2,190.00	RM -990.12	BU0073	RM 2,750.00	RM 500.00	RM 2,250.00
BU0018	RM 744.40	RM 1,190.00	RM -445.60	BU0074	RM 13,750.00	RM 10,200.00	RM 3,550.00
BU0019	RM 593.40	RM 1,140.00	RM -546.60	BU0075	RM 5,500.00	RM 4,000.00	RM 1,500.00
BU0020	RM 935.96	RM 1,650.00	RM -714.04	BU0076	RM 5,500.00	RM 2,700.00	RM 2,800.00
BU0026	RM 12,100.00	RM 4,700.00	RM 7,400.00	BU0077	RM 11,000.00	RM 4,500.00	RM 6,500.00
BU0027	RM 6,000.00	RM 6,100.00	RM -500.00	BU0078	RM 5,500.00	RM 6,400.00	RM -900.00
BU0028	RM 12,650.00	RM 9,900.00	RM 2,750.00	BU0079	RM 2,750.00	RM 2,000.00	RM 750.00
BU0029	RM 9,600.00	RM 10,000.00	RM -400.00	BU0080	RM 22,000.00	RM 11,300.00	RM 10,700.00
BU0030	RM 6,000.00	RM 7,500.00	RM -900.00	BU0081	RM 16,500.00	RM 4,000.00	RM 12,500.00
BU0031	RM 5,060.00	RM 2,700.00	RM 2,360.00	BU0082	RM 40,000.00	RM 13,200.00	RM 26,800.00
BU0032	RM 3,740.00	RM 2,000.00	RM 1,740.00	BU0083	RM 22,000.00	RM 10,500.00	RM 11,500.00
BU0033	RM 11,550.00	RM 10,400.00	RM 1,150.00	BU0084	RM 22,000.00	RM 6,500.00	RM 15,500.00
BU0034	RM 7,920.00	RM 8,100.00	RM -180.00	BU0085	RM 11,000.00	RM 3,700.00	RM 7,300.00
BU0035	RM 1,900.00	RM 1,200.00	RM 700.00	BU0086	RM 16,500.00	RM 3,600.00	RM 12,900.00
BU0036	RM 5,010.00	RM 4,400.00	RM 610.00	BU0087	RM 16,500.00	RM 5,600.00	RM 10,900.00
BU0037	RM 2,530.00	RM 0.00	RM 2,530.00	BU0088	RM 16,500.00	RM 3,700.00	RM 12,800.00
BU0038	RM 10,450.00	RM 8,900.00	RM 1,550.00	BU0089	RM 11,000.00	RM 3,700.00	RM 7,300.00
BU0039	RM 11,000.00	RM 8,100.00	RM 2,900.00	BU0090	RM 16,500.00	RM 8,000.00	RM 8,500.00
BU0040	RM 2,060.00	RM 2,000.00	RM 60.00	BU0091	RM 11,000.00	RM 4,000.00	RM 7,000.00
BU0041	RM 9,350.00	RM 5,200.00	RM 4,150.00	BU0092	RM 5,500.00	RM 500.00	RM 5,000.00
BU0042	RM 3,520.00	RM 4,400.00	RM -880.00	BU0093	RM 16,500.00	RM 8,500.00	RM 8,000.00
BU0043	RM 0.50	RM 6,100.00	RM -6,099.50	BU0094	RM 22,000.00	RM 8,100.00	RM 13,900.00
BU0044	RM 10,120.00	RM 7,200.00	RM 2,920.00	BU0095	RM 11,000.00	RM 2,600.00	RM 8,400.00
BU0045	RM 5,610.00	RM 4,800.00	RM 810.00	BU0096	RM 22,000.00	RM 7,600.00	RM 14,400.00
BU0046	RM 1,430.00	RM 3,500.00	RM -2,070.00	BU0097	RM 30,500.00	RM 10,700.00	RM 19,800.00
BU0047	RM 3,025.00	RM 4,500.00	RM -1,475.00	BU0098	RM 5,500.00	RM 2,000.00	RM 3,500.00
BU0048	RM 1,900.00	RM 1,800.00	RM 100.00	BU0099	RM 22,000.00	RM 12,300.00	RM 9,700.00
BU0049	RM 6,000.00	RM 4,500.00	RM 1,500.00	BU0100	RM 16,500.00	RM 3,200.00	RM 13,300.00
BU0050	RM 4,100.00	RM 4,000.00	RM 100.00	=====			
BU0051	RM 11,000.00	RM 3,700.00	RM 7,300.00	=====END OF REPORT=====			
BU0052	RM 5,500.00	RM 6,000.00	RM -500.00				

4.2 Ethel Ng Yi Yan

4.2.1 Query 1: Query of Booking Details for Each Booking

Purpose:

The purpose of this query is to provide a complete and structured overview of bus bookings, integrating data from bookings, booking details, tickets and bus schedules. It delivers a detailed record for each passenger, including their name, seat number, ticket type, ticket price, departure and arrival stations and the exact date and time of travel. This comprehensive report enables management to monitor booking activity, evaluate revenue generated per booking and per route and gain insights into passenger distribution across different routes. Furthermore, it supports strategic decision-making by identifying high-demand routes, analyzing travel patterns and assessing operational efficiency. The inclusion of aggregated totals alongside individual booking details allows for precise financial reporting, resource planning and informed marketing initiatives, ensuring that business decisions are grounded in accurate and actionable data.

SQL statement:

```

SET LINESIZE 180;
SET PAGESIZE 50;
ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MM-YYYY';

CREATE OR REPLACE VIEW Bus_Booking_Info AS
SELECT
    b.BookingID,
    bs.DepartureDate,
    TO_CHAR(bs.DepartureTime, 'HH:MI AM') AS DepartureTime,
    bs.DepartBusStation,
    bs.ArrivalBusStation,
    bd.PassengerName,
    t.SeatNo,
    t.TicketType,
    t.TicketPrice
FROM
    Booking b
JOIN BookingDetails bd ON b.BookingID = bd.BookingID
JOIN Ticket t ON bd.TicketID = t.TicketID
JOIN BusSchedule bs ON t.BusScheduleID = bs.BusScheduleID
ORDER BY b.BookingID;

TTITLE LEFT '<<-Detailed Bus Booking Information->>' SKIP 2

COLUMN BookingID          FORMAT A20          HEADING 'Booking ID'
COLUMN DepartureDate       FORMAT A15           HEADING 'Departure Date'
COLUMN DepartureTime       FORMAT A15           HEADING 'Departure Time'
COLUMN DepartBusStation    FORMAT A20           HEADING 'Departure Station'
COLUMN ArrivalBusStation   FORMAT A20           HEADING 'Arrival Station'
COLUMN PassengerName       FORMAT A20           HEADING 'Passenger Name'
COLUMN SeatNo              FORMAT A10           HEADING 'Seat No'
COLUMN TicketType          FORMAT A15           HEADING 'Ticket Type'
COLUMN TicketPrice         FORMAT $9,999.99     HEADING 'Ticket Price'

BREAK ON BookingID SKIP 2
      ON DepartureDate
      ON DepartureTime
      ON DepartBusStation
      ON ArrivalBusStation

COMPUTE COUNT LABEL 'Total Passengers: ' -
SUM      LABEL 'Total Price: ' -
OF TicketPrice ON BookingID

```

```
SELECT * FROM Bus_Booking_Info;  
  
CLEAR COLUMNS;  
CLEAR BREAKS;  
CLEAR COMPUTES;  
TTITLE OFF;
```

Sample Output:

<<-Detailed Bus Booking Information-->								
Booking ID	Departure Date	Departure Time	Departure Station	Arrival Station	Passenger Name	Seat No	Ticket Type	Ticket Price
B00001	12-01-2023	08:00 AM	Kuala Lumpur	Penang Sentral	Tina Hughes	A1	Adult	\$50.00
	12-01-2023	08:00 AM	Kuala Lumpur	Penang Sentral	Matthew Adams	A3	Adult	\$50.00
	12-01-2023	08:00 AM	Kuala Lumpur	Penang Sentral	Jordan Alvarado	A4	Adult	\$50.00
	12-01-2023	08:00 AM	Kuala Lumpur	Penang Sentral	Katie Smith	A5	Adult	\$50.00
	12-01-2023	08:00 AM	Kuala Lumpur	Penang Sentral	Kristen Jenkins	A2	Adult	\$50.00
*****								5
Total Passengers:								3
Total Price:								\$250.00
B00002	12-01-2023	08:00 AM	Kuala Lumpur	Penang Sentral	Jeffrey Weaver	B2	Child	\$50.00
	12-01-2023	08:00 AM	Kuala Lumpur	Penang Sentral	Jonathan Barron	B1	Adult	\$50.00
	12-01-2023	08:00 AM	Kuala Lumpur	Penang Sentral	Ana Williams	B3	Adult	\$50.00
*****								3
Total Passengers:								3
Total Price:								\$150.00
B00003	12-01-2023	08:00 AM	Kuala Lumpur	Penang Sentral	Becky Walter	B5	Adult	\$50.00
	12-01-2023	08:00 AM	Kuala Lumpur	Penang Sentral	Hayley Hawkins	B4	Child	\$50.00
*****								2
Total Passengers:								2
Total Price:								\$100.00
B00004	12-01-2023	08:00 AM	Kuala Lumpur	Penang Sentral	Michael Moreno	C1	Adult	\$50.00
	12-01-2023	08:00 AM	Kuala Lumpur	Penang Sentral	Mark Barnes	C2	Adult	\$50.00
	12-01-2023	08:00 AM	Kuala Lumpur	Penang Sentral	Autumn Wright	C3	Adult	\$50.00
	12-01-2023	08:00 AM	Kuala Lumpur	Penang Sentral	Julia Campos	C4	Adult	\$50.00
*****								4
Total Passengers:								4
Total Price:								\$200.00

4.2.2 Query 2: Query for Distribution of Driver Allocation**Purpose:**

The purpose of this query is to provide a detailed overview of driver assignments across all bus schedules, integrating information from driver records, driver allocation lists, bus schedules, buses and bus companies. It delivers a structured report showing which driver is assigned to which bus schedule, their role such as the main driver or the assistant driver, departure date and time, departure station and the associated bus company. This comprehensive report allows management to monitor driver workload, ensure fair and efficient distribution of assignments and track driver participation across routes and schedules. The inclusion of aggregated totals, such as the number of assignments per driver, supports operational planning, compliance with labor regulations and helps identify potential staffing gaps. By providing visibility into driver allocation, this report aids in optimizing scheduling, improving operational efficiency and ensuring that drivers are assigned in alignment with company policies and route demands.

SQL statement:

```

SET SERVEROUTPUT ON;
SET LINESIZE 180;
SET PAGESIZE 50;

CREATE OR REPLACE VIEW Driver_Allocation AS
SELECT
    d.DriverID,
    d.DriverName,
    dl.BusScheduleID,
    dl.DriverRole,
    bs.DepartureDate,
    TO_CHAR(bs.DepartureTime, 'HH24:MI') AS DepartureTime,
    bs.DepartBusStation
FROM DriverList dl
JOIN Driver d ON dl.DriverID = d.DriverID
JOIN BusSchedule bs ON dl.BusScheduleID = bs.BusScheduleID
JOIN Bus b ON bs.PlateNumber = b.PlateNumber
ORDER BY d.DriverID, bs.DepartureDate;

TTITLE LEFT '<<-Driver Work Allocation-->' SKIP 2

COLUMN DriverID FORMAT A25 HEADING 'Driver ID'
COLUMN DriverName FORMAT A25 HEADING 'Driver Name'

```

```

COLUMN BusScheduleID FORMAT A15 HEADING 'Bus Schedule ID'
COLUMN DriverRole FORMAT A20 HEADING 'Driver Role'
COLUMN DepartureDate FORMAT A15 HEADING 'Departure Date'
COLUMN DepartureTime FORMAT A15 HEADING 'Departure Time'
COLUMN DepartBusStation FORMAT A20 HEADING 'Departure Station'

BREAK ON DriverID SKIP 2 ON DriverName
COMPUTE COUNT LABEL 'Total Assignments: ' OF BusScheduleID ON DriverID

-- Display results
SELECT * FROM Driver_Allocation;

CLEAR COLUMNS;
CLEAR BREAKS;
CLEAR COMPUTES;
TTITLE OFF;

```

Sample Output:

<<-Driver Work Allocation-->						
Driver ID	Driver Name	Bus Schedule ID	Driver Role	Departure Date	Departure Time	Departure Station
D00078	Nabila Hanim	BS0280	Assistant Driver	05-07-2025	16:16	Sungai Petani
		BS0317	Main Driver	06-07-2025	09:43	Batu Pahat
		BS0323	Assistant Driver	07-07-2025	08:13	Singapore
		BS0387	Assistant Driver	09-07-2025	10:35	Taiping
		BS0420	Assistant Driver	10-07-2025	20:01	Kuala Terengganu
		BS0415	Main Driver	10-07-2025	13:45	Kuala Terengganu
		BS0478	Assistant Driver	12-07-2025	16:32	Cameron Highlands
		BS0518	Assistant Driver	13-07-2025	08:22	Ipoh
		BS0547	Assistant Driver	14-07-2025	12:12	Taiping
		BS0601	Assistant Driver	16-07-2025	06:28	Kuala Terengganu
		BS0650	Assistant Driver	17-07-2025	19:55	Taiping
		BS0640	Main Driver	17-07-2025	12:05	Sungai Petani
		BS0654	Assistant Driver	18-07-2025	20:23	Kota Bharu
		BS0837	Main Driver	24-07-2025	09:51	Singapore
		BS0831	Assistant Driver	24-07-2025	18:38	Kuantan
		BS0919	Main Driver	26-07-2025	18:22	Kota Bharu
		BS0947	Assistant Driver	27-07-2025	15:08	Malacca
		BS0987	Main Driver	29-07-2025	11:20	Shah Alam
		BS1022	Assistant Driver	30-07-2025	12:17	Penang
		BS1011	Assistant Driver	30-07-2025	07:37	Butterworth
		BS1047	Assistant Driver	31-07-2025	16:56	Sungai Petani
		BS1254	Assistant Driver	01-08-2025	19:42	Hatyai
		BS1399	Main Driver	04-08-2025	16:51	Sungai Petani
		BS1385	Assistant Driver	04-08-2025	18:03	Shah Alam
		BS1391	Assistant Driver	04-08-2025	08:26	Cameron Highlands
		BS1283	Main Driver	06-08-2025	10:52	Kuala Terengganu
		BS1304	Assistant Driver	06-08-2025	19:08	Seremban
		BS1492	Main Driver	07-08-2025	15:41	Johor Bahru
		BS1514	Main Driver	07-08-2025	17:26	Johor Bahru
		BS1438	Assistant Driver	13-08-2025	12:12	Penang
		BS1219	Assistant Driver	25-08-2025	15:48	Sungai Petani
		BS1352	Assistant Driver	26-08-2025	16:50	Butterworth
		BS1546	Assistant Driver	01-09-2025	15:25	Shah Alam
		BS1582	Main Driver	05-09-2025	20:59	Cameron Highlands
		BS1770	Main Driver	06-09-2025	15:49	Penang
		BS1804	Assistant Driver	11-09-2025	12:43	Alor Setar
		BS1794	Main Driver	11-09-2025	20:25	Penang
		BS1963	Main Driver	15-09-2025	17:47	Hatyai
		BS1967	Main Driver	15-09-2025	15:58	Cameron Highlands
		BS1632	Assistant Driver	20-09-2025	08:31	Penang
		BS1557	Assistant Driver	23-09-2025	14:58	Butterworth
		BS1751	Assistant Driver	26-09-2025	07:40	Johor Bahru
		BS0813	Main Driver	07-10-2025	07:25	Kuala Lumpur
		BS1971	Assistant Driver	09-10-2025	19:57	Shah Alam
		BS2206	Assistant Driver	12-10-2025	13:42	Cameron Highlands
Driver ID	Driver Name	Bus Schedule ID	Driver Role	Departure Date	Departure Time	Departure Station
D00078	Nabila Hanim	BS2277	Main Driver	15-10-2025	18:40	Kuala Terengganu
		BS2279	Main Driver	15-10-2025	15:43	Kuala Lumpur
		BS2299	Assistant Driver	15-10-2025	20:25	Kuala Lumpur
		BS2130	Assistant Driver	16-10-2025	17:13	Butterworth
		BS2123	Assistant Driver	16-10-2025	08:32	Shah Alam
		BS2167	Assistant Driver	29-10-2025	06:41	Kuantan
		BS2781	Main Driver	05-11-2025	11:02	Seremban
		BS2779	Assistant Driver	15-11-2025	15:04	Ipoh
		BS2752	Assistant Driver	15-11-2025	15:40	Malacca
		BS2569	Main Driver	16-11-2025	12:26	Shah Alam
		BS2539	Assistant Driver	18-11-2025	20:15	Kuala Lumpur
		BS2522	Main Driver	18-11-2025	15:06	Shah Alam
		BS2449	Assistant Driver	21-11-2025	08:30	Kuantan
		BS2427	Assistant Driver	21-11-2025	20:03	Kuala Lumpur
		BS2422	Main Driver	21-11-2025	10:29	Kota Bharu
		BS2646	Main Driver	26-11-2025	15:44	Sungai Petani
		BS2632	Assistant Driver	26-11-2025	18:56	Seremban
		BS2616	Assistant Driver	28-11-2025	14:18	Sungai Petani
		BS2742	Main Driver	30-11-2025	19:12	Johor Bahru
		BS3287	Main Driver	06-12-2025	13:16	Sungai Petani
		BS3172	Assistant Driver	11-12-2025	17:04	Batu Pahat
		BS2913	Assistant Driver	12-12-2025	18:23	Alor Setar
		BS3121	Main Driver	15-12-2025	17:20	Sungai Petani
		BS3123	Assistant Driver	15-12-2025	06:22	Alor Setar
		BS3239	Assistant Driver	19-12-2025	14:54	Sungai Petani
		BS3234	Assistant Driver	19-12-2025	06:52	Ipoh
		BS3164	Main Driver	23-12-2025	07:45	Penang
		BS2972	Main Driver	25-12-2025	17:01	Kuala Terengganu
		BS3085	Assistant Driver	31-12-2025	15:35	Kuala Terengganu
		BS3510	Main Driver	05-01-2026	20:33	Kuala Lumpur
		BS3522	Assistant Driver	05-01-2026	08:06	Kuala Terengganu
		BS3559	Main Driver	15-01-2026	08:37	Seremban
		BS3667	Assistant Driver	16-01-2026	14:56	Singapore

Driver ID	Driver Name	Bus Schedule ID	Driver Role	Departure Date	Departure Time	Departure Station
D00078	Nabila Hanim	BS8449	Main Driver	12-12-2026	19:37	Kuala Lumpur
		BS8556	Main Driver	13-12-2026	11:33	Kuala Terengganu
		BS8595	Assistant Driver	14-12-2026	18:06	Kuala Terengganu
		BS8676	Assistant Driver	16-12-2026	10:54	Shah Alam
		BS8716	Main Driver	21-12-2026	15:26	Shah Alam
		BS8631	Main Driver	30-12-2026	09:19	Ipoh
		BS8660	Main Driver	30-12-2026	13:03	Batu Pahat

Total Assignments:		196				

4.2.3 Procedure 1: Advanced Search for Bus Schedule

Purpose:

The purpose of the procedure is to allow users to search for bus schedules based on various criteria such as bus plate number, bus company, destination, departure date and departure time. It provides a flexible way to retrieve detailed schedule information, including departure and arrival times, stations, ticket availability and the number of sold tickets. This procedure helps management, staff or customers quickly find relevant bus trips, plan journeys efficiently and monitor ticket availability. It can be used for operational planning, customer service and decision-making by providing real-time insights into bus schedules and seat occupancy.

Procedure code:

```

SET SERVEROUTPUT ON;
SET LINESIZE 180;
SET PAGESIZE 50;

-- Indexes (only create once, outside the procedure)
CREATE INDEX idx_bus_destination ON BusSchedule(UPPER(Destination));
CREATE INDEX idx_bus_date ON BusSchedule(DepartureDate);
CREATE INDEX idx_bus_time ON BusSchedule(DepartureTime);
CREATE INDEX idx_ticket_price ON Ticket(TicketPrice);
CREATE INDEX idx_company_name ON BusCompany(UPPER(CompanyName));

COLUMN ScheduleID FORMAT A12;
COLUMN Bus FORMAT A12;
COLUMN Company FORMAT A25;
COLUMN Destination FORMAT A20;
COLUMN DepartDate FORMAT A15;
COLUMN DepartTime FORMAT A15;
COLUMN ArrivalTime FORMAT A15;
COLUMN Price FORMAT 9999.99;

CREATE OR REPLACE PROCEDURE prc_search_bus_schedule(
    p_busPlate      IN VARCHAR2 DEFAULT NULL,
    p_company       IN VARCHAR2 DEFAULT NULL,
    p_destination   IN VARCHAR2 DEFAULT NULL,
    p_departDate    IN DATE      DEFAULT NULL,
    p_departTime    IN DATE      DEFAULT NULL
) IS
    -- Declare variables using %TYPE
    v_scheduleID    BusSchedule.BusScheduleID%TYPE;
    v_bus           Bus.PlatoNumber%TYPE;
    v_company       BusCompany.CompanyName%TYPE;
    v_departPlace   BusSchedule.DepartBusStation%TYPE;
    v_destination   BusSchedule.Destination%TYPE;
    v_departDate    BusSchedule.DepartureDate%TYPE;
    v_departTime    BusSchedule.DepartureTime%TYPE;
    v_arrivalTime   BusSchedule.ArrivalTime%TYPE;
    v_sold          NUMBER;
    v_remaining     NUMBER;

    v_found BOOLEAN := FALSE;

```

```

-- Cursor
CURSOR bus_cursor IS
    SELECT
        bs.BusScheduleID,
        b.PlateNumber,
        bc.CompanyName,
        bs.DepartBusStation,
        bs.Destination,
        bs.DepartureDate,
        bs.DepartureTime,
        bs.ArrivalTime,
        COUNT(CASE WHEN t.TicketStatus = 'Booked' THEN 1 END)      AS
SoldTickets,
        COUNT(CASE WHEN t.TicketStatus = 'Available' THEN 1 END) AS
RemainingTickets
    FROM BusSchedule bs
    JOIN Bus b ON bs.PlateNumber = b.PlateNumber
    JOIN BusCompany bc ON b.CompanyID = bc.CompanyID
    LEFT JOIN Ticket t ON bs.BusScheduleID = t.BusScheduleID
    WHERE
        (p_busPlate IS NULL OR UPPER(b.PlateNumber) =
UPPER(p_busPlate))
        AND (p_company IS NULL OR UPPER(bc.CompanyName) LIKE
UPPER(p_company) || '%')
        AND (p_destination IS NULL OR UPPER(bs.Destination) LIKE
UPPER(p_destination) || '%')
        AND (p_departDate IS NULL OR bs.DepartureDate = p_departDate)
        AND (p_departTime IS NULL OR bs.DepartureTime >= p_departTime)
    GROUP BY bs.BusScheduleID, b.PlateNumber, bc.CompanyName,
bs.DepartBusStation,
        bs.Destination, bs.DepartureDate, bs.DepartureTime,
bs.ArrivalTime
    ORDER BY bs.DepartureDate, bs.DepartureTime;

BEGIN
    -- Fetch loop
    OPEN bus_cursor;
    LOOP
        FETCH bus_cursor INTO v_scheduleID, v_bus, v_company,
v_departPlace,
                                v_destination, v_departDate, v_departTime,
                                v_arrivalTime, v_sold, v_remaining;
        EXIT WHEN bus_cursor%NOTFOUND;

        -- Print header only once when first record is found
        IF NOT v_found THEN
            DBMS_OUTPUT.PUT_LINE(
                RPAD('ScheduleID', 12) || RPAD('Bus', 12) ||
RPAD('Company', 25) ||
                RPAD('DepartPlace', 20) || RPAD('Destination', 20) ||
                RPAD('DepartDate', 15) || RPAD('DepartTime', 15) ||
                RPAD('ArrivalTime', 15) || RPAD('Sold', 8) || 'Remaining'
            );
            DBMS_OUTPUT.PUT_LINE(
                RPAD('-', 12, '-') || RPAD('-', 12, '-') || RPAD('-', 25,
'-' ) ||
                RPAD('-', 20, '-') || RPAD('-', 20, '-') || RPAD('-', 15,
'-' ) ||
                RPAD('-', 15, '-') || RPAD('-', 15, '-') || RPAD('-', 8,
'-' ) ||
                RPAD('-', 10, '-')
            );
            END IF;

            v_found := TRUE;

            DBMS_OUTPUT.PUT_LINE(

```

```

        RPAD(v_scheduleID, 12) ||
        RPAD(v_bus, 12) ||
        RPAD(v_company, 25) ||
        RPAD(v_departPlace, 20) ||
        RPAD(v_destination, 20) ||
        RPAD(TO_CHAR(v_departDate,'DD-MM-YYYY'), 15) ||
        RPAD(TO_CHAR(v_departTime,'HH24:MI'), 15) ||
        RPAD(TO_CHAR(v_arrivalTime,'HH24:MI'), 15) ||
        RPAD(v_sold, 8) ||
        v_remaining
    );
END LOOP;
CLOSE bus_cursor;

-- Raise NO_DATA_FOUND
IF NOT v_found THEN
    RAISE NO_DATA_FOUND;
END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No Bus Schedule found for filters: ' ||
            NVL(p_busPlate,'Any Bus') || ', ' ||
            NVL(p_company,'Any Company') || ', ' ||
            NVL(p_destination,'Any Destination') || ', '
        ||
            NVL(TO_CHAR(p_departDate,'DD-MM-YYYY'),'Any
Date') || ', ' ||
            NVL(TO_CHAR(p_departTime,'HH24:MI'),'Any
Time'));
END;
/

```

```

-- Filter by bus plate
EXEC prc_search_bus_schedule(p_busPlate => 'BU0010');

-- Filter by company
EXEC prc_search_bus_schedule(p_company => 'Transnasional');

-- Filter by destination
EXEC prc_search_bus_schedule(p_destination => 'Penang');

```

Sample Output:

```

-- Show all schedules
EXEC prc_search_bus_schedule;
SQL> EXEC prc_search_bus_schedule;
ScheduleID Bus Company DepartPlace Destination DepartDate DepartTime ArrivalTime Sold Remaining
-----
B50001 BU0001 Starline Express Kuala Lumpur Penang 12-01-2023 08:00 12:30 26 14
B50002 BU0002 Metro Bus Services Kuala Lumpur Ipoh 22-01-2023 09:30 13:45 57 23
B50003 BU0003 Rapid Travels Kuala Lumpur Johor Bahru 05-02-2023 07:00 11:15 10 0
B50004 BU0004 Greenway Transport Kuala Lumpur Kuantan 15-02-2023 10:00 14:45 8 2
B50005 BU0005 BlueSky Coaches Kuala Lumpur Melaka 25-02-2023 06:30 11:00 10 0
B50006 BU0006 Citylink Travels Penang Sentral Alor Setar 05-03-2023 08:15 12:45 10 0
B50007 BU0007 Sunrise Express Kuantan Sentral Kuala Terengganu 15-03-2023 09:00 13:30 10 0
B50008 BU0008 Galaxy Busline Kuala Lumpur Kota Bharu 25-03-2023 14:00 18:30 10 0
B50009 BU0009 Speedway Coaches Kuala Lumpur Seremban 05-04-2023 07:45 12:00 10 0
B50010 BU0010 Silverline Transport Penang Sentral Ipoh 15-04-2023 06:00 10:20 10 0
B50011 BU0011 Unity Express Penang Sentral Ipoh 15-04-2023 06:00 10:20 1 0
B50012 BU0012 Mega Transit Kuala Lumpur Kuantan 05-05-2023 08:15 12:45 8 2
B50013 BU0013 EcoRide Coaches Ipoh Amanjaya Penang 15-05-2023 07:00 11:15 8 2
B50014 BU0014 Royal Express Kuala Lumpur Melaka 25-05-2023 10:00 14:30 8 2
B50015 BU0015 Golden Way Travels Kuala Lumpur Kota Bharu 05-06-2023 06:45 11:00 8 2
B50016 BU0016 Rapidlink Express Penang Sentral Ipoh 15-06-2023 08:30 12:45 8 2
B50017 BU0017 TransCity Coaches Kuala Lumpur Johor Bahru 25-06-2023 14:15 18:30 10 0
B50018 BU0018 NextBus Travel Penang Sentral Alor Setar 05-07-2023 07:20 11:40 8 2
B50019 BU0019 Orbit Express Kuantan Sentral Kuala Terengganu 15-07-2023 09:00 13:30 8 2
B50020 BU0020 PrimeLine Travels Kuala Lumpur Seremban 25-07-2023 06:00 10:30 6 4

--With Certain Conditions
EXEC prc_search_bus_schedule(p_company=>'Starline Express',
p_destination=>'Penang',
p_departDate=>TO_DATE('12-01-2023','DD-MM-YYYY'));
SQL> EXEC prc_search_bus_schedule(p_company=>'Starline Express', p_destination=>'Penang', p_departDate=>TO_DATE('12-01-2023','DD-MM-YYYY'));
ScheduleID Bus Company DepartPlace Destination DepartDate DepartTime ArrivalTime Sold Remaining
-----
B50001 BU0001 Starline Express Kuala Lumpur Penang 12-01-2023 08:00 12:30 26 14

PL/SQL procedure successfully completed.

```

4.2.4 Procedure 2: Procedure of Full Refund of Bus Schedule Cancellation

Purpose:

The purpose of this procedure is to handle full refunds for a specific bus schedule when it is cancelled. In general, this procedure can be used to automatically identify all booked tickets associated with a given bus schedule, generate refund records, update booking and payment statuses and ensure that the bus schedule itself is marked as cancelled. It streamlines the refund process by performing all necessary updates in a single operation, including creating unique refund IDs, calculating total refund amounts, marking individual tickets as refunded, updating associated payments and committing all changes. This procedure is useful for operational efficiency, customer service and financial accuracy when managing schedule cancellations and ensuring all affected passengers are properly refunded.

Procedure code:

```
SET SERVEROUTPUT ON;
SET LINESIZE 180;
SET PAGESIZE 50;

CREATE OR REPLACE PROCEDURE prc_FullRefundSchedule (
    p_BusScheduleID IN BusSchedule.BusScheduleID%TYPE
) IS
    v_RefundID          Refund.RefundID%TYPE;
    v_Amount            Ticket.TicketPrice%TYPE;
    v_TotalRefund       NUMBER := 0;
    v_count             NUMBER;
BEGIN
    -- Check if the BusSchedule exists
    SELECT COUNT(*) INTO v_count
    FROM BusSchedule
    WHERE BusScheduleID = p_BusScheduleID;

    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20031, 'Bus schedule ' || p_BusScheduleID
        || ' not found.');
```

```
    END IF;

    -- Check if there are booked tickets to refund (that have not been
    refunded yet)
    SELECT COUNT(*) INTO v_count
    FROM Ticket t
    JOIN BookingDetails bd ON t.TicketID = bd.TicketID
    WHERE t.BusScheduleID = p_BusScheduleID
        AND t.TicketStatus = 'Booked'
        AND bd.RefundID IS NULL;

    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20032, 'No refundable tickets found for
    schedule ' || p_BusScheduleID);
    END IF;

    -- Update schedule status to Cancelled
    UPDATE BusSchedule
    SET ScheduleStatus = 'Cancelled'
    WHERE BusScheduleID = p_BusScheduleID;

    -- Loop through refundable tickets
    FOR rec IN (
        SELECT t.TicketID, b.PaymentID, t.TicketPrice
        FROM Ticket t
        JOIN BookingDetails bd ON t.TicketID = bd.TicketID
        JOIN Booking b ON bd.BookingID = b.BookingID
        WHERE t.BusScheduleID = p_BusScheduleID
            AND t.TicketStatus = 'Booked'
```

```
        AND bd.RefundID IS NULL
    ) LOOP
        -- Generate unique RefundID (6 chars total)
        v_RefundID := 'R' || LPAD(refundid_seq.NEXTVAL, 5, '0');

        v_Amount := rec.TicketPrice;
        v_TotalRefund := v_TotalRefund + v_Amount;

        -- Insert into Refund table
        INSERT INTO Refund (RefundID, RefundDate, RefundReason,
RefundAmount, RefundStatus)
        VALUES (v_RefundID, SYSDATE, 'Bus schedule cancelled', v_Amount,
'Approved');

        -- Update BookingDetails
        UPDATE BookingDetails
        SET RefundID = v_RefundID,
            BookingDetailsStatus = 'Cancelled'
        WHERE TicketID = rec.TicketID;

        -- Update Payment status
        UPDATE Payment
        SET PaymentStatus = 'Refunded'
        WHERE PaymentID = rec.PaymentID;
    END LOOP;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Total refund processed for BusSchedule ' ||
p_BusScheduleID ||
                        '. Total amount refunded: RM' || v_TotalRefund);

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20030, 'Unexpected error: ' || SQLERRM);
END prc_FullRefundSchedule;
/
```

Sample Output:

Before:

BOOKIN	TICKET	REFUND	BOOKINGDETAILSSTATU
B00001	T00001		Completed
B00001	T00002		Expired
B00001	T00003		Expired
B00001	T00004		Extended
B00001	T00005		Completed
B00002	T00006	R00021	Cancelled
B00002	T00007		Completed
B00002	T00008		Expired
B00003	T00009	R00022	Cancelled
B00003	T00010		Expired
B00004	T00011		Expired
B00004	T00012		Completed
B00004	T00013		Completed
B00004	T00014		Completed
B00005	T00015		Expired
B00005	T00016		Completed
B00005	T00017		Extended
B00005	T00018		Completed
B00006	T00019		Expired
B00006	T00020		Completed
B00007	T00021	R00023	Cancelled
B00007	T00022		Completed
B00007	T00023		Extended
B00007	T00024		Expired
B00007	T00025	R00024	Cancelled
B00008	T00026		Expired
B00008	T00027		Extended
B00008	T00028		Expired
B00008	T00029		Extended
B00009	T00030	R00025	Cancelled
B00009	T00031		Extended
B00009	T00032		Expired
B00009	T00033		Expired
B00010	T00034		Completed
B00010	T00035	R00026	Cancelled
B00010	T00036		Expired
B00011	T00037	R00027	Cancelled
B00011	T00038		Extended
B00012	T00039		Completed
B00012	T00040		Completed

40 rows selected.

■

After Procedure :

```
SQL> EXEC prc_FullRefundSchedule('BS0001');
Total refund processed for BusSchedule BS0001. Total amount refunded: RM1250
```

PL/SQL procedure successfully completed.

BOOKIN	TICKET	REFUND	BOOKINGDETAILSSTATUS
B00001	T00001	R00056	Cancelled
B00001	T00002	R00055	Cancelled
B00001	T00003	R00054	Cancelled
B00001	T00004		Extended
B00001	T00005	R00053	Cancelled
B00002	T00006	R00021	Cancelled
B00002	T00007	R00058	Cancelled
B00002	T00008	R00057	Cancelled
B00003	T00009	R00022	Cancelled
B00003	T00010	R00059	Cancelled
B00004	T00011	R00063	Cancelled
B00004	T00012	R00062	Cancelled
B00004	T00013	R00061	Cancelled
B00004	T00014	R00060	Cancelled
B00005	T00015	R00066	Cancelled
B00005	T00016	R00065	Cancelled
B00005	T00017		Extended
B00005	T00018	R00064	Cancelled
B00006	T00019	R00068	Cancelled
B00006	T00020	R00067	Cancelled
B00007	T00021	R00023	Cancelled
B00007	T00022	R00069	Cancelled
B00007	T00023		Extended
B00007	T00024		Expired
B00007	T00025	R00024	Cancelled
B00008	T00026	R00071	Cancelled
B00008	T00027		Extended
B00008	T00028	R00070	Cancelled
B00008	T00029		Extended
B00009	T00030	R00025	Cancelled
B00009	T00031		Extended
B00009	T00032	R00073	Cancelled
B00009	T00033	R00072	Cancelled
B00010	T00034	R00075	Cancelled
B00010	T00035	R00026	Cancelled
B00010	T00036	R00074	Cancelled
B00011	T00037	R00027	Cancelled
B00011	T00038		Extended
B00012	T00039	R00077	Cancelled
B00012	T00040	R00076	Cancelled

4.2.5 Trigger 1: Calculation of Refund Amount and Update Booking Refund ID

Purpose:

The purpose of this trigger is to automatically process and validate refunds whenever a refund is requested, for example when the RefundID of a BookingDetails record is updated. In general, this trigger ensures that all refund requests comply with business rules, such as requiring refunds to be requested at least 48 hours before the bus departure. If the request is made too late, the refund is rejected and the refund amount is set to zero. If it meets the timing condition, the refund is approved with 70% of the ticket price, the booking is marked as cancelled, the ticket is made available again and the seat count for the bus schedule is updated. Essentially, this trigger enforces automated, consistent refund management, reduces manual intervention and ensures proper adjustment of tickets, bookings and bus seat availability according to company policies.

Trigger Code:

```

CREATE OR REPLACE TRIGGER trg_newRefund
BEFORE UPDATE OF RefundID ON BookingDetails
FOR EACH ROW
DECLARE
    v_departureTime    BusSchedule.DepartureTime%TYPE;
    v_ticketPrice      Ticket.TicketPrice%TYPE;
    v_busScheduleID    Ticket.BusScheduleID%TYPE;
    v_refundDate       Refund.RefundDate%TYPE;
BEGIN
    IF :NEW.RefundID IS NOT NULL THEN

        -- Get ticket price, schedule ID, and departure time
        SELECT t.TicketPrice, t.BusScheduleID, bs.DepartureTime
        INTO v_ticketPrice, v_busScheduleID, v_departureTime
        FROM Ticket t
        JOIN BusSchedule bs ON t.BusScheduleID = bs.BusScheduleID
        WHERE t.TicketID = :NEW.TicketID;

        -- Get refund request date
        SELECT RefundDate
        INTO v_refundDate
        FROM Refund
        WHERE RefundID = :NEW.RefundID;

        -- Check refund eligibility (must be ≥ 48 hours before departure)
        IF (v_refundDate > v_departureTime - 2) THEN
            -- Reject refund (no updates persist because of error
rollback)
            RAISE_APPLICATION_ERROR(-20001,
                'Refund must be requested at least 48 hours before
departure. Refund has been rejected.');
```

Sample Output:

```

--Sample Data To Test Trigger : (Test Condition)

INSERT INTO Bus VALUES ('BU1000', 79, 'Double Decker', 'C00011');
INSERT INTO BusSchedule VALUES ('BS9999', TO_DATE('15-04-2023
06:00','DD-MM-YYYY HH24:MI'), TO_DATE('15-04-2023 10:20','DD-MM-YYYY
HH24:MI'), TO_DATE('15-04-2023','DD-MM-YYYY'), 'Ipoh', 'Scheduled',
'Penang Sentral', 'Ipoh Amanjaya', 'P10', 39, 10, 'BU1000');
```

```

INSERT INTO Ticket VALUES ('T10000', 50.00, 'Booked', 'Adult', 'B5',
'BS9999');
INSERT INTO Booking VALUES ('B10000', TO_DATE('10-01-2022','DD-MM-YYYY'),
'Pending', 125.60, 'M00010', 'P00010');
INSERT INTO BookingDetails VALUES ('B10000', 'T10000', '565894262680',
'Becky Walter', 'Completed', NULL, NULL);

```

```

-- Insert a refund requested less than 48 hours before departure
INSERT INTO Refund (RefundID, RefundDate, RefundReason, RefundAmount,
RefundStatus)
VALUES ('R20000', TO_DATE('14-04-2023 08:00','DD-MM-YYYY HH24:MI'),
'Emergency', 0, 'Pending'
);

SQL> UPDATE BookingDetails
2   SET RefundID = 'R20000'
3   WHERE BookingID = 'B10000'
4   AND TicketID = 'T10000';
UPDATE BookingDetails
*
ERROR at line 1:
ORA-20001: Refund must be requested at least 48 hours before departure. Refund has been rejected.
ORA-06512: at "ADM.TRG_NEWREFUND", line 30
ORA-04088: error during execution of trigger 'ADM.TRG_NEWREFUND'

SQL> SELECT * FROM Ticket WHERE TicketID = 'T10000';

```

TICKET	TICKETPRICE	TICKETSTATUS	TICKETTYPE	SEA	BUSSCH
T10000	50	Booked	Adult	B5	BS9999

```

-- Insert a refund requested more than 48 hours before departure
INSERT INTO Refund (RefundID, RefundDate, RefundReason, RefundAmount,
RefundStatus)
VALUES ('R20001', TO_DATE('10-04-2023 09:00','DD-MM-YYYY HH24:MI'),'Change
of plan',0, 'Pending');

```

```

SQL> UPDATE BookingDetails
2   SET RefundID = 'R20001'
3   WHERE BookingID = 'B10000'
4   AND TicketID = 'T10000';

1 row updated.

SQL> SELECT refundid, refundamount, refundstatus FROM Refund WHERE RefundID = 'R20001';

```

REFUND	REFUNDAMOUNT	REFUNDSTATUS
R20001	35	Approved

Before :

```

SQL> SELECT * FROM Ticket WHERE TicketID = 'T10000';

```

TICKET	TICKETPRICE	TICKETSTATUS	TICKETTYPE	SEA	BUSSCH
T10000	50	Booked	Adult	B5	BS9999

After:

```

SQL> SELECT * FROM Ticket WHERE TicketID = 'T10000'
2 ;

```

TICKET	TICKETPRICE	TICKETSTATUS	TICKETTYPE	SEA	BUSSCH
T10000	50	Available	Adult	B5	BS9999

4.2.6 Trigger 2: Audit Trail For Booking Table

Purpose:

The purpose of this trigger is to maintain an audit trail for all changes made to the booking table. It automatically logs everything, such as insert, update or delete action into an action table, capturing key information such as booking details, member ID, payment ID, user

performing the action, transaction date and time and the type of action. For users when performing an update, it also records both the old and new values of booking status and total price, enabling full tracking of changes over time. This trigger ensures accountability, allows management to monitor booking modifications and supports data integrity and auditing requirements without requiring manual logging.

Trigger Code:

```
CREATE TABLE ActionBooking (
    ActionID          NUMBER PRIMARY KEY,
    BookingID         VARCHAR2(6) NOT NULL,
    BookingStatus     VARCHAR2(50),
    NewBookingStatus  VARCHAR2(50),
    TotalPrice        NUMBER(10,2),
    NewTotalPrice     NUMBER(10,2),
    MemberID          VARCHAR2(6),
    PaymentID         VARCHAR2(6),
    UserID            VARCHAR2(50),
    TransDate         DATE,
    TransTime         VARCHAR2(20),
    TransAction       VARCHAR2(10) -- INSERT / UPDATE / DELETE
);

CREATE OR REPLACE TRIGGER trg_track_Booking
AFTER INSERT OR UPDATE OR DELETE ON Booking
FOR EACH ROW
BEGIN
    CASE
        WHEN INSERTING THEN
            INSERT INTO ActionBooking(
                BookingID, BookingStatus, TotalPrice, MemberID, PaymentID,
                UserID, TransDate, TransTime, TransAction
            )
            VALUES (
                :NEW.BookingID, :NEW.BookingStatus, :NEW.TotalPrice,
                :NEW.MemberID, :NEW.PaymentID,
                USER, SYSDATE, TO_CHAR(SYSDATE, 'HH24:MI:SS'), 'INSERT'
            );

        WHEN UPDATING THEN
            INSERT INTO ActionBooking(
                BookingID, BookingStatus, NewBookingStatus,
                TotalPrice, NewTotalPrice, MemberID, PaymentID,
                UserID, TransDate, TransTime, TransAction
            )
            VALUES (
                :OLD.BookingID, :OLD.BookingStatus, :NEW.BookingStatus,
                :OLD.TotalPrice, :NEW.TotalPrice, :OLD.MemberID, :OLD.PaymentID,
                USER, SYSDATE, TO_CHAR(SYSDATE, 'HH24:MI:SS'), 'UPDATE'
            );

        WHEN DELETING THEN
            INSERT INTO ActionBooking(
                BookingID, BookingStatus, TotalPrice, MemberID, PaymentID,
                UserID, TransDate, TransTime, TransAction
            )
            VALUES (
                :OLD.BookingID, :OLD.BookingStatus, :OLD.TotalPrice,
                :OLD.MemberID, :OLD.PaymentID,
                USER, SYSDATE, TO_CHAR(SYSDATE, 'HH24:MI:SS'), 'DELETE'
            );
    END CASE;
END;
/
```

Sample Output:

<<-Action Booking Audit Log-->											
Action ID	Booking ID	Old Status	New Status	Old Price	New Price	Member ID	Payment ID	User	Transaction	Time	Action
2	B90009	Completed		\$200.00		M00002	P00002	ADM	07-09-2025 19:22:41	9:22:41	INSERT
3	B90010	Pending		\$150.00		M00001	P00001	ADM	07-09-2025 19:22:48	9:22:48	INSERT
4	B90012	Pending		\$180.00		M00001	P00001	ADM	07-09-2025 19:23:15	9:23:15	INSERT
5	B90013	Completed		\$250.00		M00002	P00002	ADM	07-09-2025 19:23:15	9:23:15	INSERT
6	B90012	Pending	Cancelled	\$180.00	\$.00	M00001	P00001	ADM	07-09-2025 19:23:21	9:23:21	UPDATE
7	B90013	Completed		\$250.00		M00002	P00002	ADM	07-09-2025 19:23:27	9:23:27	DELETE
8	B90012	Cancelled	Cancelled	\$.00	\$.00	M00001	P00001	ADM	07-09-2025 19:24:55	9:24:55	UPDATE


```

--Test Data used :
-- Insert a new booking
INSERT INTO Booking (BookingID, PurchaseDate, BookingStatus, TotalPrice,
MemberID, PaymentID)VALUES ('B90010', SYSDATE, 'Pending', 150.00,
'M00001', 'P00001');
INSERT INTO Booking (BookingID, PurchaseDate, BookingStatus, TotalPrice,
MemberID, PaymentID)
VALUES ('B90011', SYSDATE, 'Completed', 320.00, 'M00002', 'P00002');
INSERT INTO Booking (BookingID, PurchaseDate, BookingStatus, TotalPrice,
MemberID, PaymentID)
VALUES ('B90012', SYSDATE, 'Pending', 180.00, 'M00001', 'P00001');
INSERT INTO Booking (BookingID, PurchaseDate, BookingStatus, TotalPrice,
MemberID, PaymentID)
VALUES ('B90013', SYSDATE, 'Completed', 250.00, 'M00002', 'P00002');
-- Change status + price for B90012
UPDATE Booking
    SET BookingStatus = 'Cancelled',
        TotalPrice = 0
    WHERE BookingID = 'B90012';

-- Delete booking B90013
DELETE FROM Booking
    WHERE BookingID = 'B90013';

```

4.2.7 Report 1: Bus Utilisation Efficiency Report

Purpose:

The purpose of this report is to analyze and report the utilization of buses for each bus company in the system. It calculates how efficiently each bus is being used by comparing the number of tickets sold with completed payments against the bus's total seating capacity. The procedure iterates through all bus companies, then through all schedules for each company and computes the utilization percentage for each schedule. This procedure can be used by management to monitor operational efficiency, identify underused or overbooked buses and make informed decisions regarding fleet allocation, scheduling and capacity planning. The output is a neatly formatted report displaying each schedule's ID, bus plate number, capacity, tickets sold and utilization percentage, along with a summary for each company showing total capacity, total tickets sold and overall utilization. This provides a clear overview of how well resources are being utilized across the company's fleet.

Report Code:

```

SET PAGESIZE 30;
SET LINESIZE 200;
SET SERVEROUTPUT ON;

CREATE INDEX idx_ticket_schedule_status ON Ticket (BusScheduleID,

```

```

TicketStatus);

CREATE OR REPLACE PROCEDURE prc_bus_utilization IS
    -- Variables using %TYPE for strong typing
    v_companyID    BusCompany.CompanyID%TYPE;
    v_companyName  BusCompany.CompanyName%TYPE;

    v_schedID      BusSchedule.BusScheduleID%TYPE;
    v_plateNo      Bus.PlateNumber%TYPE;
    v_capacity      Bus.Capacity%TYPE;
    v_totalSold    NUMBER;
    v_utilization  NUMBER(6,2);

    v_companyTotalCapacity NUMBER := 0;
    v_companyTotalSold    NUMBER := 0;

    v_lineWidth CONSTANT NUMBER := 70;

    -- Cursor for all companies
    CURSOR compCursor IS
        SELECT CompanyID, CompanyName
        FROM BusCompany
        ORDER BY CompanyName;

    -- Cursor for all schedules in a company
    CURSOR schedCursor(p_companyID BusCompany.CompanyID%TYPE) IS
        SELECT bs.BusScheduleID, b.PlateNumber, b.Capacity
        FROM BusSchedule bs
        JOIN Bus b ON bs.PlateNumber = b.PlateNumber
        WHERE b.CompanyID = p_companyID
        ORDER BY bs.BusScheduleID;

    -- Helper function to center text
    FUNCTION center_text(p_text VARCHAR2, p_width NUMBER) RETURN VARCHAR2
IS
    BEGIN
        RETURN LPAD(p_text, (p_width + LENGTH(p_text)) / 2, ' ');
    END;
BEGIN
    OPEN compCursor;
    LOOP
        FETCH compCursor INTO v_companyID, v_companyName;
        EXIT WHEN compCursor%NOTFOUND;

        -- Reset totals for this company
        v_companyTotalCapacity := 0;
        v_companyTotalSold := 0;
        DBMS_OUTPUT.PUT_LINE(RPAD('=', v_lineWidth, '='));
        DBMS_OUTPUT.PUT_LINE(CHR(10) || center_text('=== Bus Utilization
Report for Company: ' || v_companyName || ' ===', v_lineWidth));
        DBMS_OUTPUT.PUT_LINE(RPAD('=', v_lineWidth, '='));
        DBMS_OUTPUT.PUT_LINE(RPAD('Schedule ID', 15) ||
                                RPAD('Plate No', 15) ||
                                RPAD('Capacity', 12) ||
                                RPAD('Tickets Sold', 15) ||
                                'Utilization %');
        DBMS_OUTPUT.PUT_LINE(RPAD('-', v_lineWidth, '-'));

        OPEN schedCursor(v_companyID);
        LOOP
            FETCH schedCursor INTO v_schedID, v_plateNo, v_capacity;
            EXIT WHEN schedCursor%NOTFOUND;

            -- Count tickets sold = booked tickets with completed payments
            SELECT COUNT(*)
            INTO v_totalSold
            FROM Ticket t

```

```

        JOIN BookingDetails bd ON t.TicketID = bd.TicketID
        JOIN Booking bk ON bd.BookingID = bk.BookingID
        JOIN Payment p ON bk.PaymentID = p.PaymentID
        WHERE t.BusScheduleID = v_schedID
              AND t.TicketStatus = 'Booked'
              AND p.PaymentStatus = 'Completed';

        -- Compute utilization
        IF v_capacity > 0 THEN
            v_utilization := ROUND((v_totalSold / v_capacity) * 100,
2);
        ELSE
            v_utilization := 0;
        END IF;

        -- Only display if tickets sold > 0
        IF v_totalSold > 0 THEN
            DBMS_OUTPUT.PUT_LINE(RPAD(v_schedID, 15) ||
                                RPAD(v_plateNo, 15) ||
                                RPAD(v_capacity, 12) ||
                                RPAD(v_totalSold, 15) ||
                                TO_CHAR(v_utilization, '990.00') ||
                '%');

            -- Update totals
            v_companyTotalCapacity := v_companyTotalCapacity +
v_capacity;
            v_companyTotalSold := v_companyTotalSold + v_totalSold;
        END IF;

    END LOOP;
    CLOSE schedCursor;

    -- Print grand total row per company only if total sold > 0
    IF v_companyTotalSold > 0 THEN
        DBMS_OUTPUT.PUT_LINE(RPAD('-', v_lineWidth, '-'));
        IF v_companyTotalCapacity > 0 THEN
            v_utilization := ROUND((v_companyTotalSold /
v_companyTotalCapacity) * 100, 2);
        ELSE
            v_utilization := 0;
        END IF;

        DBMS_OUTPUT.PUT_LINE('Overall Summary:');
        DBMS_OUTPUT.PUT_LINE('Total Capacity = ' ||
v_companyTotalCapacity);
        DBMS_OUTPUT.PUT_LINE('Total Ticket Sold = ' ||
v_companyTotalSold);
        DBMS_OUTPUT.PUT_LINE('Overall Utilization = ' ||
TO_CHAR(v_utilization, '990.00') || '%');
    END IF;

    END LOOP;
    CLOSE compCursor;

    DBMS_OUTPUT.PUT_LINE(RPAD('=', v_lineWidth, '='));
    DBMS_OUTPUT.PUT_LINE(CHR(10) || center_text('=== END OF REPORT ===',
v_lineWidth));
    DBMS_OUTPUT.PUT_LINE(RPAD('=', v_lineWidth, '='));
    DBMS_OUTPUT.PUT_LINE('Generated on: ' || TO_CHAR(SYSDATE, 'DD-MON-YYYY
HH24:MI:SS'));
    DBMS_OUTPUT.PUT_LINE(RPAD('=', v_lineWidth, '='));
END;
/

EXEC prc_bus_utilization

```

Sample Output:

```

SQL> EXEC prc_bus_utilization
=====

      === Bus Utilization Report for Company: BlueSky Coaches ===
=====
Schedule ID   Plate No      Capacity   Tickets Sold   Utilization %
-----
BS0005        BU0005        40          1           2.50%
BS0045        BU0045        40         10          25.00%
BS0065        BU0065        40         10          25.00%
BS0085        BU0085        40         10          25.00%
-----

Overall Summary:
Total Capacity =160
Total Ticket Sold =31
Overall Utilization = 19.38%
=====

      === Bus Utilization Report for Company: Citylink Travels ===
=====
Schedule ID   Plate No      Capacity   Tickets Sold   Utilization %
-----
BS0006        BU0006        80          3           3.75%
BS0026        BU0026        80         10          12.50%
BS0046        BU0046        80         10          12.50%
BS0066        BU0066        80         10          12.50%
BS0086        BU0086        80         10          12.50%
-----

Overall Summary:
Total Capacity =400
Total Ticket Sold =43
Overall Utilization = 10.75%
=====

```

=== Bus Utilization Report for Company: TransCity Coaches ===				
Schedule ID	Plate No	Capacity	Tickets Sold	Utilization %
BS0037	BU0037	40	10	25.00%
BS0057	BU0057	40	10	25.00%
BS0077	BU0077	40	10	25.00%
BS0097	BU0097	40	10	25.00%
Overall Summary:				
Total Capacity =160				
Total Ticket Sold =40				
Overall Utilization = 25.00%				
=== Bus Utilization Report for Company: Unity Express ===				
Schedule ID	Plate No	Capacity	Tickets Sold	Utilization %
BS0011	BU0011	30	5	16.67%
BS0031	BU0031	30	10	33.33%
BS0051	BU0051	30	10	33.33%
BS0071	BU0071	30	10	33.33%
BS0091	BU0091	30	10	33.33%
Overall Summary:				
Total Capacity =150				
Total Ticket Sold =45				
Overall Utilization = 30.00%				
=== END OF REPORT ===				
Generated on: 07-SEP-2025 18:54:59				

4.2.8 Report 2: Report of Bus Route and Revenue

Purpose:

The purpose of this report is to analyze and report the profitability of all bus routes in the system. Specifically, it calculates key metrics for each route, including the total revenue generated from booked tickets, the number of trips operated, the average revenue per trip and the total number of tickets sold. The procedure loops through all distinct departure and destination combinations, aggregates revenue across all schedules for each route and tracks which routes are the most and least profitable. This report can be used by management to evaluate the financial performance of routes, identify high-earning or underperforming routes and make informed decisions about scheduling, pricing and resource allocation. The output is a neatly formatted report showing revenue details, trip counts and ticket sales for each route, along with overall totals and highlights of the most and least profitable routes. It essentially serves as a route-level profitability dashboard that helps in strategic planning, performance monitoring and optimizing bus operations.

Report Code:

```
SET SERVEROUTPUT ON;
SET PAGESIZE 30;
SET LINESIZE 200;
```

```

CREATE INDEX idx_busschedule_route ON BusSchedule(DepartBusStation,
Destination);

CREATE OR REPLACE PROCEDURE prc_route_profitability IS
    -- Variables
    v_depart      BusSchedule.DepartBusStation%TYPE;
    v_dest        BusSchedule.Destination%TYPE;
    v_schedID     BusSchedule.BusScheduleID%TYPE;
    v_ticketPrice  Ticket.TicketPrice%TYPE;

    v_revenue     NUMBER;
    v_tripCount   NUMBER;
    v_avgRevenue  NUMBER;
    v_ticketCount NUMBER; -- Total tickets sold for route

    -- Track for most profitable route only
    v_maxRevenue  NUMBER := 0;
    v_maxRoute    VARCHAR2(100);
    v_lineWidth   CONSTANT NUMBER := 110;

    -- Outer cursor: distinct routes
    CURSOR routeCursor IS
        SELECT DISTINCT DepartBusStation, Destination
        FROM BusSchedule
        ORDER BY DepartBusStation, Destination;

    -- Inner cursor: schedules per route
    CURSOR schedCursor(p_depart BusSchedule.DepartBusStation%TYPE,
        p_dest BusSchedule.Destination%TYPE) IS
        SELECT BusScheduleID
        FROM BusSchedule
        WHERE DepartBusStation = p_depart
        AND Destination = p_dest;

    -- Ticket cursor
    CURSOR ticketCursor(p_schedID BusSchedule.BusScheduleID%TYPE) IS
        SELECT t.TicketPrice
        FROM Ticket t
        JOIN BookingDetails bd ON t.TicketID = bd.TicketID
        JOIN Booking b ON bd.BookingID = b.BookingID
        WHERE t.BusScheduleID = p_schedID
        AND t.TicketStatus = 'Booked';

    v_ticketRec ticketCursor%ROWTYPE;

    -- Totals
    v_grandTotal  NUMBER := 0;
    v_totalRoutes NUMBER := 0;

    -- Helper function to center text
    FUNCTION center_text(p_text IN VARCHAR2, p_width IN NUMBER) RETURN
    VARCHAR2 IS
        v_spaces NUMBER;
        v_result VARCHAR2(4000);
    BEGIN
        v_spaces := (p_width - LENGTH(p_text)) / 2;
        IF v_spaces < 0 THEN
            v_spaces := 0;
        END IF;
        v_result := LPAD(' ', TRUNC(v_spaces)) || p_text;
        RETURN v_result;
    END;

BEGIN
    -- Header
    DBMS_OUTPUT.PUT_LINE(RPAD('=', v_lineWidth, '='));
    DBMS_OUTPUT.PUT_LINE(CHR(10) || center_text('==== Bus Route

```

```

Profitability Report =====', v_lineWidth) || CHR(10));
    DBMS_OUTPUT.PUT_LINE(RPAD('=', v_lineWidth, '='));

    -- Column headings
    DBMS_OUTPUT.PUT_LINE(
        RPAD('Departure',20) ||
        RPAD('Destination',20) ||
        RPAD('Trips',8) ||
        RPAD('Revenue (RM)',15) ||
        RPAD('Avg/Trip (RM)',15) ||
        RPAD('Tickets Sold',12)
    );
    DBMS_OUTPUT.PUT_LINE(RPAD('=', v_lineWidth, '='));

    -- Outer loop: for each route
    OPEN routeCursor;
    LOOP
        FETCH routeCursor INTO v_depart, v_dest;
        EXIT WHEN routeCursor%NOTFOUND;

        v_revenue := 0;
        v_tripCount := 0;
        v_ticketCount := 0;

        -- Inner loop: each schedule for the route
        OPEN schedCursor(v_depart, v_dest);
        LOOP
            FETCH schedCursor INTO v_schedID;
            EXIT WHEN schedCursor%NOTFOUND;

            v_tripCount := v_tripCount + 1;

            -- Loop tickets for this schedule
            OPEN ticketCursor(v_schedID);
            LOOP
                FETCH ticketCursor INTO v_ticketRec;
                EXIT WHEN ticketCursor%NOTFOUND;
                v_revenue := v_revenue + v_ticketRec.TicketPrice;
                v_ticketCount := v_ticketCount + 1; -- Count tickets
            END LOOP;
            CLOSE ticketCursor;
        END LOOP;
        CLOSE schedCursor;

        -- Average revenue per trip
        IF v_tripCount > 0 THEN
            v_avgRevenue := v_revenue / v_tripCount;
        ELSE
            v_avgRevenue := 0;
        END IF;

        -- Track most profitable route only
        IF v_revenue > v_maxRevenue THEN
            v_maxRevenue := v_revenue;
            v_maxRoute := v_depart || ' -> ' || v_dest;
        END IF;

        -- Print route row ONLY if revenue, avg revenue, and tickets sold
        > 0
        IF v_revenue > 0 AND v_avgRevenue > 0 AND v_ticketCount > 0 THEN
            DBMS_OUTPUT.PUT_LINE(
                RPAD(v_depart,20) ||
                RPAD(v_dest,20) ||
                RPAD(TO_CHAR(v_tripCount),8) ||
                RPAD(TO_CHAR(v_revenue,'999,999'),15) ||
                RPAD(TO_CHAR(v_avgRevenue,'999,999.99'),15) ||
                RPAD(TO_CHAR(v_ticketCount),12)
            );
        END IF;
    END LOOP;
    CLOSE routeCursor;

```

```

);

-- Include in grand totals only if non-zero
v_grandTotal := v_grandTotal + v_revenue;
v_totalRoutes := v_totalRoutes + 1;
END IF;

END LOOP;
CLOSE routeCursor;

-- Footer
DBMS_OUTPUT.PUT_LINE(RPAD('=', v_lineWidth, '='));
DBMS_OUTPUT.PUT_LINE('Total Routes Processed: ' || v_totalRoutes);
DBMS_OUTPUT.PUT_LINE('Overall Revenue: RM' ||
TO_CHAR(v_grandTotal, '999,999,999.99'));

-- Most profitable route only
DBMS_OUTPUT.PUT_LINE(CHR(10) || 'Most Profitable Route: ' ||
v_maxRoute ||
' | Revenue: RM' ||
TO_CHAR(v_maxRevenue, '999,999.99'));

-- End of report
DBMS_OUTPUT.PUT_LINE(RPAD('=', v_lineWidth, '='));
DBMS_OUTPUT.PUT_LINE(CHR(10) || center_text('=== END OF REPORT ===',
v_lineWidth));
DBMS_OUTPUT.PUT_LINE(RPAD('=', v_lineWidth, '='));
DBMS_OUTPUT.PUT_LINE('Generated on: ' || TO_CHAR(SYSDATE, 'DD-MON-YYYY
HH24:MI:SS'));
DBMS_OUTPUT.PUT_LINE(RPAD('=', v_lineWidth, '='));
END;
/

EXEC prc_route_profitability

```

Sample Output:

```

=====
===== Bus Route Profitability Report =====
=====
Departure      Destination    Trips  Revenue (RM)  Avg/Trip (RM)  Tickets Sold
=====
Ipoh Amanjaya  Penang         1       400           400.00         8
Kuala Lumpur  Ipoh           46      4,850         105.43         97
Kuala Lumpur  Johor Bahru    40      6,500         162.50        130
Kuala Lumpur  Kota Bharu     37      4,400         118.92         88
Kuala Lumpur  Kuantan        41      6,300         153.66        126
Kuala Lumpur  Melaka         18      6,400         355.56        128
Kuala Lumpur  Penang         41      5,800         141.46        116
Kuala Lumpur  Seremban       38      2,800          73.68         56
Kuantan Sentral Kuala Terengganu 9      2,900         322.22         58
Melaka Sentral Johor Bahru     2        950         475.00         18
Penang Sentral Alor Setar     13      4,500         346.15         88
Penang Sentral Ipoh          12      4,400         366.67         88
=====
Total Routes Processed: 12
Overall Revenue: RM    50,200.00

Most Profitable Route: Kuala Lumpur -> Johor Bahru | Revenue: RM    6,500.00
=====

=== END OF REPORT ===
=====
Generated on: 07-SEP-2025 18:59:15
=====

PL/SQL procedure successfully completed.

```

4.3 Gan Khai Li

4.3.1 Query 1: Most Frequent Destinations by Member

Purpose:

The purpose of this query is to provide a summary of the most frequent travel destinations for each member, including the number of trips taken to each destination. It helps management analyze travel behaviour, identify popular routes among members and support decision making for marketing, customer loyalty programs and route planning. By understanding which destinations are most commonly visited, the company can design targeted promotions and personalized offers to strengthen customer relationships, optimize route scheduling and resource allocation to match demand and enhance overall service efficiency. The query transforms raw transactional data into actionable business intelligence that highlights both individual member preferences and broader travel trends across the customer base.

SQL statement:

```
SET LINESIZE 200
SET PAGESIZE 800
ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MM-YYYY';

CREATE OR REPLACE VIEW MostFrequentDestinationsView AS
SELECT
    m.MemberID,
    m.MemberName,
    bs.Destination,
    COUNT(bd.TicketID) AS TotalTrips
FROM Booking b
JOIN BookingDetails bd ON b.BookingID = bd.BookingID
JOIN Ticket t ON bd.TicketID = t.TicketID
JOIN BusSchedule bs ON t.BusScheduleID = bs.BusScheduleID
JOIN Member m ON b.MemberID = m.MemberID
GROUP BY m.MemberID, m.MemberName, bs.Destination;

TTITLE LEFT '                        Most Frequent Destinations by
Member' SKIP 2
COLUMN MemberID FORMAT A10 HEADING 'Member ID'
COLUMN MemberName FORMAT A25 HEADING 'Member Name'
COLUMN Destination FORMAT A30 HEADING 'Destination'
COLUMN TotalTrips FORMAT 999 HEADING 'Total Trip'

BREAK ON MemberID SKIP 1 ON MemberName
COMPUTE SUM LABEL 'Total Trips ' OF TotalTrips ON MemberID

SELECT *
FROM MostFrequentDestinationsView
ORDER BY MemberID, TotalTrips DESC;

CLEAR COLUMNS;
CLEAR BREAKS;
CLEAR COMPUTES;
TTITLE OFF;
```

Sample Output:

Most Frequent Destinations by Member			
Member ID	Member Name	Destination	Total Trip
M00001	Alice Tan Mei Ling	Ipoh	6
		Penang	5
		Kuantan	4
		Melaka	3
		Johor Bahru	2
		*****	-----
Total Trip		20	
M00002	Benjamin Lee Wei	Johor Bahru	10
		Penang	7
		Kuantan	3
		Ipoh	3
		Kuala Terengganu	1
		*****	-----
Total Trip		24	
M00003	Chong Li Hua	Ipoh	10
		Kuantan	4
		Johor Bahru	3
		Alor Setar	3
		Penang	2
		*****	-----
Total Trip		22	
M00004	Daniel Wong Jun	Kuantan	15
		Melaka	13
		Penang	4
		Ipoh	3
		Alor Setar	3
		*****	-----
Total Trip		38	
M00005	Emily Ng Jia	Penang	8
		Ipoh	5
		Alor Setar	4
		Melaka	3
		Kuantan	2
		*****	-----
Total Trip		22	
M00006	Farid Ahmad	Melaka	7
		Ipoh	5
		Kuantan	4
		Kuala Terengganu	2
		Penang	2
		*****	-----
Total Trip		20	

M00007	Grace Lim Siew	Penang	14
		Ipoh	5
		Kuala Terengganu	3
		Melaka	1
*****	*****		-----
	Total Trip		23
M00008	Hafiz Roslan	Kota Bharu	10
		Penang	8
		Ipoh	4
		Alor Setar	3
		Kuala Terengganu	2
*****	*****		-----
	Total Trip		27
M00009	Isabella Tan	Penang	6
		Alor Setar	3
		Kuala Terengganu	2
		Ipoh	2
*****	*****		-----
	Total Trip		13
M00010	Jason Ong	Melaka	5
		Penang	3
		Kuala Terengganu	3
		Ipoh	2
		Alor Setar	2
		Seremban	1
*****	*****		-----
	Total Trip		16
M00011	Kelly Chan	Penang	6
		Ipoh	5
		Melaka	5
		Alor Setar	2
		Seremban	2
*****	*****		-----
	Total Trip		20
M00012	Leonard Goh	Ipoh	12
		Melaka	10
		Kuala Terengganu	6
		Kota Bharu	4
		Seremban	2
		Penang	2
		Kuantan	1
*****	*****		-----
	Total Trip		37

M00013	Monica Teo	Ipoh	8
		Johor Bahru	5
		Kuala Terengganu	4
		Kota Bharu	3
		Seremban	1
*****	*****	-----	
	Total Trip		21
M00014	Nathan Lim	Kota Bharu	7
		Ipoh	6
		Seremban	1
*****	*****	-----	
	Total Trip		14
M00015	Olivia Tan	Ipoh	11
		Kota Bharu	6
		Seremban	1
*****	*****	-----	
	Total Trip		18
M00016	Peter Chua	Ipoh	9
		Alor Setar	5
		Seremban	4
		Johor Bahru	2
*****	*****	-----	
	Total Trip		20
M00017	Rachel Lim	Ipoh	5
		Kota Bharu	5
		Seremban	4
		Johor Bahru	3
		Penang	2
*****	*****	-----	
	Total Trip		19
M00018	Samuel Ong	Ipoh	9
		Johor Bahru	4
		Seremban	4
		Melaka	2
		Penang	1
*****	*****	-----	
	Total Trip		20
M00019	Tina Foo	Ipoh	10
		Kuantan	5
		Penang	5
		Johor Bahru	5
*****	*****	-----	
	Total Trip		25

M00020	Aiden Tan	Johor Bahru	10
*****	*****	*****	-----
Total Trip			10
M00021	Brenda Lim	Alor Setar	10
		Kuala Terengganu	5
		Johor Bahru	4
*****	*****	*****	-----
Total Trip			19
M00022	Charles Wong	Ipoh	4
		Melaka	2
*****	*****	*****	-----
Total Trip			6
M00024	Ethan Goh	Kota Bharu	5
*****	*****	*****	-----
Total Trip			5
M00025	Fiona Lee	Johor Bahru	5
*****	*****	*****	-----
Total Trip			5
M00026	Gavin Lim	Johor Bahru	9
		Ipoh	5
		Alor Setar	4
*****	*****	*****	-----
Total Trip			18
M00027	Hannah Tan	Alor Setar	6
*****	*****	*****	-----
Total Trip			6
M00028	Ivan Chong	Kuantan	5
		Ipoh	5
		Johor Bahru	2
*****	*****	*****	-----
Total Trip			12
M00029	Jasmine Teo	Kota Bharu	10
		Alor Setar	1
*****	*****	*****	-----
Total Trip			11
M00031	Laura Chua	Johor Bahru	10
*****	*****	*****	-----
Total Trip			10

M00032	Marcus Lim	Ipoh	7
		Seremban	5
		Melaka	4
*****	*****		-----
Total Trip			16
M00033	Nadia Tan	Kuantan	6
		Johor Bahru	5
		Melaka	5
		Alor Setar	1
*****	*****		-----
Total Trip			17
M00035	Patricia Goh	Johor Bahru	5
		Alor Setar	5
*****	*****		-----
Total Trip			10
M00036	Quentin Chan	Melaka	13
		Ipoh	5
*****	*****		-----
Total Trip			18
M00038	Steve Low	Alor Setar	10
*****	*****		-----
Total Trip			10
M00039	Tania Yeo	Alor Setar	5
*****	*****		-----
Total Trip			5
M00040	Umar Zain	Johor Bahru	9
		Penang	2
*****	*****		-----
Total Trip			11
M00041	Vanessa Tan	Penang	6
		Kuala Terengganu	6
*****	*****		-----
Total Trip			12
M00042	Wilson Goh	Melaka	5
*****	*****		-----
Total Trip			5
M00043	Xinyi Lim	Kuantan	5
		Ipoh	2
*****	*****		-----
Total Trip			7

M00045	Zara Wong	Melaka	6
		Penang	5
*****	*****		-----
Total Trip			11
M00046	Adrian Teh	Ipoh	10
		Penang	10
		Seremban	10
		Melaka	5
		Kuantan	5
		Johor Bahru	1
*****	*****		-----
Total Trip			41
M00047	Belinda Ng	Kota Bharu	10
*****	*****		-----
Total Trip			10
M00048	Caleb Chia	Kuantan	5
		Melaka	4
*****	*****		-----
Total Trip			9
M00049	Danielle Liew	Kuala Terengganu	6
		Penang	5
		Seremban	5
*****	*****		-----
Total Trip			16
M00050	Edwin Toh	Penang	1
*****	*****		-----
Total Trip			1
M00051	Faye Sim	Kota Bharu	2
*****	*****		-----
Total Trip			2
M00052	Gordon Tan	Kota Bharu	4
		Kuala Terengganu	2
*****	*****		-----
Total Trip			6
M00053	Hazel Yong	Johor Bahru	10
		Ipoh	5
*****	*****		-----
Total Trip			15
M00054	Isaac Leong	Penang	5
		Melaka	5
		Kuantan	3
*****	*****		-----
Total Trip			13
M00055	Jolene Choo	Kuantan	20
*****	*****		-----
Total Trip			20
M00056	Ken Yap	Johor Bahru	6
*****	*****		-----
Total Trip			6

M00057	Lily Chong	Kuantan	10
		Kuala Terengganu	5
	*****		-----
Total Trip			15
M00058	Matthew Chan	Ipoh	5
		Melaka	4
	*****		-----
Total Trip			9
M00059	Nicole Lim	Kuantan	5
		Penang	5
		Kota Bharu	5
	*****		-----
Total Trip			15
M00061	Penelope Ong	Seremban	10
		Penang	3
	*****		-----
Total Trip			13
M00063	Renee Yeo	Kuantan	5
	*****		-----
Total Trip			5
M00064	Shaun Lim	Johor Bahru	2
	*****		-----
Total Trip			2
M00065	Tiffany Goh	Kuantan	3
	*****		-----
Total Trip			3
M00066	Ulysses Chong	Ipoh	5
		Alor Setar	2
	*****		-----
Total Trip			7
M00067	Vivian Lee	Ipoh	1
	*****		-----
Total Trip			1
M00068	Wayne Chua	Kuantan	6
		Ipoh	4
	*****		-----
Total Trip			10
M00070	Yong Sheng	Ipoh	10
		Johor Bahru	5
	*****		-----
Total Trip			15
M00071	Zhi Ying	Ipoh	6
		Kuala Terengganu	5
	*****		-----
Total Trip			11

M00073	Bernice Chin	Kuantan	3
		Penang	1
*****	*****	-----	
Total Trip			4
M00074	Clarence Wong	Penang	5
		Johor Bahru	3
*****	*****	-----	
Total Trip			8
M00075	Daphne Teo	Melaka	2
*****	*****	-----	
Total Trip			2
M00076	Eric Tan	Alor Setar	1
*****	*****	-----	
Total Trip			1
M00077	Felicia Ng	Melaka	1
*****	*****	-----	
Total Trip			1
M00078	Gerald Toh	Johor Bahru	13
*****	*****	-----	
Total Trip			13
M00079	Hui Ling	Seremban	10
*****	*****	-----	
Total Trip			10
M00080	Ian Low	Penang	6
		Johor Bahru	1
*****	*****	-----	
Total Trip			7
M00081	Jia Wen	Kuantan	1
*****	*****	-----	
Total Trip			1
M00082	Keith Ong	Ipoh	2
*****	*****	-----	
Total Trip			2
M00083	Lydia Chan	Melaka	3
*****	*****	-----	
Total Trip			3
M00086	Owen Chia	Melaka	5
*****	*****	-----	
Total Trip			5
M00087	Priscilla Wong	Ipoh	2
		Johor Bahru	1
*****	*****	-----	
Total Trip			3
M00088	Raymond Goh	Kuala Terengganu	3
		Melaka	3
*****	*****	-----	
Total Trip			6

M00089	Stephanie Tan	Penang	4
*****	*****	*****	-----
Total Trip			4
M00090	Terrence Lim	Kota Bharu	5
		Johor Bahru	5
		Penang	2
*****	*****	*****	-----
Total Trip			12
M00091	Umairah Zulkifli	Penang	1
*****	*****	*****	-----
Total Trip			1
M00093	Wendy Lau	Alor Setar	10
*****	*****	*****	-----
Total Trip			10
M00094	Xavier Loo	Kuantan	5
*****	*****	*****	-----
Total Trip			5
M00095	Yin Mei	Alor Setar	2
*****	*****	*****	-----
Total Trip			2
M00096	Zikri Hassan	Melaka	9
		Johor Bahru	5
*****	*****	*****	-----
Total Trip			14
M00097	Alyssa Tan	Kota Bharu	6
		Kuala Terengganu	5
		Alor Setar	5
		Kuantan	5
*****	*****	*****	-----
Total Trip			21
M00098	Bryan Leong	Ipoh	6
		Kota Bharu	5
		Melaka	5
*****	*****	*****	-----
Total Trip			16
M00099	Cheryl Ho	Alor Setar	3
		Kota Bharu	3
*****	*****	*****	-----
Total Trip			6
M99999	Walk-In Customer	Ipoh	6
		Johor Bahru	5
		Penang	4
*****	*****	*****	-----
Total Trip			15

4.3.2 Query 2: Top 5 Redeemed Gifts with Recent Member Activity

Purpose:

The purpose of this query is to generate a report on the top five most frequently redeemed gifts in the loyalty program, showing the members who redeemed them, the most recent redemption date and the total number of times redeemed. By combining gift, member and redemption data, it reveals both overall gift popularity and individual member engagement patterns. This helps management monitor loyalty program activity, ensure sufficient inventory for high-demand gifts and design targeted marketing campaigns or personalized offers. Ultimately, the query provides

insights that support better inventory planning, stronger customer relationships and improved member retention.

SQL statement:

```
SET LINESIZE 150
SET PAGESIZE 100
SET SERVEROUTPUT ON

ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MM-YYYY';

CREATE OR REPLACE VIEW Top5GiftMemberRecentView AS
SELECT
    g.GiftName,
    g.PointRequired,
    m.MemberID,
    m.MemberName,
    MAX(pr.RedemptionDate) AS LastRedemption,
    COUNT(*) AS RedemptionCount
FROM PointRedemption pr
JOIN Gift g ON pr.GiftID = g.GiftID
JOIN Member m ON pr.MemberID = m.MemberID
WHERE pr.GiftID IN (
    SELECT GiftID
    FROM (
        SELECT GiftID
        FROM PointRedemption
        GROUP BY GiftID
        ORDER BY COUNT(*) DESC
    ) top5
    WHERE ROWNUM <= 5
)
GROUP BY g.GiftID, g.GiftName, g.PointRequired, m.MemberID,
m.MemberName
ORDER BY g.GiftName, LastRedemption DESC;

TTITLE LEFT '                                Top 5 Redeemed
Gifts with Recent Member Activity' SKIP 2
COLUMN GiftName          FORMAT A40 HEADING 'Gift Name'
COLUMN PointRequired     FORMAT 99999 HEADING 'Points Req'
COLUMN MemberID          FORMAT A10 HEADING 'Member ID'
COLUMN MemberName        FORMAT A25 HEADING 'Member Name'
COLUMN LastRedemption     FORMAT DD-MM-YYYY HEADING 'Last Redemption'
COLUMN RedemptionCount    FORMAT 999 HEADING 'Times Redeemed'

BREAK ON GiftName SKIP 1 ON PointRequired
COMPUTE SUM LABEL 'Total Redemption: ' OF RedemptionCount ON
GiftName

SELECT *
FROM Top5GiftMemberRecentView;

CLEAR COLUMNS;
CLEAR BREAKS;
CLEAR COMPUTES;
TTITLE OFF;
```

Sample Output:

Top 5 Redeemed Gifts with Recent Member Activity							
Gift Name	Points Req	Member ID	Member Name	LASTREDEMP	Times Redeemed		
Foldable Shopping Tote	850	M00019	Tina Foo	24-07-2024	3		
		M00012	Leonard Goh	04-07-2024	2		
		M00011	Kelly Chan	30-06-2024	1		
		M00004	Daniel Wong Jun	20-06-2024	3		
		M00005	Emily Ng Jia	01-05-2024	1		
		M00003	Chong Li Hua	15-04-2024	2		
		M00002	Benjamin Lee Wei	23-03-2024	1		
		M00018	Samuel Ong	24-02-2024	2		
		M00014	Nathan Lim	27-01-2024	1		
		M00016	Peter Chua	23-01-2024	2		
		M00010	Jason Ong	08-09-2023	1		
		M00017	Rachel Lim	25-02-2023	1		

Total Redemption:					20		
Insulated Lunch Box	1200	M00012	Leonard Goh	29-06-2024	1		
		M00016	Peter Chua	12-05-2024	1		
		M00005	Emily Ng Jia	09-05-2024	2		
		M00009	Isabella Tan	02-03-2024	1		
		M00010	Jason Ong	24-01-2024	3		
		M00015	Olivia Tan	15-12-2023	1		
		M00008	Hafiz Roslan	21-11-2023	1		
		M00004	Daniel Wong Jun	17-11-2023	1		
		M00017	Rachel Lim	15-09-2023	1		
		M00007	Grace Lim Siew	28-07-2023	4		
		M00002	Benjamin Lee Wei	09-07-2023	1		
		M00013	Monica Teo	29-04-2023	1		
		M00006	Farid Ahmad	24-04-2023	1		

Total Redemption:					19		
Portable Mini Fan	2000	M00016	Peter Chua	21-08-2024	2		
		M00011	Kelly Chan	31-05-2024	2		
		M00009	Isabella Tan	18-05-2024	1		
		M00017	Rachel Lim	30-03-2024	1		
		M00007	Grace Lim Siew	21-03-2024	1		
		M00002	Benjamin Lee Wei	20-03-2024	2		
		M00006	Farid Ahmad	23-02-2024	3		
		M00003	Chong Li Hua	02-12-2023	1		
		M00013	Monica Teo	26-11-2023	1		
		M00012	Leonard Goh	14-11-2023	1		
		M00015	Olivia Tan	24-09-2023	2		
		M00010	Jason Ong	04-08-2023	1		
		M00004	Daniel Wong Jun	24-06-2023	2		

Total Redemption:					20		
Souvenir Fridge Magnet (KL/TBS Landmark)	800	M00006	Farid Ahmad	04-07-2024	1		
		M00004	Daniel Wong Jun	24-06-2024	1		
		M00005	Emily Ng Jia	04-06-2024	1		
		M00017	Rachel Lim	13-05-2024	4		
		M00011	Kelly Chan	03-03-2024	2		
		M00016	Peter Chua	16-02-2024	1		
		M00012	Leonard Goh	20-11-2023	1		
		M00009	Isabella Tan	07-10-2023	2		
		M00019	Tina Foo	04-09-2023	1		
		M00001	Alice Tan Mei Ling	29-08-2023	2		
		M00008	Hafiz Roslan	02-07-2023	2		
		M00003	Chong Li Hua	01-06-2023	1		
		M00018	Samuel Ong	26-04-2023	1		

Total Redemption:					20		
Travel Pouch (Passport, Tickets, Coins)	1800	M00009	Isabella Tan	13-08-2024	2		
		M00005	Emily Ng Jia	01-08-2024	1		
		M00002	Benjamin Lee Wei	19-07-2024	2		
		M00006	Farid Ahmad	14-07-2024	2		
		M00012	Leonard Goh	30-01-2024	1		
		M00010	Jason Ong	06-01-2024	2		
		M00011	Kelly Chan	08-12-2023	1		
		M00019	Tina Foo	24-11-2023	2		
		M00008	Hafiz Roslan	12-08-2023	2		
		M00014	Nathan Lim	27-06-2023	2		
		M00007	Grace Lim Siew	26-05-2023	1		
		M00015	Olivia Tan	31-01-2023	1		

		Total Redemption:					19

4.3.3 Procedure 1: Manage Bus Maintenance

Purpose:

The purpose of this procedure is to automate the process of recording bus maintenance services by validating the bus existence and service cost, generating unique service and transaction IDs and inserting related records into the Service, ServiceDetails and StaffAllocation tables. It ensures accurate data entry, assigns staff and time slots randomly, commits the transaction atomically and provides clear

success or error messages, streamlining maintenance record management while enforcing business rules.

Procedure code:

```

DROP SEQUENCE servicetrx_seq;
CREATE SEQUENCE servicetrx_seq
MINVALUE 301
MAXVALUE 99999
START WITH 301
INCREMENT BY 1
NOCACHE;

SET SERVEROUTPUT ON
CREATE OR REPLACE PROCEDURE prc_ManageBusMaintenance (
    p_plateNumber      IN Bus.PlateNumber%TYPE,
    p_serviceType      IN Service.ServiceType%TYPE,
    p_serviceCost      IN Service.ServiceCost%TYPE
) IS
    v_serviceID        Service.ServiceID%TYPE;
    v_trxID            ServiceDetails.ServiceTransactionID%TYPE;
    v_staffID          Staff.StaffID%TYPE;
    v_timeSlot         VARCHAR(20);
    v_serviceDate       DATE := TRUNC(SYSDATE);
    v_serviceCompleteDate DATE := TRUNC(SYSDATE + 3);

    e_BusNotExist      EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_BusNotExist, -20001);

    e_InvalidCost      EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_InvalidCost, -20003);

BEGIN
    DECLARE
        v_tmpPlate Bus.PlateNumber%TYPE;
    BEGIN
        SELECT PlateNumber INTO v_tmpPlate
        FROM Bus
        WHERE PlateNumber = p_plateNumber;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE e_BusNotExist;
    END;

    IF p_serviceCost < 20 THEN
        RAISE e_InvalidCost;
    END IF;

    v_serviceID := 'SV' || LPAD(servicetrx_seq.NEXTVAL, 4, '0');

    INSERT INTO Service(ServiceID, ServiceType, ServiceCost)
    VALUES (v_serviceID, p_serviceType, p_serviceCost);

    v_trxID := 'ST' || LPAD(servicetrx_seq.NEXTVAL, 4, '0');

    INSERT INTO ServiceDetails(ServiceTransactionID, ServiceStatus,
    ServiceDate, ServiceCompleteDate, PlateNumber, ServiceID)
    VALUES (v_trxID, 'Pending', v_serviceDate,
    v_serviceCompleteDate, p_plateNumber, v_serviceID);

    v_staffID := CASE DBMS_RANDOM.VALUE(0,1) WHEN 0 THEN 'S00004'
    ELSE 'S00011' END;

    v_timeSlot := CASE DBMS_RANDOM.VALUE(0,1) WHEN 0 THEN 'Morning'
    ELSE 'Afternoon' END;

    INSERT INTO StaffAllocation(ServiceID, StaffID, AllocationDate,

```

```
TimeSlot)
VALUES (v_serviceID, v_staffID, v_serviceDate, v_timeSlot);

COMMIT;

DBMS_OUTPUT.PUT_LINE('Maintenance Record Created Successfully
for ' || p_plateNumber || '.');

EXCEPTION
WHEN e_BusNotExist THEN
    DBMS_OUTPUT.PUT_LINE('Error: Bus ' || p_plateNumber || '
does not exist. Maintenance record NOT created.');
```

```
WHEN e_InvalidCost THEN
    DBMS_OUTPUT.PUT_LINE('Error: Service cost must be at least
RM 20. Maintenance record NOT created.');
```

```
WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' ||
SQLERRM);

END;
/
```

Sample Output:

```
SQL> EXEC prc_ManageBusMaintenance('BU0100', 'Repair', 500);
Maintenance Record Created Successfully for BU0100.

PL/SQL procedure successfully completed.

Error Handling:

SQL> EXEC prc_ManageBusMaintenance('BU0202', 'Repair', 500);
Error: Bus BU0202 does not exist. Maintenance record NOT created.

SQL> EXEC prc_ManageBusMaintenance('BU0100', 'Repair', 10);
Error: Service cost must be at least RM 20. Maintenance record NOT created.

PL/SQL procedure successfully completed.
```

4.3.4 Procedure 2: Member Registration

Purpose:

The purpose of this procedure is to register a new member into the system while ensuring data validity and system consistency. It performs multiple validations, including age verification (minimum age of 12), uniqueness of the identification number (IC) and proper email format. Upon successful validation, it generates a unique Member ID using a sequence and also creates a corresponding Payment record with a uniquely generated Payment ID and random payment details. This procedure ensures that each new member is accurately and securely onboarded into the system with minimal manual intervention, improving data integrity and operational efficiency.

Procedure code:

```
DROP SEQUENCE member_seq;
CREATE SEQUENCE member_seq
MINVALUE 101
MAXVALUE 99998
START WITH 101
INCREMENT BY 1
```

```

NOCACHE
/

SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION calculate_age(p_dob DATE)
RETURN NUMBER
IS
    v_age NUMBER;
BEGIN
    v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, p_dob) / 12);
    RETURN v_age;
END calculate_age;
/

CREATE OR REPLACE PROCEDURE register_member(
    p_name      IN Member.MemberName%TYPE,
    p_dob       IN Member.DateOfBirth%TYPE,
    p_gender    IN Member.MemberGender%TYPE,
    p_email     IN Member.MemberEmail%TYPE,
    p_phone     IN Member.MemberPhoneNo%TYPE,
    p_ic        IN Member.MemberIC%TYPE
)
IS
    v_age        NUMBER;
    v_memberID   Member.MemberID%TYPE;
    v_paymentID  Payment.PaymentID%TYPE;
    v_maxPayID   NUMBER;
    v_count      NUMBER;
    v_method     VARCHAR2(20);
    v_bankAcc    VARCHAR2(10);
BEGIN
    v_age := calculate_age(p_dob);
    IF v_age < 12 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Member must be at least 12
years old.');
```

years old.');

```

    END IF;

    SELECT COUNT(*) INTO v_count
    FROM Member
    WHERE MemberIC = p_ic;

    IF v_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Member IC already
exists.');
```

exists.');

```

    END IF;

    IF INSTR(p_email, '@') = 0 THEN
        RAISE_APPLICATION_ERROR(-20003, 'Invalid email format.');
```

Invalid email format.');

```

    END IF;

    v_method := CASE TRUNC(DBMS_RANDOM.VALUE(1,4))
        WHEN 1 THEN 'Debit Card'
        WHEN 2 THEN 'Credit Card'
        ELSE 'Online Banking'
    END;

    v_bankAcc := TO_CHAR(TRUNC(DBMS_RANDOM.VALUE(1000000000,
9999999999)));

    SELECT NVL(MAX(TO_NUMBER(SUBSTR(PaymentID,2))), 507)
    INTO v_maxPayID
    FROM Payment;

    v_paymentID := 'P' || LPAD(v_maxPayID + 1, 5, '0');
```

```

BEGIN
    INSERT INTO Payment (
        PaymentID, PaymentDate, PaymentMethod, PaymentStatus,
        BankAccount
    ) VALUES (
        v_paymentID, SYSDATE, v_method, 'Completed', v_bankAcc
    );
EXCEPTION
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20004, 'Payment record could
not be created: ' || SQLERRM);
END;

v_memberID := 'M' || LPAD(member_seq.NEXTVAL, 5, '0');

INSERT INTO Member (
    MemberID, MemberName, DateOfBirth, MemberGender,
    MemberEmail, MemberPhoneNo,
    RegistrationDate, MemberPoints, MemberStatus, MemberIC,
    PaymentID
) VALUES (
    v_memberID, p_name, p_dob, p_gender, p_email, p_phone,
    SYSDATE, 0, 'Active', p_ic, v_paymentID
);

DBMS_OUTPUT.PUT_LINE('Member registered successfully!');
DBMS_OUTPUT.PUT_LINE('Member ID: ' || v_memberID);
DBMS_OUTPUT.PUT_LINE('Payment ID: ' || v_paymentID);
DBMS_OUTPUT.PUT_LINE('Registration Fee: RM 10.00');

EXCEPTION
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20099, 'An unexpected error
occurred: ' || SQLERRM);
END;
/

```

Sample Output:

```

SQL> EXEC register_member('Alice', TO_DATE('1997-05-05','YYYY-MM-DD'), 'Female', 'alice.tan@example.com', '01122334455', '970505-05-5679');
Member registered successfully!
Member ID: M00103
Payment ID: P00671
Registration Fee: RM 10.00

PL/SQL procedure successfully completed.

Error Handling:

SQL> EXEC register_member('Chong', TO_DATE('2015-01-01','YYYY-MM-DD'), 'Male', 'chong@gmail.com', '0123456789', '150101-01-0001');
BEGIN register_member('Chong', TO_DATE('2015-01-01','YYYY-MM-DD'), 'Male', 'chong@gmail.com', '0123456789', '150101-01-0001'); END;

*
ERROR at line 1:
ORA-20099: An unexpected error occurred: ORA-20001: Member must be at least 12
years old.
ORA-06512: at "GANKHAILI.REGISTER_MEMBER", line 78
ORA-06512: at line 1

SQL> EXEC register_member('Lau', TO_DATE('2005-03-20','YYYY-MM-DD'), 'Male', 'lau.email.com', '01122334455', '960606-06-6789');
BEGIN register_member('Lau', TO_DATE('2005-03-20','YYYY-MM-DD'), 'Male', 'lau.email.com', '01122334455', '960606-06-6789'); END;

*
ERROR at line 1:
ORA-20099: An unexpected error occurred: ORA-20003: Invalid email format.
ORA-06512: at "GANKHAILI.REGISTER_MEMBER", line 78
ORA-06512: at line 1

SQL> EXEC register_member('Alice Tan', TO_DATE('2005-03-20','YYYY-MM-DD'), 'Female', 'alice.tan@example.com', '01122334455', '950505-05-5679');
BEGIN register_member('Alice Tan', TO_DATE('2005-03-20','YYYY-MM-DD'), 'Female', 'alice.tan@example.com', '01122334455', '950505-05-5679'); END;

*
ERROR at line 1:
ORA-20099: An unexpected error occurred: ORA-20002: Member IC already exists.
ORA-06512: at "GANKHAILI.REGISTER_MEMBER", line 78
ORA-06512: at line 1

```

4.3.5 Trigger 1: Check Extension Timing and Update Booking

Purpose:

The purpose of this trigger is to enforce the policy that booking extensions must be requested at least 48 hours before the scheduled departure time by automatically approving or rejecting extension requests based on this timing rule. If the extension is requested too late, it sets the extension status to “Rejected” and removes any extension fee, raising an error to prevent the update. If requested on time, it approves the extension, updates the booking detail status to “Extended”, marks the associated ticket as “Available” and adjusts the total booking price by adding the applicable extension fee. This automated process ensures consistent enforcement of business rules, accurate billing and proper management of seat availability.

Trigger Code:

```
SET SERVEROUTPUT ON

CREATE OR REPLACE TRIGGER trg_newExtension
BEFORE UPDATE OF ExtensionID ON BookingDetails
FOR EACH ROW
DECLARE
    v_departureTime    BusSchedule.DepartureTime%TYPE;
    v_busScheduleID    Ticket.BusScheduleID%TYPE;
    v_extensionTarget  Extension.ExtendedDate%TYPE;
    v_extensionFee      Extension.ExtensionFee%TYPE;
BEGIN
    IF :NEW.ExtensionID IS NOT NULL THEN
        SELECT t.BusScheduleID, bs.DepartureTime
        INTO v_busScheduleID, v_departureTime
        FROM Ticket t
        JOIN BusSchedule bs ON t.BusScheduleID = bs.BusScheduleID
        WHERE t.TicketID = :NEW.TicketID;

        SELECT ExtendedDate, ExtensionFee
        INTO v_extensionTarget, v_extensionFee
        FROM Extension
        WHERE ExtensionID = :NEW.ExtensionID;

        IF SYSDATE > v_departureTime - 2 THEN

            UPDATE Extension
            SET ExtensionStatus = 'Rejected',
                ExtensionFee = 0
            WHERE ExtensionID = :NEW.ExtensionID;

            RAISE_APPLICATION_ERROR(
                -20002,
                'Extension must be requested at least 48 hours
                before departure. Extension has been rejected.'
            );
        ELSE

            UPDATE Extension
            SET ExtensionStatus = 'Approved',
                ExtendedDate = v_extensionTarget
            WHERE ExtensionID = :NEW.ExtensionID;

            :NEW.BookingDetailsStatus := 'Extended';

            UPDATE Ticket
            SET TicketStatus = 'Available'
            WHERE TicketID = :NEW.TicketID;

            UPDATE Booking
            SET TotalPrice = TotalPrice + NVL(v_extensionFee,0)
```

```

        WHERE BookingID = :NEW.BookingID;

        DBMS_OUTPUT.PUT_LINE('Extension approved.');
```

```

        DBMS_OUTPUT.PUT_LINE('TicketID: ' || :NEW.TicketID ||
', Extension Fee: ' || v_extensionFee);
    END IF;
END IF;
END;
/
```

Sample Output:

```

SQL> UPDATE BookingDetails
  2  SET ExtensionID = 'E00065'
  3  WHERE BookingID = 'B00270' AND TicketID = 'T02000';
Extension approved.
TicketID: T02000, Extension Fee: 5

1 row updated.
```

Error handling:

```

SQL> UPDATE BookingDetails
  2  SET ExtensionID = 'E00003'
  3  WHERE BookingID = 'B00001' AND TicketID = 'T00001';
UPDATE BookingDetails
      *
```

```

ERROR at line 1:
ORA-20002: Extension must be requested at least 48 hours before departure.
Extension has been rejected.
ORA-06512: at "GANKHAILI.TRG_NEWEXTENSION", line 26
ORA-04088: error during execution of trigger 'GANKHAILI.TRG_NEWEXTENSION'
```

4.3.6 Trigger 2: Audit Trail for RequestComplain Table**Purpose:**

The purpose of this trigger is to maintain a detailed audit trail by automatically recording every insert, update and delete operation on the RequestComplain table into the ActionRequestComplain table. For inserts it logs the new request details along with the user, date and time. For updates it captures both old and new values when fields such as topic, description or status are changed. For deletes it preserves the removed record before deletion. By consistently tracking the action type, responsible user and timestamp the trigger ensures accountability, data integrity and transparency. This audit trail provides management with a reliable record of all modifications and supports monitoring, troubleshooting, compliance and better decision-making.

Trigger Code:

```

DROP TABLE ActionRequestComplain;

CREATE TABLE ActionRequestComplain(
    RequestID          VARCHAR(6),
    RequestTopic       VARCHAR(100),
    RequestDesc        VARCHAR(100),
    RequestStatus      VARCHAR(50),
    RequestMemberID    VARCHAR(6),
    RequestStaffID     VARCHAR(6),
    UserID             VARCHAR(30),
    TransDate          DATE,
```

```

        TransTime          VARCHAR(8),
        TransAction        VARCHAR(10)
    );

SET SERVEROUTPUT ON

CREATE OR REPLACE TRIGGER trg_track_RequestComplain
AFTER INSERT OR UPDATE OR DELETE ON RequestComplain
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO ActionRequestComplain(
            RequestID, RequestTopic, RequestDesc, RequestStatus,
            RequestMemberID, RequestStaffID, UserID, TransDate,
            TransTime, TransAction
        )
        VALUES (
            :NEW.RequestID, :NEW.RequestTopic, :NEW.RequestDesc,
            :NEW.RequestStatus, -- Insert: just new data
            :NEW.RequestMemberID, :NEW.RequestStaffID, USER,
            SYSDATE, TO_CHAR(SYSDATE, 'HH24:MI:SS'), 'INSERT'
        );

    ELSIF UPDATING THEN
        INSERT INTO ActionRequestComplain(
            RequestID, RequestTopic, RequestDesc, RequestStatus,
            RequestMemberID, RequestStaffID, UserID, TransDate,
            TransTime, TransAction
        )
        VALUES (
            :NEW.RequestID,
            CASE
                WHEN :OLD.RequestTopic = :NEW.RequestTopic THEN
:OLD.RequestTopic
            ELSE :OLD.RequestTopic || '-->' || :NEW.RequestTopic
            END,
            CASE
                WHEN :OLD.RequestDesc = :NEW.RequestDesc THEN
:OLD.RequestDesc
            ELSE :OLD.RequestDesc || '-->' || :NEW.RequestDesc
            END,
            CASE
                WHEN :OLD.RequestStatus = :NEW.RequestStatus THEN
:OLD.RequestStatus
            ELSE :OLD.RequestStatus || '-->' ||
:NEW.RequestStatus
            END,
            :NEW.RequestMemberID, :NEW.RequestStaffID, USER,
            SYSDATE, TO_CHAR(SYSDATE, 'HH24:MI:SS'), 'UPDATE'
        );

    ELSIF DELETING THEN
        INSERT INTO ActionRequestComplain(
            RequestID, RequestTopic, RequestDesc, RequestStatus,
            RequestMemberID, RequestStaffID, UserID, TransDate,
            TransTime, TransAction
        )
        VALUES (
            :OLD.RequestID, :OLD.RequestTopic, :OLD.RequestDesc,
:OLD.RequestStatus,
            :OLD.RequestMemberID, :OLD.RequestStaffID, USER,
            SYSDATE, TO_CHAR(SYSDATE, 'HH24:MI:SS'), 'DELETE'
        );
    END IF;
END;
/

```

Sample Output:

REQUESTID	REQUESTTOPIC	REQUESTDESC	REQUESTSTATUS	REQUESTID	REQUESTUSERID	TRANSDATE	TRANSACTIONID
RQ0381	Bus Delay	The bus was delayed by 30 minutes	Submitted	H00002	S00001	GANKHALLI 07-SEP-25	18:19:31 INSERT
RQ0382	Broken AC	AC not working in room 101	Submitted	H00003	S00002	GANKHALLI 07-SEP-25	18:19:38 INSERT
RQ0383	Wi-Fi Issue	Wi-Fi disconnecting frequently	Submitted	H00004	S00003	GANKHALLI 07-SEP-25	18:19:43 INSERT
RQ0001	Crowded waiting Area	There are too few seats for waiting passengers.	In Progress-->Resolved	H00005	S00014	GANKHALLI 07-SEP-25	18:20:01 UPDATE
RQ0002	Late Bus Announcements-->Technician assigned	Delays are not announced in time.	Rejected-->In Progress	H00006	S00002	GANKHALLI 07-SEP-25	18:20:14 UPDATE
RQ0003	No Information Display	There is no functioning screen for bus schedules.-->Wi-Fi fi Submitted-->Resolved xed, tested successfully		H00013	S00018	GANKHALLI 07-SEP-25	18:20:21 UPDATE
RQ0381	Bus Delay	The bus was delayed by 30 minutes	Submitted	H00002	S00001	GANKHALLI 07-SEP-25	18:21:05 DELETE
RQ0382	Broken AC	AC not working in room 101	Submitted	H00003	S00002	GANKHALLI 07-SEP-25	18:21:09 DELETE
RQ0383	Wi-Fi Issue	Wi-Fi disconnecting frequently	Submitted	H00004	S00003	GANKHALLI 07-SEP-25	18:21:17 DELETE
9 rows selected.							

4.3.7 Report 1: Most Popular vs Least Popular Destinations Report**Purpose:**

The purpose of this report is to analyze and compare ticket sales performance across all bus destinations by summarizing both the total number of tickets sold and the total revenue generated for each destination. By identifying the most and least popular destinations based on these metrics, the report provides valuable insights into passenger demand patterns. These insights enable management to optimize route planning, allocate resources more effectively and develop targeted marketing strategies to improve overall operational efficiency and revenue growth. Additionally, the report serves as a key tool for monitoring market trends and making informed business decisions.

Report Code:

```

CREATE INDEX idx_busschedule_destination ON
BusSchedule(Destination);

SET SERVEROUTPUT ON;
set pagesize 100
set linesize 120

CREATE OR REPLACE PROCEDURE prc_DestReport IS

    CURSOR curDestData IS
        SELECT DISTINCT bs.Destination
        FROM BusSchedule bs
        ORDER BY bs.Destination;

    CURSOR curDestStats(p_dest VARCHAR2) IS
        SELECT COUNT(t.TicketID) AS ticket_count,
               NVL(SUM(t.TicketPrice),0) AS total_price
        FROM BusSchedule bs
        LEFT JOIN Ticket t ON t.BusScheduleID = bs.BusScheduleID
        WHERE bs.Destination = p_dest;

    v_dest          VARCHAR2(100);
    v_count          NUMBER;
    v_price          NUMBER(15,2);
    v_max_dest       VARCHAR2(100);
    v_min_dest       VARCHAR2(100);
    v_max_count      NUMBER := NULL;
    v_min_count      NUMBER := NULL;
    v_max_price      NUMBER := 0;
    v_min_price      NUMBER := 0;
    v_line_width     CONSTANT NUMBER := 110;

    v_stats_count    NUMBER;
    v_stats_price    NUMBER(15,2);

BEGIN

    DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));

```

```

    DBMS_OUTPUT.PUT_LINE(LPAD('Most Popular vs Least Popular
Destinations Report', v_line_width, ' '));
    DBMS_OUTPUT.PUT_LINE(LPAD('=', v_line_width, '='));

    DBMS_OUTPUT.PUT_LINE(
        RPAD('Destination',35) ||
        LPAD('Ticket Volume',15) ||
        LPAD('Total Price (RM)',20)
    );
    DBMS_OUTPUT.PUT_LINE(LPAD('-', v_line_width, '-'));

    OPEN curDestData;
    LOOP
        FETCH curDestData INTO v_dest;
        EXIT WHEN curDestData%NOTFOUND;

        OPEN curDestStats(v_dest);
        FETCH curDestStats INTO v_stats_count, v_stats_price;
        CLOSE curDestStats;

        IF v_stats_count = 0 THEN
            CONTINUE;
        END IF;

        DBMS_OUTPUT.PUT_LINE(
            RPAD(v_dest,35) ||
            LPAD(v_stats_count,15) ||
            LPAD(TO_CHAR(v_stats_price,'999,999,999.00'),20)
        );

        IF v_max_count IS NULL OR v_stats_count > v_max_count THEN
            v_max_count := v_stats_count;
            v_max_dest := v_dest;
            v_max_price := v_stats_price;
        END IF;

        IF v_min_count IS NULL OR v_stats_count < v_min_count THEN
            v_min_count := v_stats_count;
            v_min_dest := v_dest;
            v_min_price := v_stats_price;
        END IF;

    END LOOP;
    CLOSE curDestData;

    DBMS_OUTPUT.PUT_LINE(LPAD('-', v_line_width, '-'));
    DBMS_OUTPUT.PUT_LINE(
        RPAD('Most Popular Destination : ' || v_max_dest, 50) ||
        ' | Tickets: ' || LPAD(v_max_count,5) ||
        ' | Total Price: RM ' ||
        LPAD(TO_CHAR(v_max_price,'999,999,999.00'),15)
    );
    DBMS_OUTPUT.PUT_LINE(
        RPAD('Least Popular Destination : ' || v_min_dest, 50) ||
        ' | Tickets: ' || LPAD(v_min_count,5) ||
        ' | Total Price: RM ' ||
        LPAD(TO_CHAR(v_min_price,'999,999,999.00'),15)
    );

    DBMS_OUTPUT.PUT_LINE(LPAD('-', v_line_width, '-'));
    DBMS_OUTPUT.PUT_LINE('Generated on: ' ||
    TO_CHAR(SYSDATE,'DD-MON-YYYY HH24:MI:SS'));
    DBMS_OUTPUT.PUT_LINE(LPAD('=', 50, '=') || RPAD(' END OF REPORT
', v_line_width - 50, '='));

EXCEPTION
    WHEN OTHERS THEN

```

```

        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END prc_DestReport;
/

EXEC PRC_DESTREPORT;

```

Sample Output:

```

=====
Most Popular vs Least Popular Destinations Report
=====
Destination                Ticket Volume    Total Price (RM)
-----
Alor Setar                  90              4,600.00
Ipoh                        220             11,000.00
Johor Bahru                 150             7,550.00
Kota Bharu                  100             5,000.00
Kuala Terengganu            60              3,000.00
Kuantan                     140             7,000.00
Melaka                      140             7,000.00
Penang                      140             7,000.00
Seremban                    60              3,000.00
-----
Most Popular Destination   : Ipoh           | Tickets: 220 | Total Price: RM    11,000.00
Least Popular Destination  : Kuala Terengganu | Tickets:  60 | Total Price: RM     3,000.00
-----
Generated on: 07-SEP-2025 20:07:17
===== END OF REPORT =====

```

4.3.8 Report 2: Overall Refund Approval vs Rejection Report by Reason**Purpose:**

The purpose of this report is to provide a comprehensive summary of refund requests segmented by their refund reasons. The purpose of this report is to provide a comprehensive summary of refund requests segmented by their refund reasons. It details the counts of refunds approved and rejected for each reason, along with corresponding approval and rejection rates. This report helps stakeholders analyze refund trends, evaluate the effectiveness of refund policies and identify reasons with unusually high approval or rejection rates for further investigation or action.

Report Code:

```

CREATE INDEX idx_refund_status_reason ON Refund(RefundStatus,
RefundReason);

SET SERVEROUTPUT ON;
set pagesize 100
set linesize 120

CREATE OR REPLACE PROCEDURE prc_RefundSumRpt_Total IS

    CURSOR cur_reason IS
        SELECT DISTINCT r.RefundReason
        FROM Refund r
        WHERE r.RefundStatus IN ('Approved','Rejected')
        ORDER BY 1;

    CURSOR cur_detail(p_reason Refund.RefundReason%TYPE) IS
        SELECT r.RefundStatus,
               COUNT(*) AS status_count
        FROM Refund r
        WHERE r.RefundStatus IN ('Approved','Rejected')
              AND r.RefundReason = p_reason
        GROUP BY r.RefundStatus;

    v_total_approved NUMBER := 0;
    v_total_rejected NUMBER := 0;
    v_total_count     NUMBER := 0;
    v_reason_approved NUMBER;

```

```

v_reason_rejected NUMBER;
v_reason_total     NUMBER;
v_approval_rate    NUMBER(6,2);
v_reject_rate      NUMBER(6,2);
v_line_width CONSTANT NUMBER := 120;

FUNCTION format_reason(p_reason VARCHAR2, p_len NUMBER) RETURN
VARCHAR2 IS
BEGIN
    RETURN RPAD(
        SUBSTR(p_reason,1,p_len-3) ||
        CASE WHEN LENGTH(p_reason) > p_len-3 THEN '...'
ELSE '' END,
        p_len
    );
END;

BEGIN
    DBMS_OUTPUT.PUT_LINE(RPAD('=', v_line_width, '='));
    DBMS_OUTPUT.PUT_LINE(RPAD('Overall Refund Approval vs Rejection
Report by Reason', (v_line_width+55)/2));
    DBMS_OUTPUT.PUT_LINE(RPAD('=', v_line_width, '='));

    DBMS_OUTPUT.PUT_LINE(
        RPAD('Reason',60) ||
        RPAD('Approved Qty',15) ||
        RPAD('Rejected Qty',15) ||
        RPAD('Approval Rate',15) ||
        RPAD('Reject Rate',15)
    );
    DBMS_OUTPUT.PUT_LINE(RPAD('-', v_line_width, '-'));

    FOR r_rec IN cur_reason LOOP

        v_reason_approved := 0;
        v_reason_rejected := 0;

        FOR d_rec IN cur_detail(r_rec.RefundReason) LOOP
            IF d_rec.RefundStatus = 'Approved' THEN
                v_reason_approved := d_rec.status_count;
            ELSEIF d_rec.RefundStatus = 'Rejected' THEN
                v_reason_rejected := d_rec.status_count;
            END IF;
        END LOOP;

        v_reason_total := v_reason_approved + v_reason_rejected;

        IF v_reason_total > 0 THEN
            v_approval_rate := ROUND(v_reason_approved /
v_reason_total * 100,2);
            v_reject_rate  := ROUND(v_reason_rejected /
v_reason_total * 100,2);
        ELSE
            v_approval_rate := 0;
            v_reject_rate  := 0;
        END IF;

        DBMS_OUTPUT.PUT_LINE(
            format_reason(r_rec.RefundReason,60) ||
            LPAD(v_reason_approved,10) ||
            LPAD(v_reason_rejected,15) ||
            LPAD(TO_CHAR(v_approval_rate,'990.00') || '%',15) ||
            LPAD(TO_CHAR(v_reject_rate,'990.00') || '%',15)
        );

        v_total_approved := v_total_approved + v_reason_approved;
        v_total_rejected := v_total_rejected + v_reason_rejected;
    
```

```

        v_total_count      := v_total_count + v_reason_total;
    END LOOP;

    IF v_total_count > 0 THEN
        v_approval_rate := ROUND(v_total_approved / v_total_count *
100,2);
        v_reject_rate   := ROUND(v_total_rejected / v_total_count *
100,2);
    ELSE
        v_approval_rate := 0;
        v_reject_rate   := 0;
    END IF;

    DBMS_OUTPUT.PUT_LINE(RPAD('-', v_line_width, '-'));
    DBMS_OUTPUT.PUT_LINE(
        RPAD('Overall Total',60) ||
        LPAD(v_total_approved,10) ||
        LPAD(v_total_rejected,15) ||
        LPAD(TO_CHAR(v_approval_rate,'990.00') || '%',15) ||
        LPAD(TO_CHAR(v_reject_rate,'990.00') || '%',15)
    );
    DBMS_OUTPUT.PUT_LINE(RPAD('-', v_line_width, '-'));
    DBMS_OUTPUT.PUT_LINE('Generated on: ' ||
TO_CHAR(SYSDATE,'DD-MON-YYYY HH24:MI:SS'));
    DBMS_OUTPUT.PUT_LINE(LPAD('=', 50, '=') || RPAD(' END OF REPORT
', v_line_width - 50, '='));

END prc_RefundSumRpt_Total;
/

EXEC prc_RefundSumRpt_Total;

```

Sample Output:

```

=====
Overall Refund Approval vs Rejection Report by Reason
=====
Reason                                     Approved Qty   Rejected Qty   Approval Rate   Reject Rate
-----
Bus breakdown caused trip cancellation           1             0         100.00%        0.00%
Bus delayed beyond acceptable limit             2             0         100.00%        0.00%
Bus rerouted without prior notice               3             0         100.00%        0.00%
Cancellation requested after departure time      2             1          66.67%        33.33%
Conflict with Job                             1             1          50.00%        50.00%
Double booking by system, one trip canceled by operator 2             0         100.00%        0.00%
Missed the bus                                0             2           0.00%       100.00%
Operator rescheduled departure time             3             0         100.00%        0.00%
Passenger missed connecting service             2             1          66.67%        33.33%
Rescheduled my plan                           1             0         100.00%        0.00%
Route canceled by bus company on short notice    1             0         100.00%        0.00%
Trip canceled                                  2             0         100.00%        0.00%
Trip canceled due to fuel shortage              3             0         100.00%        0.00%
Trip removed from schedule after booking confirmation 11             0         100.00%        0.00%
Trip rescheduled by company causing conflict    1             0         100.00%        0.00%
Trip time changed unexpectedly by operator       1             0         100.00%        0.00%
Unexpected route diversion causing delay         2             0         100.00%        0.00%
-----
Overall Total                                38             5          88.37%        11.63%
-----
Generated on: 07-SEP-2025 18:25:47
===== END OF REPORT =====

```

4.4 Kit Chin Jie Ying

4.4.1 Query 1: Refund History of Member

Purpose:

The purpose of this query is to generate a detailed report of refund transactions made by members. It provides information such as member details, refund IDs, dates, reasons, statuses and refund amounts, allowing for a complete view of all refunds linked to ticket bookings. The report is structured to display subtotals of refund amounts for each member and a grand total across all members, helping management track both individual and overall refund trends. This supports financial reconciliation, highlights refund patterns and ensures transparency in customer-related refund processing.

SQL statement:

```
SET SERVEROUTPUT ON
SET LINESIZE 300
SET PAGESIZE 200

-- Set up the view
CREATE OR REPLACE VIEW RefundHistory AS
SELECT
    m.MemberID,
    m.MemberName,
    r.RefundID,
    TO_CHAR(r.RefundDate, 'DD-MON-YYYY') AS RefundDate,
    r.RefundReason,
    r.RefundStatus,
    r.RefundAmount
FROM Member m
JOIN Booking b
    ON m.MemberID = b.MemberID
JOIN BookingDetails bd
    ON b.BookingID = bd.BookingID
JOIN Refund r
    ON bd.RefundID = r.RefundID;

-- Column formatting
COLUMN MemberID          FORMAT A15 HEADING 'Member ID'
COLUMN MemberName        FORMAT A20 TRUNCATE HEADING 'Member Name'
COLUMN RefundID          FORMAT A10 HEADING 'Refund ID'
COLUMN RefundDate        FORMAT A15 HEADING 'Refund Date'
COLUMN RefundReason      FORMAT A40 WORD_WRAPPED HEADING 'Reason'
COLUMN RefundStatus      FORMAT A15 HEADING 'Status'
COLUMN RefundAmount      FORMAT 999,999.99 HEADING 'Amount (RM)'

-- Title and pagination settings
TTITLE COL 55 'Refund History of Member' -
        SKIP 1 ' ' -
        LEFT 'Page:' FORMAT 999 SQL.PNO -
        SKIP 1 ' '

-- Break and compute settings for subtotals
BREAK ON MemberID SKIP 1
COMPUTE SUM LABEL 'Subtotal' OF RefundAmount ON MemberID

-- Query with grand total included via UNION ALL
SELECT MemberID, MemberName, RefundID, RefundDate, RefundReason,
RefundStatus, RefundAmount
FROM RefundHistory
UNION ALL
SELECT NULL, NULL, NULL, NULL, 'Grand Total', NULL,
SUM(RefundAmount)
FROM RefundHistory
```

```
ORDER BY MemberID NULLS LAST, RefundDate;

-- Clean up
CLEAR COLUMNS
CLEAR BREAKS
CLEAR COMPUTES
TTITLE OFF;
```

Sample Output:

Refund History of Member						
Page: 1						
Member ID	Member Name	Refund ID	Refund Date	Reason	Status	Amount (RM)
M00001	Alice Tan Mei Ling	R00031	18-JAN-2023	Bus rerouted without prior notice	Approved	36.54

Subtotal						36.54
M00002	Benjamin Lee Wei	R00041	01-MAY-2023	Bus rerouted without prior notice	Approved	36.54
	Benjamin Lee Wei	R00040	12-FEB-2023	Trip removed from schedule after booking confirmation	Approved	36.54
	Benjamin Lee Wei	R00021	19-JAN-2023	Trip removed from schedule after booking confirmation	Approved	36.54

Subtotal						109.62
M00003	Chong Li Hua	R00047	10-JUL-2023	Unexpected route diversion causing delay	Approved	36.54
	Chong Li Hua	R00022	18-JAN-2023	Trip removed from schedule after booking confirmation	Approved	36.54

Subtotal						73.08
M00004	Daniel Wong Jun	R00048	10-JUL-2023	Trip removed from schedule after booking confirmation	Approved	36.54
	Daniel Wong Jun	R00042	10-MAY-2023	Cancellation requested after departure time	Approved	36.54

Subtotal						73.08
M00005	Emily Ng Jia	R00032	17-JAN-2023	Cancellation requested after departure time	Approved	36.54

Subtotal						36.54
M00006	Farid Ahmad	R00043	10-MAY-2023	Bus delayed beyond acceptable limit	Approved	36.54
	Farid Ahmad	R00033	17-JAN-2023	Bus delayed beyond acceptable limit	Approved	36.54
	Farid Ahmad	R00049	22-JUL-2023	Trip removed from schedule after booking confirmation	Approved	36.54

Subtotal						109.62
M00007	Grace Lim Siew	R00034	16-JAN-2023	Trip canceled due to fuel shortage	Approved	36.54
	Grace Lim Siew	R00024	18-JAN-2023	Trip canceled	Approved	36.54
	Grace Lim Siew	R00023	19-JAN-2023	Rescheduled my plan	Approved	36.54

Subtotal						109.62
M00008	Hafiz Roslan	R00035	16-JAN-2023	Passenger missed connecting service	Approved	36.54

Subtotal						36.54
M00009	Isabella Tan	R00025	17-JAN-2023	Trip removed from schedule after booking confirmation	Approved	36.54

Subtotal						36.54

M00010	Jason Ong	R00044	01-JUN-2023	Trip canceled due to fuel shortage	Approved	36.54
	Jason Ong	R00036	15-JAN-2023	Operator rescheduled departure time	Approved	36.54
	Jason Ong	R00026	17-JAN-2023	Trip canceled	Approved	36.54

Subtotal						109.62
M00011	Kelly Chan	R00050	02-AUG-2023	Trip removed from schedule after booking confirmation	Approved	36.54
	Kelly Chan	R00037	15-JAN-2023	Unexpected route diversion causing delay	Approved	36.54
	Kelly Chan	R00027	17-JAN-2023	Trip removed from schedule after booking confirmation	Approved	36.54

Subtotal						109.62
M00014	Nathan Lim	R00045	10-JUN-2023	Passenger missed connecting service	Approved	36.54
	Nathan Lim	R00038	14-JAN-2023	Trip removed from schedule after booking confirmation	Approved	36.54
	Nathan Lim	R00028	19-JAN-2023	Conflict with Job	Approved	36.54

Subtotal						109.62
M00015	Olivia Tan	R00039	14-JAN-2023	Trip removed from schedule after booking confirmation	Approved	36.54
	Olivia Tan	R00046	20-JUN-2023	Operator rescheduled departure time	Approved	36.54

Subtotal						73.08
M00019	Tina Foo	R00030	18-JAN-2023	Trip time changed unexpectedly by operator	Approved	36.54
	Tina Foo	R00029	19-JAN-2023	Double booking by system, one trip canceled by operator	Approved	36.54

Subtotal						73.08

Grand Total						1,096.20

Subtotal						1,096.20

4.4.2 Query 2: Top 10 Members Purchasing Tickets

Purpose:

The purpose of this query is to generate a ranked report of the top ten members based on their ticket purchases. It retrieves each member's total number of tickets purchased and the total amount spent, ranks them in descending order of ticket volume and displays the top contributors along with overall totals. This helps identify the most valuable and active members for business analysis and customer engagement strategies.

SQL statement:

```

SET SERVEROUTPUT ON
SET LINESIZE 300
SET PAGESIZE 200

CREATE OR REPLACE VIEW Top10MemberPurchasing AS
SELECT *
FROM (
    SELECT
        RANK() OVER (ORDER BY COUNT(*) DESC) AS RankNo,
        m.MemberID,
        m.MemberName,
        COUNT(bd.TicketID) AS TotalTickets,
        SUM(t.TicketPrice) AS TotalSpent
    FROM Member m
    JOIN Booking b
        ON m.MemberID = b.MemberID
    JOIN BookingDetails bd
        ON b.BookingID = bd.BookingID
    JOIN Ticket t
        ON bd.TicketID = t.TicketID
    GROUP BY m.MemberID, m.MemberName
    ORDER BY COUNT(bd.TicketID) DESC
)
WHERE RankNo <= 10;

```

```
-- Title
TTITLE COL 10 'Top 10 Members Purchasing Tickets' SKIP 2 -
      LEFT 'Page:' FORMAT 999 SQL.PNO SKIP 2
-- Column formatting
COLUMN RankNo          FORMAT 99          HEADING 'Rank'
COLUMN MemberID        FORMAT A10         HEADING 'Member ID'
COLUMN MemberName      FORMAT A20 TRUNCATE HEADING 'Member Name'
COLUMN TotalTickets    FORMAT 999,999     HEADING 'Tickets'
COLUMN TotalSpent      FORMAT 9,999,999.00 HEADING 'Total Spent (RM) '

-- Compute totals
BREAK ON REPORT
COMPUTE SUM LABEL 'Sum:' OF TotalTickets TotalSpent ON REPORT

SELECT *
FROM Top10MemberPurchasing;

CLEAR COLUMNS
CLEAR BREAKS
CLEAR COMPUTES
TTITLE OFF;
```

Sample Output:

Top 10 Members Purchasing Tickets				
Page: 1				
Rank	Member ID	Member Name	Tickets	Total Spent(RM)
1	M00046	Adrian Teh	41	2,050.00
2	M00004	Daniel Wong Jun	38	1,900.00
3	M00012	Leonard Goh	37	1,850.00
4	M00008	Hafiz Roslan	27	1,350.00
5	M00019	Tina Foo	25	1,250.00
6	M00002	Benjamin Lee Wei	24	1,205.00
7	M00007	Grace Lim Siew	23	1,150.00
8	M00003	Chong Li Hua	22	1,100.00
8	M00005	Emily Ng Jia	22	1,100.00
10	M00013	Monica Teo	21	1,050.00
10	M00097	Alyssa Tan	21	1,050.00
Sum:			301	15,055.00

4.4.3 Procedure 1: Ticket Generation for Bus Scheduled**Purpose:**

The procedure is designed to automate the generation of tickets for a specific bus schedule. Its purpose is to ensure that tickets are created systematically based on the bus type, capacity and schedule details while maintaining consistency in ticket numbering. When executed, the procedure first checks whether the given bus schedule exists and is marked as scheduled. It then aligns the ticket sequence with the latest ticket ID in the system to avoid numbering conflicts. Using the bus information linked to the schedule, the procedure determines the seating capacity and applies predefined pricing rules according to the type of bus, such as single deck, double decker, luxury coach or mini bus. For each available seat, a new ticket is generated with a unique identifier, seat number, ticket type, price and an initial status of

available seats. The procedure also prevents duplicate seat assignments by verifying whether a ticket for the same seat and schedule already exists before inserting. In addition, it includes exception handling to skip duplicates gracefully and capture any unexpected errors. This approach provides a reliable and automated method for creating tickets, ensuring accurate pricing, consistent seat allocation and proper integration with the bus scheduling system.

Procedure code:

```

SET SERVEROUTPUT ON
SET LINESIZE 300
SET PAGESIZE 200

DROP SEQUENCE ticketid_seq;
CREATE SEQUENCE ticketid_seq
MINVALUE 1101
MAXVALUE 999999
START WITH 1101
INCREMENT BY 1
NOCACHE;

CREATE OR REPLACE PROCEDURE prc_generate_tickets (
    p_BusScheduleID IN BusSchedule.BusScheduleID%TYPE
) IS
    v_plateNo      BusSchedule.PlatoNumber%TYPE;
    v_capacity      Bus.Capacity%TYPE;
    v_busType       Bus.BusType%TYPE;
    v_seatNo        VARCHAR2(3);
    v_price         NUMBER(10,2);
    v_max_id        NUMBER;
    v_curr_val      NUMBER;

    -- Outer cursor: schedule
    CURSOR c_schedule IS
        SELECT PlateNumber
        FROM BusSchedule
        WHERE BusScheduleID = p_BusScheduleID
        AND ScheduleStatus = 'Scheduled';

    -- Inner cursor: bus details
    CURSOR c_bus(p_plateNo VARCHAR2) IS
        SELECT Capacity, BusType
        FROM Bus
        WHERE PlateNumber = p_plateNo;
BEGIN
    -- Check if schedule exists
    OPEN c_schedule;
    FETCH c_schedule INTO v_plateNo;
    IF c_schedule%NOTFOUND THEN
        RAISE_APPLICATION_ERROR(-20001, 'Bus schedule not found or
not scheduled. ');
    END IF;
    CLOSE c_schedule;

    -- Prevent duplicate ticket generation
    SELECT COUNT(*)
    INTO v_max_id
    FROM Ticket
    WHERE BusScheduleID = p_BusScheduleID;

    IF v_max_id > 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Tickets already generated
for schedule '||p_BusScheduleID||'. ');
    END IF;

```

```

-- Align sequence with the highest TicketID
SELECT NVL(MAX(TO_NUMBER(SUBSTR(TicketID, 2))), 0)
INTO v_max_id
FROM Ticket;

-- Get current sequence value
SELECT ticketid_seq.NEXTVAL INTO v_curr_val FROM dual;

-- Only adjust if needed (avoid INCREMENT BY 0 error)
IF v_curr_val <> v_max_id + 1 THEN
    EXECUTE IMMEDIATE 'ALTER SEQUENCE ticketid_seq INCREMENT BY
,
                        || (v_max_id + 1 - v_curr_val);
-- Advance to correct value
SELECT ticketid_seq.NEXTVAL INTO v_curr_val FROM dual;
-- Reset increment to 1
EXECUTE IMMEDIATE 'ALTER SEQUENCE ticketid_seq INCREMENT BY
1';
END IF;

-- Process outer cursor (schedule)
OPEN c_schedule;
LOOP
    FETCH c_schedule INTO v_plateNo;
    EXIT WHEN c_schedule%NOTFOUND;

    -- Process inner cursor (bus details)
    OPEN c_bus(v_plateNo);
    LOOP
        FETCH c_bus INTO v_capacity, v_busType;
        EXIT WHEN c_bus%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE('Generating tickets for Schedule
'||p_BusScheduleID||
                                ' | Bus: '||v_plateNo||' | Type:
'||v_busType);

        FOR i IN 1..v_capacity LOOP
            v_seatNo := TO_CHAR(i);

            -- Pricing logic
            v_price := CASE v_busType
                        WHEN 'Single Deck' THEN 20
                        WHEN 'Double Decker' THEN 35
                        WHEN 'Luxury Coach' THEN 50
                        WHEN 'Mini Bus' THEN 15
                        ELSE 25
            END;

            BEGIN
                -- Insert ticket if seat doesn't already exist
                INSERT INTO Ticket (TicketID, TicketPrice,
TicketStatus, TicketType, SeatNo, BusScheduleID)
                SELECT
                    'T' || LPAD(ticketid_seq.NEXTVAL, 5, '0'),
                    v_price,
                    'Available',
                    'Adult', -- default ticket type, can be
extended to adult/child later
                    v_seatNo,
                    p_BusScheduleID
                FROM dual
                WHERE NOT EXISTS (
                    SELECT 1
                    FROM Ticket
                    WHERE SeatNo = v_seatNo
                    AND BusScheduleID = p_BusScheduleID

```

```
);  
  
DBMS_OUTPUT.PUT_LINE('>> Seat '||v_seatNo||' |  
Price '||v_price);  
  
EXCEPTION  
    WHEN DUP_VAL_ON_INDEX THEN  
        DBMS_OUTPUT.PUT_LINE('** Seat '||v_seatNo||'  
already exists, skipping...');  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('** Error inserting  
seat '||v_seatNo||': '||SQLERRM);  
    END;  
END LOOP;  
  
END LOOP;  
CLOSE c_bus;  
END LOOP;  
CLOSE c_schedule;  
  
DBMS_OUTPUT.PUT_LINE('Ticket generation completed for schedule  
'||p_BusScheduleID||'.');  
  
EXCEPTION  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('** Unexpected error: '||SQLERRM);  
END;  
/  
  
EXEC prc_generate_tickets('BS0001');  
  
SELECT * FROM Ticket WHERE BusScheduleID = 'BS0001' ORDER BY SeatNo;  
  
EXEC prc_generate_tickets('BADID');
```

Sample Output:

```
SQL> EXEC prc_generate_tickets('BS0001');
Generating tickets for Schedule BS0001 | Bus: BU0001 | Type: Single Deck
>> Seat 1 | Adult RM20
>> Seat 2 | Child RM10
>> Seat 2 | Adult RM20
>> Seat 3 | Adult RM20
>> Seat 4 | Adult RM20
>> Seat 5 | Adult RM20
>> Seat 6 | Adult RM20
>> Seat 7 | Adult RM20
>> Seat 8 | Adult RM20
>> Seat 9 | Adult RM20
>> Seat 10 | Adult RM20
>> Seat 11 | Adult RM20
>> Seat 12 | Child RM10
>> Seat 12 | Adult RM20
>> Seat 13 | Adult RM20
>> Seat 14 | Adult RM20
>> Seat 15 | Adult RM20
>> Seat 16 | Adult RM20
>> Seat 17 | Adult RM20
>> Seat 18 | Adult RM20
>> Seat 19 | Adult RM20
>> Seat 20 | Adult RM20
>> Seat 21 | Adult RM20
>> Seat 22 | Child RM10
>> Seat 22 | Adult RM20
>> Seat 23 | Adult RM20
>> Seat 24 | Adult RM20
>> Seat 25 | Adult RM20
>> Seat 26 | Adult RM20
>> Seat 27 | Adult RM20
>> Seat 28 | Adult RM20
>> Seat 29 | Adult RM20
>> Seat 30 | Adult RM20
>> Seat 31 | Adult RM20
>> Seat 32 | Child RM10
>> Seat 32 | Adult RM20
>> Seat 33 | Adult RM20
>> Seat 34 | Adult RM20
>> Seat 35 | Adult RM20
>> Seat 36 | Adult RM20
>> Seat 37 | Adult RM20
>> Seat 38 | Adult RM20
>> Seat 39 | Adult RM20
>> Seat 40 | Adult RM20
Ticket generation completed for schedule BS0001.
```

```
SQL> SELECT * FROM Ticket WHERE BusScheduleID = 'BS0001' ORDER BY SeatNo;
```

TICKET	TICKETPRICE	TICKETSTATUS	TICKETTYPE	SEA	BUSSCH
T01102	20	Available	Adult	1	BS0001
T01112	20	Available	Adult	10	BS0001
T01113	20	Available	Adult	11	BS0001
T01115	20	Available	Adult	12	BS0001
T01114	10	Available	Child	12	BS0001
T01116	20	Available	Adult	13	BS0001
T01117	20	Available	Adult	14	BS0001
T01118	20	Available	Adult	15	BS0001
T01119	20	Available	Adult	16	BS0001
T01120	20	Available	Adult	17	BS0001
T01121	20	Available	Adult	18	BS0001
T01122	20	Available	Adult	19	BS0001
T01104	20	Available	Adult	2	BS0001
T01103	10	Available	Child	2	BS0001
T01123	20	Available	Adult	20	BS0001
T01124	20	Available	Adult	21	BS0001
T01125	10	Available	Child	22	BS0001
T01126	20	Available	Adult	22	BS0001
T01127	20	Available	Adult	23	BS0001
T01128	20	Available	Adult	24	BS0001
T01129	20	Available	Adult	25	BS0001
T01130	20	Available	Adult	26	BS0001
T01131	20	Available	Adult	27	BS0001
T01132	20	Available	Adult	28	BS0001
T01133	20	Available	Adult	29	BS0001
T01105	20	Available	Adult	3	BS0001
T01134	20	Available	Adult	30	BS0001
T01135	20	Available	Adult	31	BS0001
T01136	10	Available	Child	32	BS0001
T01137	20	Available	Adult	32	BS0001
T01138	20	Available	Adult	33	BS0001
T01139	20	Available	Adult	34	BS0001
T01140	20	Available	Adult	35	BS0001
T01141	20	Available	Adult	36	BS0001
T01142	20	Available	Adult	37	BS0001
T01143	20	Available	Adult	38	BS0001
T01144	20	Available	Adult	39	BS0001
T01106	20	Available	Adult	4	BS0001
T01145	20	Available	Adult	40	BS0001
T01107	20	Available	Adult	5	BS0001
T01108	20	Available	Adult	6	BS0001
T01109	20	Available	Adult	7	BS0001
T01110	20	Available	Adult	8	BS0001
T01111	20	Available	Adult	9	BS0001
T00001	50	Booked	Adult	A1	BS0001
T00002	50	Booked	Adult	A2	BS0001
T00003	50	Booked	Adult	A3	BS0001
T00004	50	Available	Adult	A4	BS0001
T00005	50	Booked	Adult	A5	BS0001
T00006	50	Available	Adult	B1	BS0001
T00007	50	Booked	Child	B2	BS0001

```
SQL> EXEC prc_generate_tickets('BS0001');
** Unexpected error: ORA-20002: Tickets already generated for schedule BS0001.
```

4.4.4 Procedure 2: Manage Shop Rental

Purpose:

The purpose of this procedure is designed to manage the process of recording shop rentals in a shopping complex or similar environment. Its main purpose is to ensure that all rental entries are accurate, consistent and free from conflicts. When a rental is registered, the procedure first confirms that the staff member handling the process exists in the system. It then checks whether the shop is already recorded; if not, a new shop entry is automatically created. Before the rental is saved, the procedure validates that the rental period does not overlap with any existing contracts for the same shop, which helps prevent scheduling conflicts. Once these checks are satisfied, the rental information is stored in the system, linking it with the appropriate staff, shop and payment details. Built-in error handling ensures that issues such as missing staff, duplicate records or overlapping rental periods are detected and reported clearly. Overall, this procedure provides a reliable way to streamline shop rental management while maintaining data integrity.

Procedure code:

```
SET SERVEROUTPUT ON
SET LINESIZE 300
SET PAGESIZE 200

CREATE OR REPLACE FUNCTION calc_rental_fee(
    p_SquareFeet IN NUMBER,
    p_StartDate IN DATE,
    p_EndDate IN DATE
```

```

) RETURN NUMBER IS
    v_months    NUMBER;
    v_baseRate  NUMBER := 10; -- RM10 per square foot per month
(example)
    v_total     NUMBER;
BEGIN
    -- Calculate rental duration in whole months
    v_months := CEIL(MONTHS_BETWEEN(p_EndDate, p_StartDate));

    IF v_months < 1 THEN
        v_months := 1; -- at least 1 month
    END IF;

    -- Total rent = rate * size * months
    v_total := v_baseRate * p_SquareFeet * v_months;

    RETURN v_total;
END;
/

CREATE OR REPLACE PROCEDURE prc_ManageShopRental (
    p_StaffID      IN RentalCollection.StaffID%TYPE,
    p_ShopName     IN Shop.ShopName%TYPE,
    p_ShopType     IN Shop.ShopType%TYPE,
    p_OwnerName   IN Shop.OwnerName%TYPE,
    p_ShopLocation IN Shop.ShopLocation%TYPE,
    p_SquareFeet   IN Shop.SquareFeet%TYPE,
    p_MonthYear    IN DATE,
    p_StartDate    IN DATE,
    p_EndDate      IN DATE,
    p_PaymentID    IN RentalCollection.PaymentID%TYPE
) IS
    v_count      NUMBER;
    v_ShopID     Shop.ShopID%TYPE;
    v_RentAmount NUMBER(12,2);

    -- User-defined exception
    ex_staff_not_found EXCEPTION;
    PRAGMA EXCEPTION_INIT(ex_staff_not_found, -20001);

BEGIN
    -- Check Staff exists
    SELECT COUNT(*) INTO v_count
    FROM Staff
    WHERE StaffID = p_StaffID;

    IF v_count = 0 THEN
        RAISE ex_staff_not_found; -- use the pragma-linked exception
    END IF;

    -- Check if the shop already exists
    BEGIN
        SELECT ShopID INTO v_ShopID
        FROM Shop
        WHERE ShopName      = p_ShopName
           AND ShopType     = p_ShopType
           AND OwnerName    = p_OwnerName
           AND ShopLocation = p_ShopLocation;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            -- Shop does not exist, create a new one
            v_ShopID := LPAD(shopid_seq.NEXTVAL, 6, '0');
            INSERT INTO Shop (ShopID, ShopName, ShopType, OwnerName,
ShopLocation, SquareFeet)
            VALUES (v_ShopID, p_ShopName, p_ShopType, p_OwnerName,
p_ShopLocation, p_SquareFeet);

```

```
END;

-- Check for overlapping rental dates
SELECT COUNT(*) INTO v_count
FROM RentalCollection
WHERE ShopID = v_ShopID
   AND (p_StartDate BETWEEN StartDate AND EndDate
        OR p_EndDate BETWEEN StartDate AND EndDate
        OR StartDate BETWEEN p_StartDate AND p_EndDate
        OR EndDate BETWEEN p_StartDate AND p_EndDate);

IF v_count > 0 THEN
    RAISE_APPLICATION_ERROR(-20006,
        'Rental for Shop ' || p_ShopName ||
        ' overlaps with existing rental dates.');
```

```
END IF;

-- Calculate rent
v_RentAmount := calc_rental_fee(p_SquareFeet, p_StartDate,
p_EndDate);

-- Insert rental record
INSERT INTO RentalCollection (
    StaffID, ShopID, MonthYear, StartDate, EndDate, RentAmount,
    PaymentID
) VALUES (
    p_StaffID, v_ShopID, p_MonthYear, p_StartDate, p_EndDate,
    v_RentAmount, p_PaymentID
);

DBMS_OUTPUT.PUT_LINE('Rental record added for ShopID ' ||
v_ShopID ||
                        ' with Rent RM' || v_RentAmount);

COMMIT;

EXCEPTION
    WHEN ex_staff_not_found THEN
        RAISE_APPLICATION_ERROR(-20001, 'Staff ' || p_StaffID || '
not found.');
```

```
    WHEN DUP_VAL_ON_INDEX THEN
        RAISE_APPLICATION_ERROR(-20003,
            'Duplicate rental record for Shop ' || v_ShopID ||
            ' in ' || TO_CHAR(p_MonthYear, 'MM-YYYY'));
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20005, 'Unexpected error: ' ||
SQLERRM);
END prc_ManageShopRental;
/
```

Sample Output:

```

SQL> BEGIN
2      prc_ManageShopRental(
3          p_StaffID      => 'S00001',
4          p_ShopName     => 'Test Shop 1',
5          p_ShopType     => 'Retail',
6          p_OwnerName    => 'Alice Tan',
7          p_ShopLocation => 'Lot 101, Main Street',
8          p_SquareFeet   => 1200,
9          p_MonthYear    => TO_DATE('01-09-2025', 'DD-MM-YYYY'),
10         p_StartDate    => TO_DATE('01-09-2025', 'DD-MM-YYYY'),
11         p_EndDate      => TO_DATE('30-09-2025', 'DD-MM-YYYY'),
12         p_PaymentID    => 'P00183'
13     );
14 END;
15 /
Rental record added for ShopID 000021 with Rent RM12000

```

```

SQL> BEGIN
2      prc_ManageShopRental(
3          p_StaffID      => 'S00001',
4          p_ShopName     => 'Test Shop 1',
5          p_ShopType     => 'Retail',
6          p_OwnerName    => 'Alice Tan',
7          p_ShopLocation => 'Lot 101, Main Street',
8          p_SquareFeet   => 1200,
9          p_MonthYear    => TO_DATE('01-09-2025', 'DD-MM-YYYY'),
10         p_StartDate    => TO_DATE('01-09-2025', 'DD-MM-YYYY'),
11         p_EndDate      => TO_DATE('30-09-2025', 'DD-MM-YYYY'),
12         p_PaymentID    => 'P00183'
13     );
14 END;
15 /
BEGIN
*
ERROR at line 1:
ORA-20005: Unexpected error: ORA-20006: Rental for Shop Test Shop 1 overlaps with existing rental dates.
ORA-06512: at "TESTING.PRC_MANAGESHOPRENTAL", line 85
ORA-06512: at line 2

```

```

SQL> BEGIN
2      prc_ManageShopRental(
3          p_StaffID      => 'S10001',
4          p_ShopName     => 'Test Shop 1',
5          p_ShopType     => 'Retail',
6          p_OwnerName    => 'Alice Tan',
7          p_ShopLocation => 'Lot 101, Main Street',
8          p_SquareFeet   => 1200,
9          p_MonthYear    => TO_DATE('01-09-2025', 'DD-MM-YYYY'),
10         p_StartDate    => TO_DATE('01-09-2025', 'DD-MM-YYYY'),
11         p_EndDate      => TO_DATE('30-09-2025', 'DD-MM-YYYY'),
12         p_PaymentID    => 'P00183'
13     );
14 END;
15 /
BEGIN
*
ERROR at line 1:
ORA-20001: Staff S10001 not found.
ORA-06512: at "TESTING.PRC_MANAGESHOPRENTAL", line 79
ORA-06512: at line 2

```

4.4.5 Trigger 1: Booking Total and Seat Availability Management**Purpose:**

The purpose of this trigger is to automatically maintain booking totals and seat availability whenever ticket details are added to or removed from a booking. When a new booking detail is inserted, the trigger retrieves the ticket price, applies tax and

updates the total booking amount accordingly. It also checks the availability of seats for the associated bus schedule, ensuring that no booking is allowed once the bus is full, while reducing the available seat count when a seat is successfully booked. Conversely, when a booking detail is deleted, the trigger reduces the booking total without allowing negative values and restores the corresponding seat back into the bus schedule. By handling these updates automatically, the trigger ensures data integrity, prevents overbooking and keeps both financial and operational records accurate in real time.

Trigger Code:

```

SET SERVEROUTPUT ON
SET LINESIZE 300
SET PAGESIZE 200

CREATE OR REPLACE TRIGGER trg_booking_total_seat
AFTER INSERT OR DELETE ON BookingDetails
FOR EACH ROW
DECLARE
    v_price          NUMBER(10,2);
    v_taxRate        NUMBER := 0.06;
    v_BusScheduleID VARCHAR2(20);
    v_seat           NUMBER;
BEGIN
    IF INSERTING THEN
        -- Get ticket price and bus schedule
        SELECT TicketPrice, BusScheduleID
        INTO v_price, v_BusScheduleID
        FROM Ticket
        WHERE TicketID = :NEW.TicketID;

        -- Update booking total
        UPDATE Booking
        SET TotalPrice = NVL(TotalPrice,0) + ROUND(v_price +
(v_price * v_taxRate), 2)
        WHERE BookingID = :NEW.BookingID;

        -- Check available seat
        SELECT AvailableSeat
        INTO v_seat
        FROM BusSchedule
        WHERE BusScheduleID = v_BusScheduleID;

        IF v_seat <= 0 THEN
            RAISE_APPLICATION_ERROR(-20001, 'No available seats for
this bus schedule. ');
        ELSE
            UPDATE BusSchedule
            SET AvailableSeat = AvailableSeat - 1
            WHERE BusScheduleID = v_BusScheduleID;
        END IF;

    ELSIF DELETING THEN
        -- Get ticket price and bus schedule
        SELECT TicketPrice, BusScheduleID
        INTO v_price, v_BusScheduleID
        FROM Ticket
        WHERE TicketID = :OLD.TicketID;

        -- Update booking total but prevent negative
        UPDATE Booking
        SET TotalPrice = GREATEST(0, NVL(TotalPrice,0) -
ROUND(v_price + (v_price * v_taxRate), 2))
        WHERE BookingID = :OLD.BookingID;

        -- Restore seat

```

```

        UPDATE BusSchedule
        SET AvailableSeat = AvailableSeat + 1
        WHERE BusScheduleID = v_BusScheduleID;
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20002, 'Ticket or BusSchedule not
found. ');
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20003, 'Unexpected error: ' ||
SQLERRM);
END;
/

```

Sample Output:

After Trigger:

```

SQL> INSERT INTO BookingDetails VALUES ('B0001','T0001','900101-01-1234','Ali','Completed',NULL,NULL);
1 row created.

SQL> SELECT TotalPrice FROM Booking WHERE BookingID = 'B0001';
TOTALPRICE
-----
      31.8

SQL> SELECT AvailableSeat FROM BusSchedule WHERE BusScheduleID = 'BS0001';
AVAILABLESEAT
-----
          39

SQL> INSERT INTO BookingDetails VALUES ('B0001','T0002','900202-01-5678','Fatimah','Completed',NULL,NULL);
1 row created.

SQL> SELECT TotalPrice FROM Booking WHERE BookingID = 'B0001';
TOTALPRICE
-----
      63.6

SQL> SELECT AvailableSeat FROM BusSchedule WHERE BusScheduleID = 'BS0001';
AVAILABLESEAT
-----
          38

SQL> DELETE FROM BookingDetails WHERE BookingID = 'B0001' AND TicketID = 'T0001';
1 row deleted.

SQL>
SQL> SELECT TotalPrice FROM Booking WHERE BookingID = 'B0001';
TOTALPRICE
-----
      31.8

SQL> SELECT AvailableSeat FROM BusSchedule WHERE BusScheduleID = 'BS0001';
AVAILABLESEAT
-----
          39

```

4.4.6 Trigger 2: Bus Maintenance Audit Logger

Purpose:

This trigger is designed to maintain an audit trail for all operations performed on the Service Details table. Whenever a new record is added, updated or removed, the trigger automatically inserts a corresponding entry into the BusMaintenance_Audit table. The audit record captures the type of action performed along with key service and maintenance information, ensuring that all changes are tracked for accountability, monitoring and historical reference.

Trigger Code:

```

SET SERVEROUTPUT ON
SET LINESIZE 300
SET PAGESIZE 200

DROP SEQUENCE seq_audit_id;

CREATE SEQUENCE seq_audit_id
  START WITH 1
  INCREMENT BY 1
  NOCACHE;

DROP TABLE BusMaintenance_Audit;

CREATE TABLE BusMaintenance_Audit (
  AuditID          NUMBER PRIMARY KEY,
  AuditDate        TIMESTAMP DEFAULT SYSTIMESTAMP,
  ActionType       VARCHAR2(10),
  PlateNumber      VARCHAR2(20),
  ServiceID        VARCHAR2(6),
  StaffID          VARCHAR2(6),
  OldStatus        VARCHAR2(50),
  NewStatus        VARCHAR2(50),
  ServiceTransactionID VARCHAR2(6)
);

SET LINESIZE 1200;  -- widen each row so columns don't wrap
SET PAGESIZE 1000;

COLUMN AuditDate FORMAT A30;
COLUMN ActionType FORMAT A10;
COLUMN PlateNumber FORMAT A15;
COLUMN ServiceID FORMAT A10;
COLUMN StaffID FORMAT A6;
COLUMN OldStatus FORMAT A20;
COLUMN NewStatus FORMAT A20;
SELECT * FROM BusMaintenance_Audit ORDER BY AuditID;

CREATE OR REPLACE TRIGGER trg_ServiceDetails_Audit
AFTER INSERT OR UPDATE OR DELETE ON ServiceDetails
FOR EACH ROW
DECLARE
  v_staffid StaffAllocation.StaffID%TYPE;
BEGIN
  -- Try to get StaffID related to Service
  BEGIN
    SELECT StaffID
    INTO v_staffid
    FROM StaffAllocation
    WHERE ServiceID = NVL(:NEW.ServiceID, :OLD.ServiceID)
    AND ROWNUM = 1;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      v_staffid := NULL;
  END;
END;
```

```

-- Insert audit record
IF INSERTING THEN
    INSERT INTO BusMaintenance_Audit
        (AuditID, ActionType, PlateNumber, ServiceID, StaffID,
        OldStatus, NewStatus)
    VALUES (seq_audit_id.NEXTVAL, 'INSERT',
        :NEW.PlateNumber, :NEW.ServiceID, v_staffid,
        NULL, :NEW.ServiceStatus);

    ELSIF UPDATING THEN
        INSERT INTO BusMaintenance_Audit
            (AuditID, ActionType, PlateNumber, ServiceID, StaffID,
            OldStatus, NewStatus)
        VALUES (seq_audit_id.NEXTVAL, 'UPDATE',
            :NEW.PlateNumber, :NEW.ServiceID, v_staffid,
            :OLD.ServiceStatus, :NEW.ServiceStatus);

    ELSIF DELETING THEN
        INSERT INTO BusMaintenance_Audit
            (AuditID, ActionType, PlateNumber, ServiceID, StaffID,
            OldStatus, NewStatus)
        VALUES (seq_audit_id.NEXTVAL, 'DELETE',
            :OLD.PlateNumber, :OLD.ServiceID, v_staffid,
            :OLD.ServiceStatus, NULL);

    END IF;
END;
/

```

Sample Output:

```

SQL>
SQL> INSERT INTO ServiceDetails (ServiceTransactionID, ServiceStatus, ServiceDate, ServiceCompleteDate, PlateNumber, ServiceID)
2 VALUES ('ST0001','In Progress',SYSDATE, SYSDATE+2, 'BU0001','SV0001');
1 row created.

SQL> SELECT * FROM BusMaintenance_Audit ORDER BY AuditID;

  AUDITID AUDITDATE                ACTIONTYPE PLATENUMBER  SERVICEID  STAFFI OLDSTATUS  NEWSTATUS  SERVIC
-----
2 07-SEP-25 08.58.57.444000 PM  INSERT     BU0001      SV0001     S0001              In Progress

SQL> UPDATE ServiceDetails
2 SET ServiceStatus = 'Completed'
3 WHERE ServiceTransactionID = 'ST0001';
1 row updated.

SQL> SELECT * FROM BusMaintenance_Audit ORDER BY AuditID;

  AUDITID AUDITDATE                ACTIONTYPE PLATENUMBER  SERVICEID  STAFFI OLDSTATUS  NEWSTATUS  SERVIC
-----
2 07-SEP-25 08.58.57.444000 PM  INSERT     BU0001      SV0001     S0001              In Progress
3 07-SEP-25 09.00.24.502000 PM  UPDATE     BU0001      SV0001     S0001 In Progress      Completed

SQL> |

SQL> DELETE FROM ServiceDetails
2 WHERE ServiceTransactionID = 'ST0001';
1 row deleted.

SQL>
SQL> SELECT * FROM BusMaintenance_Audit ORDER BY AuditID;

  AUDITID AUDITDATE                ACTIONTYPE PLATENUMBER  SERVICEID  STAFFI OLDSTATUS  NEWSTATUS  SERVIC
-----
2 07-SEP-25 08.58.57.444000 PM  INSERT     BU0001      SV0001     S0001              In Progress
3 07-SEP-25 09.00.24.502000 PM  UPDATE     BU0001      SV0001     S0001 In Progress      Completed
4 07-SEP-25 09.01.21.012000 PM  DELETE     BU0001      SV0001     S0001 Completed

SQL>

```

4.4.7 Report 1: Driver Workload Report

Purpose:

The purpose of this report is to analyze and present the workload of bus drivers across different bus companies for a selected month and year. It provides management with a detailed view of each driver's operational activity, including the total number of trips completed, total driving hours and total distance covered. By aggregating these metrics at the company level, the report highlights overall performance, workload

distribution and potential imbalances among drivers. It identifies drivers who are overloaded handling significantly more trips than average and underloaded who handle significantly fewer trips than average, enabling managers to make more informed decisions about scheduling, workload balancing and resource allocation. At a broader level, the report also summarizes the total workload across all companies and pinpoints the drivers with the maximum and minimum trips, ensuring transparency and fairness in driver management. This helps companies optimize driver assignments, improve operational efficiency and maintain a balanced workload distribution within their workforce.

Report Code:

```
-- BusSchedule: functional index on UPPER(PlateNumber) +
DepartureDate
DROP INDEX idx_bs_plate_dep;
CREATE INDEX idx_bs_plate_dep
    ON BusSchedule (UPPER(PlateNumber), DepartureDate);

-- Bus: functional index on UPPER(BusType) + CompanyID
DROP INDEX idx_bus_comp_type;
CREATE INDEX idx_bus_comp_type
    ON Bus (CompanyID, UPPER(BusType));

-- DriverList: composite index
DROP INDEX idx_dl_driver_sched;
CREATE INDEX idx_dl_driver_sched
    ON DriverList (DriverID, BusScheduleID);

-- Driver: functional index on UPPER(DriverName)
DROP INDEX idx_driver_name;
CREATE INDEX idx_driver_name
    ON Driver (UPPER(DriverName));

-- BusCompany: functional index on UPPER(CompanyName)
DROP INDEX idx_company_name;
CREATE INDEX idx_company_name
    ON BusCompany (UPPER(CompanyName));

-- Create a collection type for company IDs
CREATE OR REPLACE TYPE t_company_list AS TABLE OF VARCHAR2(6);
/

CREATE OR REPLACE PROCEDURE prc_driver_workload_company (
    p_companyID_list IN t_company_list DEFAULT NULL,
    p_year            IN NUMBER,
    p_month           IN NUMBER
) IS

    -- Cursor for companies
    CURSOR compCur IS
        SELECT CompanyID, CompanyName
        FROM BusCompany
        WHERE p_companyID_list IS NULL
            OR CompanyID IN (SELECT COLUMN_VALUE FROM
TABLE(p_companyID_list))
        ORDER BY CompanyName;

    -- Cursor for aggregated driver info per company
    CURSOR driverAggCur(p_companyID BusCompany.CompanyID%TYPE) IS
        SELECT d.DriverID,
            d.DriverName,
            COUNT(*) AS total_trips,
            NVL(SUM((s.ArrivalTime - s.DepartureTime) * 24), 0) AS
total_hours,
```

```

        NVL(SUM(s.Distance), 0) AS total_distance
FROM Driver d
JOIN DriverList dl ON d.DriverID = dl.DriverID
JOIN BusSchedule s ON dl.BusScheduleID = s.BusScheduleID
WHERE d.CompanyID = p_companyID
      AND EXTRACT(YEAR FROM s.DepartureDate) = p_year
      AND EXTRACT(MONTH FROM s.DepartureDate) = p_month
GROUP BY d.DriverID, d.DriverName
ORDER BY d.DriverName;

-- Variables
v_companyID      BusCompany.CompanyID%TYPE;
v_companyName    BusCompany.CompanyName%TYPE;

v_driverID       Driver.DriverID%TYPE;
v_driverName     Driver.DriverName%TYPE;
v_totalTrips     NUMBER;
v_totalHours     NUMBER(12,2);
v_totalDistance  NUMBER(14,2);

v_companyTrips   NUMBER;
v_companyHours   NUMBER(12,2);
v_companyDistance NUMBER(14,2);
v_driverCount    NUMBER;

v_companyMaxTrips NUMBER;
v_companyMinTrips NUMBER;
v_companyAvgTrips NUMBER;

-- Totals across all companies
v_totalTripsAll   NUMBER := 0;
v_totalHoursAll   NUMBER := 0;
v_totalDistanceAll NUMBER := 0;
v_driverCountAll  NUMBER := 0;

v_maxTrips        NUMBER := -1;
v_minTrips        NUMBER := 99999999;
v_maxDriver       VARCHAR2(100);
v_minDriver       VARCHAR2(100);
v_maxCompany      VARCHAR2(100);
v_minCompany      VARCHAR2(100);

BEGIN
  DBMS_OUTPUT.PUT_LINE('');
  DBMS_OUTPUT.PUT_LINE(LPAD('=', 90, '='));
  DBMS_OUTPUT.PUT_LINE('Driver Workload Report for ' ||
TO_CHAR(TO_DATE(p_year||'-'||p_month,'YYYY-MM'),'Month YYYY'));
  DBMS_OUTPUT.PUT_LINE(LPAD('=', 90, '='));
  DBMS_OUTPUT.PUT_LINE('');

  OPEN compCur;
  LOOP
    FETCH compCur INTO v_companyID, v_companyName;
    EXIT WHEN compCur%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('Driver workload by company: ' ||
v_companyName || CHR(10));

    DBMS_OUTPUT.PUT_LINE(
      RPAD('Driver ID',12) ||
      RPAD('Driver Name',24) ||
      LPAD('Total Trips',14) ||
      LPAD('Total Hours',16) ||
      LPAD('Total Distance (km)',22)
    );
    DBMS_OUTPUT.PUT_LINE(RPAD('-', 90, '-'));
  
```

```

-- Reset company totals
v_companyTrips      := 0;
v_companyHours      := 0;
v_companyDistance   := 0;
v_driverCount       := 0;
v_companyMaxTrips   := -1;
v_companyMinTrips   := 9999999;

OPEN driverAggCur(v_companyID);
LOOP
    FETCH driverAggCur INTO v_driverID, v_driverName,
v_totalTrips, v_totalHours, v_totalDistance;
    EXIT WHEN driverAggCur%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(
        RPAD(v_driverID,12) ||
        RPAD(v_driverName,24) ||
        LPAD(TO_CHAR(v_totalTrips,'999G999'),14) ||
        LPAD(TO_CHAR(v_totalHours,'999G999D99') || ' hrs',16) ||
        LPAD(TO_CHAR(v_totalDistance,'999G999G999D99'),22)
    );

    -- Accumulate company totals
    v_companyTrips      := v_companyTrips      + v_totalTrips;
    v_companyHours      := v_companyHours      + v_totalHours;
    v_companyDistance   := v_companyDistance + v_totalDistance;
    v_driverCount       := v_driverCount       + 1;

    -- Accumulate global totals
    v_totalTripsAll     := v_totalTripsAll     + v_totalTrips;
    v_totalHoursAll     := v_totalHoursAll     + v_totalHours;
    v_totalDistanceAll  := v_totalDistanceAll  + v_totalDistance;
    v_driverCountAll    := v_driverCountAll    + 1;

    -- Update min/max
    IF v_totalTrips > v_companyMaxTrips THEN v_companyMaxTrips :=
v_totalTrips; END IF;
    IF v_totalTrips < v_companyMinTrips THEN v_companyMinTrips :=
v_totalTrips; END IF;

    IF v_totalTrips > v_maxTrips THEN
        v_maxTrips      := v_totalTrips;
        v_maxDriver     := v_driverName;
        v_maxCompany    := v_companyName;
    END IF;

    IF v_totalTrips < v_minTrips THEN
        v_minTrips      := v_totalTrips;
        v_minDriver     := v_driverName;
        v_minCompany    := v_companyName;
    END IF;
END LOOP;
CLOSE driverAggCur;

v_companyAvgTrips := CASE WHEN v_driverCount > 0 THEN
ROUND(v_companyTrips / v_driverCount,2) ELSE 0 END;

-- Company totals
DBMS_OUTPUT.PUT_LINE(RPAD('-', 90, '-'));
DBMS_OUTPUT.PUT_LINE('Company Totals for ' || v_companyName);
DBMS_OUTPUT.PUT_LINE('    Total drivers          : ' ||
LPAD(v_driverCount,10));
DBMS_OUTPUT.PUT_LINE('    Total trips            : ' ||
LPAD(TO_CHAR(NVL(v_companyTrips,0),'999G999'),10));
DBMS_OUTPUT.PUT_LINE('    Total hours           : ' ||
LPAD(TO_CHAR(v_companyHours,'999G999D99') || ' hrs',15));
DBMS_OUTPUT.PUT_LINE('    Total distance        : ' ||

```

```

LPAD(TO_CHAR(v_companyDistance,'999G999G999D99') || ' km',20));
    DBMS_OUTPUT.PUT_LINE('    Workload Imbalance      : Max = ' ||
LPAD(v_companyMaxTrips,3) ||
                                ', Min = ' ||
LPAD(v_companyMinTrips,3) ||
                                ', Avg = ' ||
LPAD(v_companyAvgTrips,5));
    DBMS_OUTPUT.PUT_LINE('');

    -- Overload drivers (>120%)
    DBMS_OUTPUT.PUT_LINE('    Overload Drivers (>120% of avg
trips):');
    DECLARE v_overload_found BOOLEAN := FALSE;
    BEGIN
        OPEN driverAggCur(v_companyID);
        LOOP
            FETCH driverAggCur INTO v_driverID, v_driverName,
v_totalTrips, v_totalHours, v_totalDistance;
            EXIT WHEN driverAggCur%NOTFOUND;
            IF v_totalTrips > 1.2 * v_companyAvgTrips THEN
                DBMS_OUTPUT.PUT_LINE('          ' || RPAD(v_driverID,12) || '
- ' || RPAD(v_driverName,20) || ' (' || LPAD(v_totalTrips,3) || '
trips)');
                v_overload_found := TRUE;
            END IF;
        END LOOP;
        CLOSE driverAggCur;
        IF NOT v_overload_found THEN
            DBMS_OUTPUT.PUT_LINE('          -');
        END IF;
    END;

    -- Underload drivers (<50%)
    DBMS_OUTPUT.PUT_LINE('    Underload Drivers (<50% of avg
trips):');
    DECLARE v_underload_found BOOLEAN := FALSE;
    BEGIN
        OPEN driverAggCur(v_companyID);
        LOOP
            FETCH driverAggCur INTO v_driverID, v_driverName,
v_totalTrips, v_totalHours, v_totalDistance;
            EXIT WHEN driverAggCur%NOTFOUND;
            IF v_totalTrips < 0.5 * v_companyAvgTrips THEN
                DBMS_OUTPUT.PUT_LINE('          ' || RPAD(v_driverID,12) || '
- ' || RPAD(v_driverName,20) || ' (' || LPAD(v_totalTrips,3) || '
trips)');
                v_underload_found := TRUE;
            END IF;
        END LOOP;
        CLOSE driverAggCur;
        IF NOT v_underload_found THEN
            DBMS_OUTPUT.PUT_LINE('          -');
        END IF;
    END;

    DBMS_OUTPUT.PUT_LINE(LPAD('=', 90, '='));
END LOOP;
CLOSE compCur;

-- Workload summary for all companies
DBMS_OUTPUT.PUT_LINE('Workload Summary (All Companies)');
DBMS_OUTPUT.PUT_LINE(LPAD('=', 90, '='));
DBMS_OUTPUT.PUT_LINE('Driver with Maximum Trips      : ' ||
v_maxDriver || ' (' || v_maxTrips || ' trips) - ' || v_maxCompany);
DBMS_OUTPUT.PUT_LINE('Driver with Minimum Trips      : ' ||
v_minDriver || ' (' || v_minTrips || ' trips) - ' || v_minCompany);
DBMS_OUTPUT.PUT_LINE('Average Trips per Driver      : ' ||

```

```
TO_CHAR(CASE WHEN v_driverCountAll > 0 THEN ROUND(v_totalTripsAll /
v_driverCountAll, 2) ELSE 0 END));
    DBMS_OUTPUT.PUT_LINE('Total Hours (All Drivers)      : ' ||
TO_CHAR(v_totalHoursAll,'999G999D99') || ' hrs');
    DBMS_OUTPUT.PUT_LINE('Total Distance (All Drivers) : ' ||
TO_CHAR(v_totalDistanceAll,'999G999G999D99') || ' km');
    DBMS_OUTPUT.PUT_LINE(LPAD('=', 90, '='));
    DBMS_OUTPUT.PUT_LINE('Generated on: ' ||
TO_CHAR(SYSDATE,'DD-MON-YYYY HH24:MI:SS'));
    DBMS_OUTPUT.PUT_LINE(LPAD('=',34, '=') || RPAD('END OF REPORT',
56, '='));
END;
/

DECLARE
    v_list t_company_list := t_company_list('C00011','C00012');
BEGIN
    prc_driver_workload_company(v_list, 2025, 7);
END;
/
```

Sample Output:

Driver Workload Report for July 2025				
Driver workload by company: Mega Transit				
Driver ID	Driver Name	Total Trips	Total Hours	Total Distance (km)
D00004	Ahmad Iqbal	11	37.53 hrs	520.00
D00029	Arman Danish	14	55.02 hrs	1,080.00
D00038	Farah Hanim	22	79.05 hrs	1,255.00
D00058	Hana Sofea	21	79.02 hrs	1,420.00
D00066	Iqbal Danish	18	67.52 hrs	880.00
D00073	Irfan Zikri	17	66.22 hrs	855.00
D00095	Siti Mariam	14	47.60 hrs	760.00
Company Totals for Mega Transit				
Total drivers	:	7		
Total trips	:	117		
Total hours	:	431.96 hrs		
Total distance	:	6,770.00 km		
Workload Imbalance	:	Max = 22, Min = 11, Avg = 16.71		
Overload Drivers (>120% of avg trips):				
D00038	- Farah Hanim	(22 trips)		
D00058	- Hana Sofea	(21 trips)		
Underload Drivers (<50% of avg trips):				
-				
Driver workload by company: Unity Express				
Driver ID	Driver Name	Total Trips	Total Hours	Total Distance (km)
D00018	Amirul Danish	19	56.77 hrs	1,000.00
D00023	Aqilah Nabila	20	73.22 hrs	1,045.00
D00042	Faiz Amir	16	63.43 hrs	830.00
D00041	Faiz Rahman	17	56.28 hrs	870.00
D00035	Farah Liyana	11	38.07 hrs	605.00
D00040	Farah Zulaikha	13	59.18 hrs	725.00
D00051	Hafizul Hakim	25	91.22 hrs	1,235.00
D00057	Hana Nabila	16	67.05 hrs	785.00
D00088	Murul Balqis	16	63.62 hrs	795.00
D00099	Syed Irfan	21	85.50 hrs	1,240.00
Company Totals for Unity Express				
Total drivers	:	10		
Total trips	:	174		
Total hours	:	654.34 hrs		
Total distance	:	9,130.00 km		
Workload Imbalance	:	Max = 25, Min = 11, Avg = 17.4		
Overload Drivers (>120% of avg trips):				
D00051	- Hafizul Hakim	(25 trips)		
D00099	- Syed Irfan	(21 trips)		
Underload Drivers (<50% of avg trips):				
-				
Workload Summary (All Companies)				
Driver with Maximum Trips	:	Hafizul Hakim (25 trips) - Unity Express		
Driver with Minimum Trips	:	Ahmad Iqbal (11 trips) - Mega Transit		
Average Trips per Driver	:	17.12		
Total Hours (All Drivers)	:	1,086.30 hrs		
Total Distance (All Drivers)	:	15,900.00 km		
Generated on: 07-SEP-2025 16:56:04				
=====END OF REPORT=====				
PL/SQL procedure successfully completed.				

4.4.8 Report 2: Revenue Earned vs Lost per Company**Purpose:**

This procedure generates a financial report that compares revenue earned versus revenue lost for each bus company, broken down by bus type. It allows filtering by company, month and year to produce customized reporting periods. The output displays earned revenue from completed bookings, lost revenue from cancellations, net revenue and the cancellation rate percentage. Additionally, the procedure provides grand totals across all companies and bus types, offering management a clear overview of financial performance and booking trends.

Report Code:

```

SET SERVEROUTPUT ON
SET LINESIZE 300
SET PAGESIZE 200

-- Booking: functional index for year+month filtering
DROP INDEX idx_booking_yrmon;
CREATE INDEX idx_booking_yrmon
  ON Booking (TO_CHAR(PurchaseDate,'YYYYMM'));

CREATE OR REPLACE PROCEDURE prc_revenue_by_company_bus (
  p_companyID IN BusCompany.CompanyID%TYPE DEFAULT NULL,
  p_month      IN NUMBER                      DEFAULT NULL,
  p_year       IN NUMBER                      DEFAULT NULL
) IS
  v_totalEarned NUMBER(15,2) := 0;
  v_totalLost   NUMBER(15,2) := 0;

  v_month_name VARCHAR2(20);
  v_year_name  VARCHAR2(10);

  v_net NUMBER(15,2);
  v_cancelPct NUMBER(5,2);

  -- Outer cursor: get companies
  CURSOR c_company IS
    SELECT DISTINCT bc.CompanyID, bc.CompanyName
    FROM BusCompany bc
    JOIN Bus b ON bc.CompanyID = b.CompanyID
    JOIN BusSchedule s ON b.PlateNumber = s.PlateNumber
    JOIN Ticket t ON s.BusScheduleID = t.BusScheduleID
    JOIN BookingDetails bd ON t.TicketID = bd.TicketID
    JOIN Booking bk ON bd.BookingID = bk.BookingID
    WHERE (p_companyID IS NULL OR bc.CompanyID = p_companyID)
    AND (p_month IS NULL OR EXTRACT(MONTH FROM
bk.PurchaseDate) = p_month)
    AND (p_year IS NULL OR EXTRACT(YEAR FROM
bk.PurchaseDate) = p_year);

  -- Inner cursor: revenues by bus type for a given company
  CURSOR c_bus(p_cid BusCompany.CompanyID%TYPE) IS
    SELECT b.BusType,
      SUM(CASE WHEN bk.BookingStatus='Completed' THEN
bk.TotalPrice ELSE 0 END) AS EarnedRevenue,
      SUM(CASE WHEN bk.BookingStatus='Cancelled' THEN
bk.TotalPrice ELSE 0 END) AS LostRevenue
    FROM BusCompany bc
    JOIN Bus b ON bc.CompanyID = b.CompanyID
    JOIN BusSchedule s ON b.PlateNumber = s.PlateNumber
    JOIN Ticket t ON s.BusScheduleID = t.BusScheduleID
    JOIN BookingDetails bd ON t.TicketID = bd.TicketID
    JOIN Booking bk ON bd.BookingID = bk.BookingID
    WHERE bc.CompanyID = p_cid
    AND (p_month IS NULL OR EXTRACT(MONTH FROM
bk.PurchaseDate) = p_month)
    AND (p_year IS NULL OR EXTRACT(YEAR FROM
bk.PurchaseDate) = p_year)
    GROUP BY b.BusType
    HAVING SUM(CASE WHEN bk.BookingStatus IN
('Completed','Cancelled') THEN bk.TotalPrice ELSE 0 END) > 0
    ORDER BY b.BusType;

  v_company c_company%ROWTYPE;
  v_bus      c_bus%ROWTYPE;

BEGIN

```

```

-- Format period
IF p_month IS NOT NULL THEN
    v_month_name :=
INITCAP(TO_CHAR(TO_DATE(p_month,'MM'),'Month'));
ELSE
    v_month_name := 'All';
END IF;

IF p_year IS NOT NULL THEN
    v_year_name := TO_CHAR(p_year);
ELSE
    v_year_name := 'All';
END IF;

-- Report header
DBMS_OUTPUT.PUT_LINE(LPAD('=', 120, '='));
DBMS_OUTPUT.PUT_LINE('Revenue Earned vs Lost per Company');
DBMS_OUTPUT.PUT_LINE('Period: ' || v_month_name || ' '
|| v_year_name);
DBMS_OUTPUT.PUT_LINE('Generated on: ' ||
TO_CHAR(SYSDATE,'DD-MON-YYYY HH24:MI:SS'));
DBMS_OUTPUT.PUT_LINE(LPAD('=', 120, '='));

DBMS_OUTPUT.PUT_LINE(
    RPAD('Company',25) ||
    RPAD('Bus Type',20) ||
    RPAD('Earned Revenue',20) ||
    RPAD('Lost Revenue',20) ||
    RPAD('Net Revenue',20) ||
    RPAD('Cancel Rate (%)',15)
);
DBMS_OUTPUT.PUT_LINE(LPAD('-', 120, '-'));

-- Loop companies
OPEN c_company;
LOOP
    FETCH c_company INTO v_company;
    EXIT WHEN c_company%NOTFOUND;

    -- Loop bus types for each company
    OPEN c_bus(v_company.CompanyID);
    LOOP
        FETCH c_bus INTO v_bus;
        EXIT WHEN c_bus%NOTFOUND;

        v_net := v_bus.EarnedRevenue - v_bus.LostRevenue;

        IF (v_bus.EarnedRevenue + v_bus.LostRevenue) > 0 THEN
            v_cancelPct := (v_bus.LostRevenue /
(v_bus.EarnedRevenue + v_bus.LostRevenue)) * 100;
        ELSE
            v_cancelPct := 0;
        END IF;

        v_totalEarned := v_totalEarned + v_bus.EarnedRevenue;
        v_totalLost := v_totalLost + v_bus.LostRevenue;

        DBMS_OUTPUT.PUT_LINE(
            RPAD(v_company.CompanyName,25) ||
            RPAD(v_bus.BusType,20) ||
            RPAD(TO_CHAR(v_bus.EarnedRevenue,'FM999,999,999.00'),20) ||
            RPAD(TO_CHAR(v_bus.LostRevenue,'FM999,999,999.00'),20) ||
            RPAD(TO_CHAR(v_net,'FM999,999,999.00'),20) ||
            RPAD(TO_CHAR(v_cancelPct,'990.00') || '%',15)
        );
    END LOOP;
END LOOP;

```

```

        END LOOP;
        CLOSE c_bus;
    END LOOP;
    CLOSE c_company;

    -- Grand total
    DBMS_OUTPUT.PUT_LINE(LPAD('-', 120, '-'));
    DBMS_OUTPUT.PUT_LINE(
        RPAD('GRAND TOTAL',40) ||
        RPAD(TO_CHAR(v_totalEarned,'FM999,999,999.00'),20) ||
        RPAD(TO_CHAR(v_totalLost,'FM999,999,999.00'),20) ||
        RPAD(TO_CHAR(v_totalEarned -
v_totalLost,'FM999,999,999.00'),20) ||
        RPAD(
            CASE WHEN (v_totalEarned + v_totalLost) > 0
            THEN
TO_CHAR((v_totalLost/(v_totalEarned+v_totalLost))*100,'990.00') ||
'%'
            ELSE '0.00%'
            END,15)
    );

    DBMS_OUTPUT.PUT_LINE(LPAD('=',44,'=') || RPAD('END OF REPORT',
76, '='));
END;
/

EXEC prc_revenue_by_company_bus;
EXEC prc_revenue_by_company_bus(NULL, '7', 2025);

```

Sample Output:

=====					
Revenue Earned vs Lost per Company and Bus Type					
Period: All All					
Generated on: 07-SEP-2025 17:54:08					
=====					
Company	Bus Type	Earned Revenue	Lost Revenue	Net Revenue	Cancel Rate (%)
BlueSky Coaches	Single Deck	11,324.00	28.50	11,295.50	0.25%
Citylink Travels	Double Decker	13,387.30	.00	13,387.30	0.00%
EcoRide Coaches	Single Deck	13,800.00	97.60	13,702.40	0.70%
Galaxy Busline	Mini Bus	13,159.96	.00	13,159.96	0.00%
Golden Way Travels	Luxury Coach	13,568.51	120.00	13,448.51	0.88%
Greenway Transport	Mini Bus	11,004.70	119.97	10,884.73	1.08%
Mega Transit	Mini Bus	13,548.80	66.00	13,482.80	0.48%
Metro Bus Services	Double Decker	22,261.70	1,721.00	20,540.70	7.18%
NextBus Travel	Double Decker	13,464.00	168.80	13,295.20	1.24%
Orbit Express	Luxury Coach	14,337.78	66.60	14,271.18	0.46%
Primeline Travels	Mini Bus	14,337.99	80.00	14,257.99	0.55%
Rapid Travels	Luxury Coach	11,480.50	270.00	11,210.50	2.30%
Rapidlink Express	Mini Bus	13,319.97	59.00	13,260.97	0.44%
Royal Express	Double Decker	13,580.00	.00	13,580.00	0.00%
Silverline Transport	Double Decker	12,430.00	85.00	12,345.00	0.68%
Speedway Coaches	Single Deck	12,807.25	90.00	12,717.25	0.70%
Starline Express	Single Deck	17,059.10	1,001.00	16,058.10	5.54%
Sunrise Express	Luxury Coach	12,925.00	180.00	12,745.00	1.37%
TransCity Coaches	Single Deck	14,175.00	99.80	14,075.20	0.70%
Unity Express	Luxury Coach	16,840.00	.00	16,840.00	0.00%
GRAND TOTAL		278,811.56	4,253.27	274,558.29	1.50%
=====END OF REPORT=====					
=====					
Revenue Earned vs Lost per Company and Bus Type					
Period: July 2025					
Generated on: 07-SEP-2025 17:55:03					
=====					
Company	Bus Type	Earned Revenue	Lost Revenue	Net Revenue	Cancel Rate (%)
Greenway Transport	Mini Bus	1,980.00	.00	1,980.00	0.00%
Mega Transit	Mini Bus	220.00	.00	220.00	0.00%
Royal Express	Double Decker	1,375.00	.00	1,375.00	0.00%
GRAND TOTAL		3,575.00	.00	3,575.00	0.00%
=====END OF REPORT=====					

Chapter 5 Extra Effort Highlights

5.1 Chan Jin Wei

5.1.1 View

5.1.1.1 View Used in Query 1

Purpose:

The purpose of this view is to provide a consolidated schedule of all bus trips by combining details from the BusCompany, Bus and BusSchedule tables. It presents key information such as company name, bus type, plate number, departure and arrival times, destination, platform, seat availability and status. By filtering only future departure dates, the view supports operational monitoring, reporting and customer-facing applications by showing upcoming active bus schedules in a single, easy-to-query dataset.

```
CREATE OR REPLACE VIEW BusScheduleView AS
SELECT
    bc.CompanyName          AS Company_Name,
    b.BusType               AS Bus_Type,      -- first column for break
    b.PlateNumber           AS Bus_Plate,     -- second column
    bs.BusScheduleID        AS ScheduleID,
    TO_CHAR(bs.DepartureDate, 'DD-MM-YYYY') AS Departure_Date,
    TO_CHAR(bs.DepartureTime, 'HH24:MI')    AS Depart_Time,
    TO_CHAR(bs.ArrivalTime, 'HH24:MI')      AS Arr_Time,
    bs.DepartBusStation     AS Departure_Station,
    bs.ArrivalBusStation    AS Arrival_Station,
    bs.Destination          AS Destination,
    bs.PlatformNo           AS Platform,
    bs.AvailableSeat         AS Seats_Available,
    bs.ScheduleStatus       AS Status
FROM   BusCompany bc
JOIN   Bus b
      ON bc.CompanyID = b.CompanyID
JOIN   BusSchedule bs
      ON b.PlateNumber = bs.PlateNumber
WHERE  bs.DepartureDate > TRUNC(SYSDATE);
```

5.1.1.2 View Used in Query 2

Purpose:

The purpose of this view is to consolidate bus service and maintenance records by integrating data from BusCompany, Bus, ServiceDetails and Service. It provides essential information such as service type, dates, cost and status for each bus, grouped by company and plate number. By restricting results to services that are *In Progress* or *Completed*, the view supports monitoring, reporting and analysis of operational and maintenance activities, ensuring better oversight of fleet performance and costs.

```
CREATE OR REPLACE VIEW BusServiceRecordsView AS
SELECT
    bc.CompanyName          AS Company_Name,
    b.BusType               AS Bus_Type,
    b.PlateNumber           AS Bus_Plate,
    sd.ServiceTransactionID AS ServiceID,
    s.ServiceType            AS Service_Type,
    TO_CHAR(sd.ServiceDate, 'DD-MM-YYYY') AS Service_Date,
    TO_CHAR(sd.ServiceCompleteDate, 'DD-MM-YYYY') AS
Service_Complete_Date,
```

```
        s.ServiceCost          AS Service_Cost,  
        sd.ServiceStatus      AS Status  
FROM    BusCompany bc  
JOIN    Bus b  
        ON bc.CompanyID = b.CompanyID  
JOIN    ServiceDetails sd  
        ON b.PlateNumber = sd.PlateNumber  
JOIN    Service s  
        ON sd.ServiceID = s.ServiceID  
WHERE   sd.ServiceStatus IN ('In Progress', 'Completed');
```

5.1.2 Indexes

5.1.2.1 Index for CompanyID in Bus Table (Used in Report 1)

Purpose:

The purpose of this index is to improve the efficiency of joins between the Bus and BusCompany tables. In the yearly revenue trend report, the procedure frequently groups revenue by company, which requires matching each bus to its company through the CompanyID field. By indexing this column, the database avoids costly full table scans and retrieves company-related data more quickly. This leads to faster report generation and better scalability as booking and company data grow.

```
CREATE INDEX idx_bus_company ON  
Bus (CompanyID);
```

5.1.2.2 Index for PlateNumber in Bus Table (Used in Report 2)

Purpose:

The purpose of this index is to optimize the join between Bus and BusSchedule using the PlateNumber column. In the prc_bus_profitability report, revenue is derived from tickets booked through bus schedules, making this join one of the most frequently executed operations. By indexing PlateNumber in BusSchedule, the database can quickly locate matching schedules for each bus without scanning the entire table. This improves the performance of profitability calculations and ensures the report runs efficiently as schedule data grows.

```
CREATE INDEX idx_schedule_plate  
ON BusSchedule (PlateNumber);
```

5.1.3 User Defined Functions

5.1.3.1 Function Used in Trigger 1

Purpose:

The purpose of this function is to determine whether a member has enough points to redeem a specific gift. It retrieves the required points for the gift and compares them with the member's current balance. The function returns 'Y' if the points are sufficient and 'N' otherwise, allowing triggers, procedures or applications to validate redemptions consistently.

```
CREATE OR REPLACE FUNCTION fn_check_points_enough (  
    p_memberID IN VARCHAR2,
```

```

        p_giftID    IN VARCHAR2
    ) RETURN VARCHAR2
IS
    v_required NUMBER := 0;
    v_balance  NUMBER := 0;
BEGIN
    -- Get required points for gift
    SELECT PointRequired
    INTO v_required
    FROM Gift
    WHERE GiftID = p_giftID;

    -- Get member points balance
    SELECT MemberPoints
    INTO v_balance
    FROM Member
    WHERE MemberID = p_memberID;

    -- Compare
    IF v_balance >= v_required THEN
        RETURN 'Y'; -- Enough points
    ELSE
        RETURN 'N'; -- Not enough
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'N'; -- If gift or member not found
    WHEN OTHERS THEN
        RETURN 'N'; -- On error, treat as not enough
END;
/

```

5.1.4 User Defined Exceptions

5.1.4.1 RAISE_APPLICATION_ERROR in Trigger 1

Purpose:

The purpose of this exception is to check on gift stock is to prevent redemptions from proceeding when a gift has no remaining stock. This validation ensures that inventory cannot go negative and members cannot redeem unavailable gifts. By raising an explicit error, it preserves data integrity, avoids over-redemption and provides a clear message to the system or user that the gift is out of stock.

```

IF v_stock <= 0 THEN
    RAISE_APPLICATION_ERROR(-20002,
        'Gift ' || :NEW.GiftID || ' is out of stock.');
```

```

SQL> UPDATE Gift SET StockRemaining = 0 WHERE GiftID = 'GF0002';

1 row updated.

SQL> INSERT INTO PointRedemption (MemberID, GiftID, RedemptionDate, RedemptionStatus, UsedDate)
  2 VALUES ('M00100', 'GF0002', SYSDATE, 'Collected', SYSDATE);
INSERT INTO PointRedemption (MemberID, GiftID, RedemptionDate, RedemptionStatus, UsedDate)
      *
ERROR at line 1:
ORA-20002: Gift GF0002 is out of stock.
ORA-06512: at "STUDENT1.TRG_MEMBER_POINTS_REDEMPTION", line 17
ORA-04088: error during execution of trigger 'STUDENT1.TRG_MEMBER_POINTS_REDEMPTION'

```

5.1.4.2 Default Exception in Procedure 2

Purpose:

The purpose of the NO_DATA_FOUND exception in this procedure is to handle cases where the provided StaffID or ServiceID does not exist in their respective tables. Instead of allowing the procedure to fail with an unhandled error, it captures the exception and outputs a clear message indicating that the staff or service record was not found. This improves user feedback, ensures graceful handling of invalid inputs and prevents incomplete or invalid allocations from being inserted into the StaffAllocation table.

```
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Staff ID ' || p_staffID || ' or Service ID '
|| p_serviceID || ' not found.');
```

```
SQL> EXEC prc_allocate_work_to_staff('ST0001', 'SV0005', 'Morning');
Staff ID ST0001 or Service ID SV0005 not found.
```

5.1.5 Sequence

5.1.5.1 Sequence Used in Procedure 1

Purpose:

The purpose of the payment_seq sequence is to automatically generate unique, incremental identifiers for the PaymentID in the Payment table. Each time a new payment record is inserted, the sequence ensures that a distinct numeric value is assigned and formatted as part of the PaymentID (e.g., P00669). This eliminates manual tracking of IDs, prevents duplication and guarantees consistency in primary key generation across payment transactions.

```
DROP SEQUENCE payment_seq;
CREATE SEQUENCE payment_seq
MINVALUE 669
MAXVALUE 99999
START WITH 669
INCREMENT BY 1
NOCACHE;
```

5.2 Ethel Ng Yi Yan

5.2.1 View

5.2.1.1 View Used in Query 1

Purpose:

The purpose of this view is to provide a consolidated and user-friendly way to see all booking-related information in one place. It combines data from multiple tables, such as the Booking table, BookingDetails table, Ticket table and BusSchedule table. Therefore, users can easily access details such as booking ID, passenger name, seat number, ticket type, price and travel schedule without writing complex joins. This view simplifies reporting, supports operational queries and improves accessibility for analyzing passenger and booking details.

```
CREATE OR REPLACE VIEW Bus_Booking_Info AS
SELECT
    b.BookingID,
    bs.DepartureDate,
    TO_CHAR(bs.DepartureTime, 'HH:MI AM') AS DepartureTime,
    bs.DepartBusStation,
    bs.ArrivalBusStation,
    bd.PassengerName,
    t.SeatNo,
    t.TicketType,
    t.TicketPrice
FROM
    Booking b
JOIN BookingDetails bd ON b.BookingID = bd.BookingID
JOIN Ticket t ON bd.TicketID = t.TicketID
JOIN BusSchedule bs ON t.BusScheduleID = bs.BusScheduleID
ORDER BY b.BookingID;
```

5.2.1.2 View Used in Query 2

Purpose:

The purpose of this view is to give a clear overview of how drivers are assigned and allocated to bus schedules. It brings together information from the Driver table, DriverList table, BusSchedule table and Bus tables so users can see which driver is allocated to which trip, their role and the corresponding departure details. This simplifies monitoring driver assignments, ensures proper workforce planning and helps management track driver responsibilities across different schedules.

```
CREATE OR REPLACE VIEW Driver_Allocation AS
SELECT
    d.DriverID,
    d.DriverName,
    dl.BusScheduleID,
    dl.DriverRole,
    bs.DepartureDate,
    TO_CHAR(bs.DepartureTime, 'HH24:MI') AS DepartureTime,
    bs.DepartBusStation
FROM DriverList dl
JOIN Driver d ON dl.DriverID = d.DriverID
JOIN BusSchedule bs ON dl.BusScheduleID = bs.BusScheduleID
JOIN Bus b ON bs.PlateNumber = b.PlateNumber
ORDER BY d.DriverID, bs.DepartureDate;
```

5.2.2 Indexes

5.2.2.1 Index for BusScheduleID, TicketStatus in Ticket Table (Used in Report 1)

Purpose:

The purpose of this index is to optimize performance when retrieving tickets for a specific bus schedule while filtering by their status. Without this index, it needs to scan the entire Ticket table, which can become very costly as the table grows large. This is especially important in reports like bus utilization, where the system frequently counts how many tickets are booked for each schedule. The index ensures these queries are efficient, scalable and responsive even when handling thousands of tickets across many schedules.

```
CREATE INDEX idx_ticket_schedule_status ON Ticket (BusScheduleID, TicketStatus);
```

5.2.2.2 Index for DepartBusStation, Destination in BusSchedule Table (Used in Report 2)

Purpose:

The purpose of this index is to optimize query performance when retrieving bus schedules based on departure station and destination. This index is crucial because the procedure frequently queries bus schedules by departure station and destination when calculating profitability per route. Without using the index, full table scanning is needed on the BusSchedule table for every route processed. By indexing these two columns together, the database can quickly locate all schedules belonging to a given route, reducing query cost, improving join efficiency and speeding up the profitability report significantly.

```
CREATE INDEX idx_busschedule_route ON BusSchedule (DepartBusStation, Destination);
```

5.2.4 User Defined Exceptions

5.2.4.1 RAISE_APPLICATION_ERROR in Trigger 1

Purpose:

The purpose of this exception is to enforce a business rule that refund requests must be made at least 48 hours before the scheduled departure. If the request is submitted later than this window, the trigger raises this error to prevent the refund from being processed and to ensure adherence to the company's refund policy.

```
RAISE_APPLICATION_ERROR(-20001, 'Refund must be requested at least 48 hours before departure. Refund has been rejected.');
```

```
SQL> UPDATE BookingDetails
  2   SET RefundID = 'R20000'
  3   WHERE BookingID = 'B10000'
  4   AND TicketID = 'T10000';
UPDATE BookingDetails
*
```

```
ERROR at line 1:
ORA-20001: Refund must be requested at least 48 hours before departure. Refund has been rejected.
ORA-06512: at "ADM.TRG_NEWREFUND", line 30
ORA-04088: error during execution of trigger 'ADM.TRG_NEWREFUND'
```

5.2.4.2 DEFAULT EXCEPTION in Procedure 1

Purpose:

The purpose of this exception is to handle situations where no bus schedule matches the user's search filters. If the query does not return any rows, the trigger raises a `NO_DATA_FOUND` exception. This ensures that instead of failing silently, the system provides a clear message to the user indicating that no results were found for the specified filters (bus plate, company, destination, date or time).

```

IF NOT v_found THEN
    RAISE NO_DATA_FOUND;
END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No Bus Schedule found for filters: ' ||
                               NVL(p_busPlate, 'Any Bus') || ', ' ||
                               NVL(p_company, 'Any Company') || ', ' ||
                               NVL(p_destination, 'Any Destination') || ', '
                               ||
                               NVL(TO_CHAR(p_departDate, 'DD-MM-YYYY'), 'Any
Date') || ', ' ||
                               NVL(TO_CHAR(p_departTime, 'HH24:MI'), 'Any
Time'));
END;
/

SQL> EXEC prc_search_bus_schedule(p_busPlate => 'BU3100');
No Bus Schedule found for filters: BU3100, Any Company, Any Destination, Any Date, Any Time

PL/SQL procedure successfully completed.

```

5.2.5 Sequence

5.2.5.1 Sequence Used in Procedure 2 : Full Refund of Bus Schedule Cancellation

Purpose:

The sequence `refundid_seq` is used to generate a unique `refundID` whenever a new admin is added. In the full refund procedure, the sequence's `NEXTVAL` is fetched, which generates the next available number in the sequence to create new refund records.

```

DROP SEQUENCE refundid_seq;
CREATE SEQUENCE refundid_seq
MINVALUE 51
MAXVALUE 99999
START WITH 51
INCREMENT BY 1
NOCACHE;

--Use in Procedure 2
    v_RefundID := 'R' || LPAD(refundid_seq.NEXTVAL, 5, '0');

```

5.3 Gan Khai Li

5.3.1 View

5.3.1.1 View Used in Query 1

Purpose:

The purpose of this view is to present a summary of a member's travel history by identifying how many trips they have taken to various destinations. It achieves this by joining multiple related tables, including Booking, BookingDetails, Ticket, BusSchedule and Member to gather all necessary information about bookings and travel routes. The view organizes this data in a simplified format that highlights the total number of trips made by each member to each destination. This aggregation allows for easier analysis of customer travel behavior, helps identify the most popular destinations and supports business decisions related to marketing, customer engagement and personalized service offerings based on travel frequency.

```
CREATE OR REPLACE VIEW MostFrequentDestinationsView AS
SELECT
    m.MemberID,
    m.MemberName,
    bs.Destination,
    COUNT(bd.TicketID) AS TotalTrips
FROM Booking b
JOIN BookingDetails bd ON b.BookingID = bd.BookingID
JOIN Ticket t ON bd.TicketID = t.TicketID
JOIN BusSchedule bs ON t.BusScheduleID = bs.BusScheduleID
JOIN Member m ON b.MemberID = m.MemberID
GROUP BY m.MemberID, m.MemberName, bs.Destination;
```

5.3.1.2 View Used in Query 2

Purpose:

The purpose of this view is to provide a clear summary of member interactions with the five most frequently redeemed gifts. This view combines data from the PointRedemption, Gift and Member tables to show which members have redeemed these popular gifts, how many times they have done so and the most recent redemption date for each gift. By focusing only on the top five gifts based on redemption frequency, the view highlights member engagement with the most in-demand rewards. This information can be used to analyze member preferences, assess the popularity of specific gifts and support decision-making for loyalty program improvements, marketing strategies and inventory planning.

```
CREATE OR REPLACE VIEW Top5GiftMemberRecentView AS
SELECT
    g.GiftName,
    g.PointRequired,
    m.MemberID,
    m.MemberName,
    MAX(pr.RedemptionDate) AS LastRedemption,
    COUNT(*) AS RedemptionCount
FROM PointRedemption pr
JOIN Gift g ON pr.GiftID = g.GiftID
JOIN Member m ON pr.MemberID = m.MemberID
WHERE pr.GiftID IN (
    SELECT GiftID
    FROM (
        SELECT GiftID
        FROM PointRedemption
        GROUP BY GiftID
        ORDER BY COUNT(*) DESC
        FETCH FIRST 5 ROWS ONLY
    )
)
```

```
ORDER BY COUNT(*) DESC
) top5
WHERE ROWNUM <= 5
)
GROUP BY g.GiftID, g.GiftName, g.PointRequired, m.MemberID,
m.MemberName
ORDER BY g.GiftName, LastRedemption DESC;
```

5.3.2 Indexes

5.3.2.1 Index for Destination in BusSchedule Table (Used in Report 1)

Purpose:

The purpose of this index is to improve the performance of queries that filter or sort by destination, making it easier to efficiently handle operations involving bus schedules by their destination. By creating an index on the relevant column, the database engine can quickly locate and retrieve the necessary rows without having to scan the entire table, which is especially beneficial when dealing with large datasets. This enhances the efficiency of operations such as retrieving all schedules going to a specific destination, generating reports grouped by destination or executing join operations involving destination-related data. Overall, the index helps optimize query execution time, reduces system load and contributes to a smoother and more responsive user experience when accessing or analyzing transportation-related information.

```
CREATE INDEX idx_busschedule_destination ON
BusSchedule(Destination);
```

5.3.2.2 Index for RefundStatus, RefundReason in Refund Table (Used in Report 2)

Purpose:

The purpose of this index is to enhance the efficiency of queries that work with refund data, particularly those that filter, group or sort records based on refund status and refund reason. It allows the database to retrieve only the relevant rows more quickly by avoiding a full table scan, which can be time-consuming when dealing with a large volume of data. This improvement is especially noticeable in systems where refund records are frequently accessed for processing, reporting or customer support. By having an index on both columns, the database can better organize and access the data needed for these operations. The result is faster query execution, more responsive applications and a smoother experience when managing or analyzing refund-related information. This optimization plays a key role in maintaining performance and scalability as the dataset grows over time.

```
CREATE INDEX idx_refund_status_reason ON Refund(RefundStatus,
RefundReason);
```

5.3.3 User Defined Functions

5.3.3.1 Function Used in Procedure 2

Purpose:

The purpose of this function is to calculate a member's age in complete years by finding the difference between the current date and their date of birth, converting that difference from months to years and returning this value for use in the procedure. This allows the system to validate that the member meets the minimum age requirement of 12 years before registration.

```

CREATE OR REPLACE FUNCTION calculate_age(p_dob DATE)
RETURN NUMBER
IS
    v_age NUMBER;
BEGIN
    v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, p_dob) / 12);
    RETURN v_age;
END calculate_age;
/

```

5.3.4 User Defined Exceptions

5.3.4.1 PRAGMA EXCEPTION_INIT Used in Procedure 1

Purpose:

The purpose of these exceptions is to stop the procedure and show clear error messages when something is wrong. The e_BusNotExist exception happens if the bus with the given plate number is not found, so no maintenance record is added for a bus that doesn't exist. The e_InvalidCost exception happens if the service cost is less than RM 20, to make sure only valid costs are accepted. When either of these problems happens, the procedure shows a clear message explaining the issue, helping users understand why the maintenance record was not created and preventing bad or incorrect data from entering the system.

```

e_BusNotExist      EXCEPTION;
PRAGMA EXCEPTION_INIT(e_BusNotExist, -20001);

e_InvalidCost      EXCEPTION;
PRAGMA EXCEPTION_INIT(e_InvalidCost, -20003);

EXCEPTION
    WHEN e_BusNotExist THEN
        DBMS_OUTPUT.PUT_LINE('Error: Bus ' || p_plateNumber || '
does not exist. Maintenance record NOT created. ');

    WHEN e_InvalidCost THEN
        DBMS_OUTPUT.PUT_LINE('Error: Service cost must be at least
RM 20. Maintenance record NOT created. ');

```

```

SQL> EXEC prc_ManageBusMaintenance('BU0101', 'Repair', 500);
Error: Bus BU0101 does not exist. Maintenance record NOT created.

PL/SQL procedure successfully completed.

SQL> EXEC prc_ManageBusMaintenance('BU0100', 'Repair', 10);
Error: Service cost must be at least RM 20. Maintenance record NOT created.

PL/SQL procedure successfully completed.

```

5.3.4.2 RAISE_APPLICATION_ERROR Exception Used in Procedure 2

Purpose:

The purpose of these exceptions is to enforce data integrity and business rules within the application by validating user input and preventing invalid or duplicate data from being processed. Specifically, they ensure that a member is at least 12 years old, that the member's identification card (IC) number is unique to avoid duplicates and that the email address provided follows a valid format. By raising these custom application errors with clear messages, the system can halt processing and inform users of the exact issue, promoting accurate and reliable data entry.

```
RAISE_APPLICATION_ERROR(-20001, 'Member must be at least 12 years old.');
```

```
RAISE_APPLICATION_ERROR(-20002, 'Member IC already exists.');
```

```
RAISE_APPLICATION_ERROR(-20003, 'Invalid email format.');
```



```
SQL> EXEC register_member('Chong', TO_DATE('2015-01-01','YYYY-MM-DD'), 'Male', 'chong@gmail.com', '0123456789', '150101-01-0001');
BEGIN register_member('Chong', TO_DATE('2015-01-01','YYYY-MM-DD'), 'Male', 'chong@gmail.com', '0123456789', '150101-01-0001'); END;
```

```
*
ERROR at line 1:
ORA-20099: An unexpected error occurred: ORA-20001: Member must be at least 12 years old.
ORA-06512: at "GANKHAILI.REGISTER_MEMBER", line 78
ORA-06512: at line 1
```



```
SQL> EXEC register_member('Lau', TO_DATE('2005-03-20','YYYY-MM-DD'), 'Male', 'lau.email.com', '01122334455', '960606-06-6789');
BEGIN register_member('Lau', TO_DATE('2005-03-20','YYYY-MM-DD'), 'Male', 'lau.email.com', '01122334455', '960606-06-6789'); END;
```

```
*
ERROR at line 1:
ORA-20099: An unexpected error occurred: ORA-20003: Invalid email format.
ORA-06512: at "GANKHAILI.REGISTER_MEMBER", line 78
ORA-06512: at line 1
```



```
SQL> EXEC register_member('Alice Tan', TO_DATE('2005-03-20','YYYY-MM-DD'), 'Female', 'alice.tan@example.com', '01122334455', '950505-05-5679');
BEGIN register_member('Alice Tan', TO_DATE('2005-03-20','YYYY-MM-DD'), 'Female', 'alice.tan@example.com', '01122334455', '950505-05-5679'); END;
```

```
*
ERROR at line 1:
ORA-20099: An unexpected error occurred: ORA-20002: Member IC already exists.
ORA-06512: at "GANKHAILI.REGISTER_MEMBER", line 78
ORA-06512: at line 1
```

5.3.5 Sequence

5.3.5.1 ServiceID Sequence Used in Procedure 1

Purpose:

The purpose of this sequence is to generate unique, sequential numeric identifiers for service records, starting at 21 and increasing by 1 with each new value, while ensuring all service IDs remain within the range of 21 to 99,999. It is designed to prevent duplication by not cycling back once the maximum value is reached and uses NOCACHE to maintain immediate consistency and accuracy of generated IDs, providing a reliable way to uniquely identify each service record within the specified numeric boundaries.

```
CREATE SEQUENCE serviceid_seq
MINVALUE 21
MAXVALUE 99999
START WITH 21
INCREMENT BY 1
NOCACHE;
```

5.4 Kit Chin Jie Ying

5.4.1 View

5.4.1.1 View Used in Query 1

Purpose:

The purpose of this view is to consolidate refund-related information for members into a single, easily accessible dataset. It retrieves each member's details along with refund records, including refund date, reason, status and amount. By joining the Member, Booking, BookingDetails and Refund tables, the view provides a streamlined way to analyze and report on refund history without repeatedly writing complex joins.

```
CREATE OR REPLACE VIEW RefundHistory AS
SELECT
    m.MemberID,
    m.MemberName,
    r.RefundID,
    TO_CHAR(r.RefundDate, 'DD-MON-YYYY') AS RefundDate,
    r.RefundReason,
    r.RefundStatus,
    r.RefundAmount
FROM Member m
JOIN Booking b
    ON m.MemberID = b.MemberID
JOIN BookingDetails bd
    ON b.BookingID = bd.BookingID
JOIN Refund r
    ON bd.RefundID = r.RefundID;
```

5.4.1.2 View Used in Query 2

Purpose:

The purpose of this view is to identify and display the top 10 members who have purchased the most tickets. It ranks members based on the total number of tickets purchased, while also showing their total spending amount. By joining the Member, Booking, BookingDetails and Ticket tables, the view provides a concise summary of high-value customers, supporting analysis of purchasing behavior and loyalty.

```
CREATE OR REPLACE VIEW Top10MemberPurchasing AS
SELECT *
FROM (
    SELECT
        RANK() OVER (ORDER BY COUNT(*) DESC) AS RankNo,
        m.MemberID,
        m.MemberName,
        COUNT(bd.TicketID) AS TotalTickets,
        SUM(t.TicketPrice) AS TotalSpent
    FROM Member m
    JOIN Booking b
        ON m.MemberID = b.MemberID
    JOIN BookingDetails bd
        ON b.BookingID = bd.BookingID
    JOIN Ticket t
        ON bd.TicketID = t.TicketID
    GROUP BY m.MemberID, m.MemberName
    ORDER BY COUNT(bd.TicketID) DESC
)
WHERE RankNo <= 10;
```

5.4.2 Indexes

5.4.2.1 Index idx_bs_plate_dep on BusSchedule (Used in Report 1)

Purpose: This index is created on the BusSchedule table using the uppercased bus plate number and the departure date. Its purpose is to improve query performance for report generation (Report 1), particularly when filtering or searching schedules by plate number (case-insensitive) and departure date. By indexing these columns together, the database can locate matching rows more efficiently, reducing full table scans and speeding up reporting queries.

```
DROP INDEX idx_bs_plate_dep;
CREATE INDEX idx_bs_plate_dep
  ON BusSchedule (UPPER(PlateNumber), DepartureDate);
```

5.4.2.2 Index idx_booking_yrmon on Booking (Used in Report 2)

Purpose: This index is created on the Booking table using the formatted purchase date (YYYYMM). Its purpose is to speed up queries and reports that group, filter or analyze bookings by year and month. By indexing the transformed date value, the database can avoid repeated function-based evaluations during searches and aggregations, resulting in faster performance for monthly or yearly booking reports.

```
DROP INDEX idx_booking_yrmon;
CREATE INDEX idx_booking_yrmon
  ON Booking (TO_CHAR(PurchaseDate, 'YYYYMM'));
```

5.4.3 User Defined Functions

5.4.3.1 Function of Rental Fee Calculation

Purpose:

The purpose of this function is to calculate the total rental fee for a space based on its size and rental duration. It determines the number of months between the start and end dates, rounding up partial months and enforcing a minimum of one month. The function then multiplies the monthly rate per square foot by the space size and rental duration, returning the total rental amount to support consistent and automated fee calculations.

```
CREATE OR REPLACE FUNCTION calc_rental_fee(
  p_SquareFeet IN NUMBER,
  p_StartDate  IN DATE,
  p_EndDate    IN DATE
) RETURN NUMBER IS
  v_months  NUMBER;
  v_baseRate NUMBER := 10; -- RM10 per square foot per month
  (example)
  v_total   NUMBER;
BEGIN
  -- Calculate rental duration in whole months
  v_months := CEIL(MONTHS_BETWEEN(p_EndDate, p_StartDate));

  IF v_months < 1 THEN
    v_months := 1; -- at least 1 month
  END IF;

  -- Total rent = rate * size * months
```

```

v_total := v_baseRate * p_SquareFeet * v_months;

RETURN v_total;
END;
/

```

Output:

```

SQL> SELECT calc_rental_fee(200, DATE '2025-01-01', DATE '2025-02-01') FROM dual;
CALC_RENTAL_FEE(200,DATE'2025-01-01',DATE'2025-02-01')
-----
2000

SQL>
SQL> -- Case 2: Less than a month (should still count as 1 month)
SQL> SELECT calc_rental_fee(200, DATE '2025-01-01', DATE '2025-01-15') FROM dual;
CALC_RENTAL_FEE(200,DATE'2025-01-01',DATE'2025-01-15')
-----
2000

SQL>
SQL> -- Case 3: Multiple months
SQL> SELECT calc_rental_fee(200, DATE '2025-01-01', DATE '2025-06-01') FROM dual;
CALC_RENTAL_FEE(200,DATE'2025-01-01',DATE'2025-06-01')
-----
10000

```

5.4.4 User Defined Exceptions**5.4.4.1 RAISE_APPLICATION_ERROR****Purpose:**

The exception is used to raise a custom error when the specified bus schedule does not exist in the database or is not marked as scheduled. It stops further execution of the procedure or trigger and returns a meaningful error message to the user or application, helping to clearly indicate why the operation failed.

```

RAISE_APPLICATION_ERROR(-20001, 'Bus schedule not
found or not scheduled. ');

SQL> EXEC prc_generate_tickets('BADID');
** Unexpected error: ORA-20001: Bus schedule not found or not scheduled.

```

5.4.4.2 PRAGMA EXCEPTION_INIT**Purpose:**

The declaration of the ex_staff_not_found exception and its association with the custom error code -20001 allows the system to handle cases where staff information is missing in a clearer and more structured way. Instead of relying on numeric error codes, the named exception improves readability and maintainability of the code, making it easier for developers to identify and manage specific error scenarios related to staff records.

```
ex_staff_not_found EXCEPTION;  
PRAGMA EXCEPTION_INIT(ex_staff_not_found, -20001);
```

```
SQL> BEGIN  
2     prc_ManageShopRental(  
3         p_StaffID      => 'S10001',  
4         p_ShopName     => 'Test Shop 1',  
5         p_ShopType     => 'Retail',  
6         p_OwnerName   => 'Alice Tan',  
7         p_ShopLocation => 'Lot 101, Main Street',  
8         p_SquareFeet   => 1200,  
9         p_MonthYear    => TO_DATE('01-09-2025', 'DD-MM-YYYY'),  
10        p_StartDate    => TO_DATE('01-09-2025', 'DD-MM-YYYY'),  
11        p_EndDate      => TO_DATE('30-09-2025', 'DD-MM-YYYY'),  
12        p_PaymentID    => 'P00183'  
13    );  
14 END;  
15 /  
BEGIN  
*  
ERROR at line 1:  
ORA-20001: Staff S10001 not found.  
ORA-06512: at "TESTING.PRC_MANAGESHOPRENTAL", line 79  
ORA-06512: at line 2
```

5.4.5 Sequence

5.4.5.1 Shop Sequence Used in Procedure 2

Purpose:

This sequence is designed to automatically generate unique identifiers for shops, starting at 21 and incrementing by 1 for each new value. It ensures that every shop record can be assigned a distinct ID without manual input, supporting consistency and avoiding duplicate keys within the system.

Sequence Code:

```
DROP SEQUENCE shopid_seq;  
CREATE SEQUENCE shopid_seq  
MINVALUE 21  
MAXVALUE 99999  
START WITH 21  
INCREMENT BY 1  
NOCACHE;
```